

حقوق نشر و مالکیت فکری

COPYRIGHT & INTELLECTUAL PROPERTY NOTICE

عنوان اثر

جزوه کامل درس بازیابی اطلاعات

(Information Retrieval – University Textbook)

پدیدآورندگان

حمیدرضا نامجومنش – نویسنده، پژوهشگر و گردآورنده محتوا، طراح ساختار آموزشی، ویراستار علمی و ادبی

امیرحسین همتی – نویسنده، پژوهشگر و گردآورنده محتوا، طراح ساختار آموزشی، ویراستار علمی و ادبی

علی مجاهد – نویسنده، پژوهشگر و گردآورنده محتوا، طراح ساختار آموزشی، ویراستار علمی و ادبی

© حق نشر و پروانه بهره‌برداری

این اثر تحت پروانه Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International (CC BY-NC-ND 4.0) منتشر می‌شود.

✓ شما مجاز هستید:

- این اثر را به صورت غیرتجاری و بدون هیچ‌گونه تغییر، با ذکر کامل نام پدیدآورندگان باز نشر دهید.
- نسخه‌های دقیقاً مشابه را در محیط‌های آموزشی غیرانتفاعی به اشتراک بگذارید.

✗ اکیداً ممنوع است:

- هرگونه استفاده تجاری (فروش، چاپ به قصد سود، قرار دادن پشت دیوار پرداخت، و نظایر آن).
- تغییر، تلخیص، ترجمه، بازترکیب فصول یا تولید آثار مشتق شده بدون رضایت کتبی همه پدیدآورندگان.
- حذف یا مخدوش کردن اعلان‌های حق نشر و واترمارک‌های دیجیتال.

⚠ هرگونه نقض این شرایط، نقض صریح حقوق مؤلف در چارچوب قوانین ایران و کنوانسیون برن (Berne Convention) محسوب می‌شود و پیگرد قانونی به دنبال خواهد داشت.

📄 متن کامل حقوقی مجوز (انگلیسی) | خلاصه فارسی مجوز

مخزن رسمی و شناسه دیجیتال

GitHub Repository: github.com/hrnrxb/IR-Textbook

(کلیه فایل‌های اصلی، امضاها و دیجیتال و گواهی‌های زمانی در مخزن فوق نگهداری می‌شوند.)

اصالت‌سنجی و گواهی مالکیت

• تمامی نسخه‌های PDF این جزوه با واترمارک دیجیتال و فراداده کپی‌رایت محافظت شده‌اند.

• هش SHA-256 یکتای فایل‌ها در سند `HASHES.txt` مخزن ثبت شده است.

🔒 تنها نسخه‌ای که از کانال رسمی (گیت‌هاب فوق) دریافت شود، معتبر است.

بازیابی اطلاعات

دانشگاه آزاد شیراز - دانشکده مهندسی کامپیوتر

ترم دوم سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل پانزدهم: Support vector machines and machine learning on documents

استاد: دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی): حمید نامجو، امیرحسین همتی و علی مجاهد

فهرست مطالب

15.1 Support vector machines: The linearly separable case

15.2 Extensions to the SVM model

15.2.1 Soft margin classification

15.2.2 Multiclass SVMs

15.2.3 Nonlinear SVMs

15.2.4 Experimental results

15.3 Issues in the classification of text documents

15.3.1 Choosing what kind of classifier to use

15.3.2 Improving classifier performance

15.4 Machine learning methods in ad hoc information retrieval

15.4.1 A simple example of machine-learned scoring

15.4.2 Result ranking by machine learning

15.5 Chapter 15 Summary

15 ماشین‌های بردار پشتیبان: یافتن مرز امن در دنیای دیجیتال

🚫 چالش کنترل محتوا در شبکه‌های اجتماعی: چگونه سخنان نفرت‌پراکنی را از گفتگوهای عادی جدا کنیم؟

تصور کنید در تیم کنترل محتوای پلتفرمی مانند متا (فیسبوک و اینستاگرام) کار می‌کنید و باید سیستمی طراحی کنید که به طور خودکار سخنان نفرت‌پراکنی را از گفتگوهای عادی جدا کند. این یک چالش بزرگ در طبقه‌بندی متن است، زیرا مرز بین این دو دسته بسیار ظریف و پیچیده است.

یک طبقه‌بند ساده ممکن است بگوید "اگر پست حاوی کلمه X است، نفرت‌پراکنی است". این رویکرد ساده اما ناکارآمد است، زیرا افراد می‌توانند با استفاده از کلمات یا استعاره‌های مختلف، همین پیام را با نیت‌های متفاوت بیان کنند. این یک طبقه‌بند با بایاس بالا است.

اینجاست که **ماشین بردار پشتیبان (SVM)** وارد می‌شود. SVM به دنبال یک مرز تصمیم نیست، بلکه به دنبال **امن‌ترین مرز ممکن** است.

SVM یک طبقه‌بند خطی است که هدف آن یافتن یک مرز تصمیم (هایپرپلن) است که به حداکثر فاصله ممکن از تمام نقاط داده داشته باشد. این مفهوم به عنوان **حداکثر حاشیه** شناخته می‌شود.

🗺️ استعاره خیابان: ایجاد امن‌ترین مرز ممکن

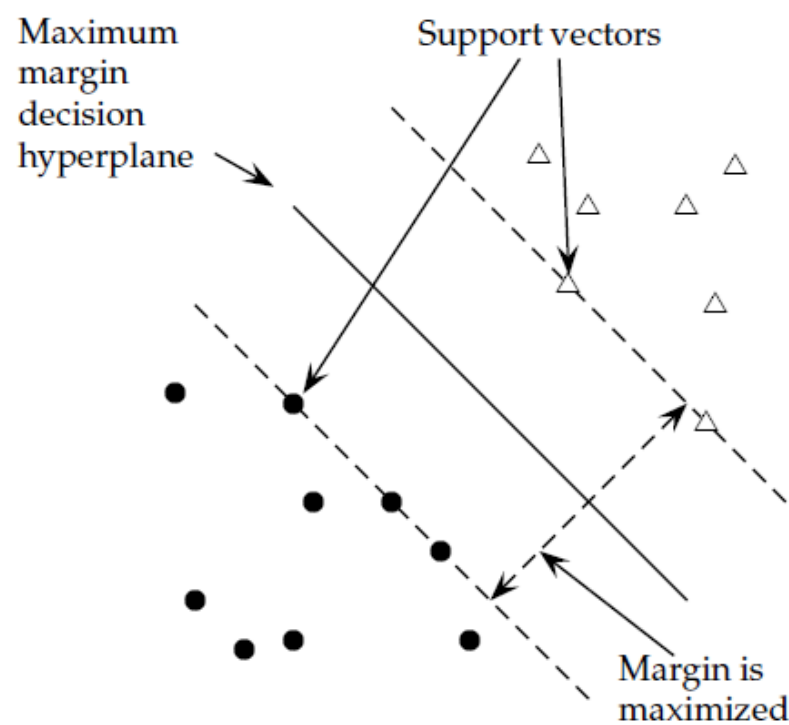
به جای تمرکز بر یافتن یک خط جداکننده، SVM مانند یک مهندس ترافیک عمل می‌کند که می‌خواهد **عریض‌ترین خیابان ممکن** را بین دو منطقه (کلاس‌ها) ایجاد کند. در این استعاره:

- **خیابان:** فضای بین دو کلاس که می‌خواهیم آن را به حداکثر برسانیم.
- **خط وسط خیابان:** مرز تصمیم (هایپرپلن) که دو کلاس را از هم جدا می‌کند.
- **لبه‌های خیابان:** نقاطی که به مرز تصمیم نزدیک‌تر هستند و تعیین‌کننده عرض خیابان هستند. این نقاط، **بردارهای پشتیبان** نام دارند.

با ایجاد عریض‌ترین خیابان ممکن، SVM یک طبقه‌بند بسیار مقاوم ایجاد می‌کند که کمتر تحت تأثیر داده‌های نویزی یا مرزی قرار می‌گیرد.

📐 شکل 15.1: بردارهای پشتیبان در مقابل حاشیه طبقه‌بند

شکل 15.1 این مفهوم را به صورت بصری نشان می‌دهد. نقاط سیاه، **بردارهای پشتیبان** هستند - آن‌ها نقاطی هستند که به مرز تصمیم (خط آبی) نزدیک‌تر هستند و تعیین‌کننده حاشیه طبقه‌بند هستند. خط آبی مرز تصمیمی است که SVM به عنوان بهینه پیدا می‌کند و حاشیه خاکستری، فاصله امن بین دو کلاس است.



► Figure 15.1 The support vectors are the 5 points right up against the margin of the classifier.

در مثال کنترل محتوا، بردارهای پشتیبان ممکن است پست‌هایی باشند که به مرز بین سخنان عادی و نفرت‌پرا بسیار نزدیک هستند - مانند استعاره‌های پیچیده یا کلماتی که در هر دو زمینه استفاده می‌شوند. این پست‌ها برای تعریف مرز امن حیاتی‌اند.

ریاضی اصلی SVM به صورت زیر است:

$$\text{minimize } \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{subject to } y_i(\mathbf{w}^T \mathbf{x}_i - b) \geq 1 \quad \text{for all } i$$

که در آن $\tilde{\mathbf{v}}(\mathbf{w})$ بردار نرمال وزن‌ها و b یک آستانه است. y_i برچسب سند i است (1 یا -1) و $\tilde{\mathbf{v}}(\mathbf{x})_i$ بردار سند i است. محدودیت $y_i(\tilde{\mathbf{v}}(\mathbf{w})^T \tilde{\mathbf{v}}(\mathbf{x})_i - b) \geq 1$ تضمین می‌کند که سند با حاشیه حداقل 1 به درستی طبقه‌بندی شود. هدف بهینه‌سازی، یافتن $\tilde{\mathbf{v}}(\mathbf{w})$ و b است که هم این محدودیت را برآورده کنند و همزمان اندازه $\tilde{\mathbf{v}}(\mathbf{w})$ را به حداقل برسانند.

🌀 مدیریت پیچیدگی: ترفند کرنل (Kernel Trick)

در دنیای واقعی، مرز بین سخنان عادی و نفرت‌پراکنی معمولاً یک خط ساده نیست. اینجاست که SVM با استفاده از ترفند کرنل قدرتمند می‌شود.

مثال واقعی: در سیستم کنترل محتوای یوتیوب، ممکن است کسی از طعنه استفاده کند تا پیام نفرت‌پراکنی کند. یک مرز خطی ساده نمی‌تواند این پیچیدگی را مدیریت کند. ترفند کرنل مانند این است که داده‌ها را به یک فضای با ابعاد بالاتر ببریم که در آن یک مرز خطی ساده می‌تواند آن‌ها را جدا کند، و سپس این مرز را به فضای اصلی بازگردانیم.

این ترفند به SVM اجازه می‌دهد تا مرزهای تصمیم غیرخطی پیچیده‌ای ایجاد کند که می‌توانند الگوهای زبانی پیچیده را مدیریت کنند. در طبقه‌بندی متن، کرنل تابع پایه شعاع (RBF) معمولاً کارآمدترین است.

SVM‌ها به دلایل متعددی در طبقه‌بندی متن موفق هستند:

- **کارایی بالا:** با ایجاد عریض‌ترین مرز ممکن، SVM‌ها معمولاً در برابر داده‌های جدید مقاوم‌تر عمل می‌کنند.
- **مقاومت در برابر ابعاد بالا:** در فضاهای با ابعاد بالا (مانند طبقه‌بندی متن با هزاران ویژگی)، احتمال جداپذیری خطی افزایش می‌یابد و SVM‌ها در این فضاها بسیار کارآمد هستند.

• **پایه تئوری قوی:** SVM ها بر اساس بهینه‌سازی محدب ساخته شده‌اند که تئوری ریاضی قوی‌ای برای آن‌ها فراهم می‌کند.

در فصل‌های بعدی، خواهیم دید که چگونه می‌توان SVM ها را برای مسائل غیرقابل تفکیک، مسائل چندکلاسی و کاربردهای عملی پیاده‌سازی کرد.

15.1 ماشین‌های بردار پشتیبان: حالت جدایپذیر خطی

📌 **فیلتر اسپم جیمیل: چگونه ایمیل‌های مهم از تبلیغات مزاحم جدا می‌شوند؟**

تصور کنید در تیم امنیتی گوگل کار می‌کنید و مسئول بهبود فیلتر اسپم جیمیل هستید. هر روز میلیاردها ایمیل ارسال می‌شود و باید سیستم شما بتواند به دقت ایمیل‌های اسپم را از ایمیل‌های مهم جدا کند. این یک چالش بزرگ در طبقه‌بندی متن است، زیرا مرز بین این دو دسته بسیار ظریف و پیچیده است.

یک طبقه‌بند ساده ممکن است بگوید "اگر ایمیل حاوی کلمه «خرید» است، اسپم است". این رویکرد ساده اما ناکارآمد است، زیرا ایمیل‌های معتبر زیادی نیز ممکن است حاوی این کلمه باشند (مانند ایمیل تأیید خرید از یک فروشگاه آنلاین).

اینجاست که **ماشین بردار پشتیبان (SVM)** وارد می‌شود. SVM به دنبال یک مرز تصمیم نیست، بلکه به دنبال **امن‌ترین مرز ممکن** است.

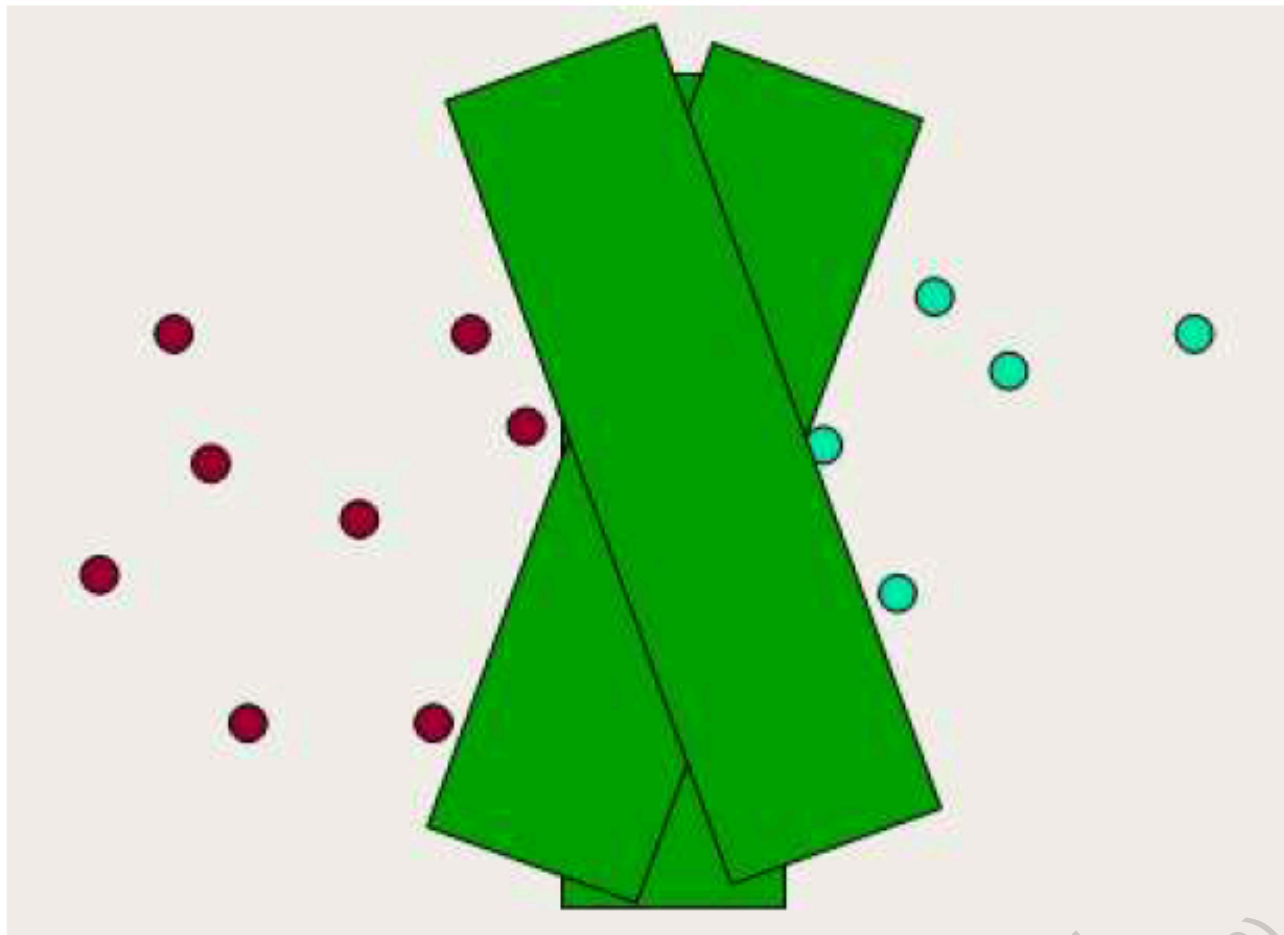
برای مجموعه داده‌های آموزشی دو کلاسه که به صورت خطی جدایپذیر هستند، مانند ایمیل‌های اسپم و غیراسپم، بسیاری از جداکننده‌های خطی ممکن وجود دارد. به صورت شهودی، یک مرز تصمیم‌گیری که در وسط فضای خالی بین داده‌های دو کلاس قرار گرفته، بهتر از جداکننده‌ای است که به نمونه‌های یک یا هر دو کلاس بسیار نزدیک است.

در مثال فیلتر اسپم، بردارهای پشتیبان ممکن است ایمیل‌هایی باشند که به مرز بین اسپم و غیراسپم بسیار نزدیک هستند - مانند ایمیل‌های تبلیغاتی با محتوای مفید یا ایمیل‌های مهم با ساختاری شبیه به اسپم. این ایمیل‌ها برای تعریف مرز امن حیاتی‌اند.

به حداکثر رساندن حاشیه به نظر خوب می‌رسد زیرا نقاط نزدیک به سطح تصمیم‌گیری تصمیمات طبقه‌بندی بسیار نامطمئن را نشان می‌دهند: تقریباً 50٪ شانس وجود دارد که طبقه‌بند به هر دو صورت تصمیم بگیرد. یک طبقه‌بند با حاشیه بزرگ هیچ تصمیم طبقه‌بندی با قطعیت پایین ایجاد نمی‌کند. این به شما یک حاشیه ایمنی طبقه‌بندی می‌دهد: یک خطای جزئی در اندازه‌گیری یا یک تغییر جزئی در سند باعث طبقه‌بندی اشتباه نخواهد شد.

📌 شکل 15.2: شهود برای طبقه‌بندی حاشیه بزرگ

شکل 15.2 نشان می‌دهد که چگونه اصرار بر حاشیه بزرگ، ظرفیت مدل را کاهش می‌دهد: محدوده زوایایی که می‌توان سطح تصمیم ضخیم را قرار داد، برای یک هایپرپلن تصمیم‌گیری کمتر است.



► Figure 15.2 An intuition for large-margin classification. Insisting on a large margin reduces the capacity of the model: the range of angles at which the fat decision surface can be placed is smaller than for a decision hyperplane (cf. Figure 14.8, page 301).

در مثال فیلتر اسپم، این به معنای ایجاد یک مدل است که کمتر تحت تأثیر ایمیل‌های مرزی قرار می‌گیرد و در نتیجه، در تشخیص ایمیل‌های جدید مطمئن‌تر عمل می‌کند.

بیایید یک SVM را با جبر رسمی کنیم. یک ابرصفحه تصمیم‌گیری را می‌توان با یک جمله تقاطع b و یک بردار نرمال ابرصفحه تصمیم‌گیری $\tilde{w}$ تعریف کرد که عمود بر ابرصفحه است. این بردار معمولاً در ادبیات یادگیری ماشین به عنوان **بردار وزن** (weight vector) شناخته می‌شود. برای انتخاب بین تمام ابرصفحه‌هایی که عمود بر بردار نرمال هستند، جمله تقاطع b را مشخص می‌کنیم.

فرض کنید مجموعه‌ای از نقاط داده آموزشی $D = \{(\tilde{x}_i, y_i)\}$ داریم، که هر عضو یک جفت از یک نقطه $\tilde{x}_i$ و یک برچسب کلاس y_i متناظر با آن است. برای SVMها، دو کلاس داده همیشه $+1$ و -1 نامیده می‌شوند (به جای 0 و 1)، و جمله تقاطع همیشه به صراحت به عنوان b نمایش داده می‌شود. طبقه‌بند خطی سپس به صورت زیر است:

$$f(\tilde{x}) = \text{sign}(\tilde{w}^T \tilde{x} + b)$$

مقدار -1 یک کلاس (مثلاً ایمیل اسپم) و مقدار $+1$ کلاس دیگر (مثلاً ایمیل غیراسپم) را نشان می‌دهد. ما از طبقه‌بندی یک نقطه مطمئن هستیم اگر از مرز تصمیم‌گیری دور باشد.

حاشیه تابعی در مقابل حاشیه هندسی

برای یک مجموعه داده و ابرصفحه تصمیم‌گیری، **حاشیه تابعی** (functional margin) مثال نام $\tilde{x}_i$ با توجه به ابرصفحه $\{\tilde{w}, b\}$ به صورت مقدار $y_i(\tilde{w}^T \tilde{x}_i + b)$ تعریف می‌شود. حاشیه تابعی یک مجموعه داده با توجه به یک سطح تصمیم‌گیری دو برابر حاشیه تابعی هر یک از نقاط در مجموعه داده با حداقل حاشیه تابعی است.

با این حال، استفاده از این تعریف به همین شکل مشکلی دارد: مقدار تحت محدودیت نیست، زیرا همیشه می‌توانیم حاشیه تابعی را با صرفاً مقیاس‌بندی $\tilde{w}$ و b به اندازه دلخواه بزرگ کنیم. برای مثال، اگر $\tilde{w}$ را با $5\tilde{w}$ و b را با $5b$

جایگزین کنیم، حاشیه تابعی $y_i(5w_{\sim xi} + 5b)$ پنج برابر بزرگتر است. این نشان می‌دهد که باید محدودیتی در اندازه بردار $w_{\sim}$ قرار دهیم.

اینجاست که **حاشیه هندسی** (geometric margin) وارد می‌شود. حاشیه هندسی طبقه‌بند حداکثر عرض نوار است که می‌توان برای جدا کردن بردارهای پشتیبان دو کلاس رسم کرد. یعنی دو برابر مقدار حداقل بر روی نقاط داده برای r داده شده در معادله (15.4)، یا به طور معادل، حداکثر عرض یکی از جداکننده‌های ضخیم نشان داده شده در شکل 15.2.

and so satisfies $\vec{w}^T \vec{x}' + b = 0$. Hence:

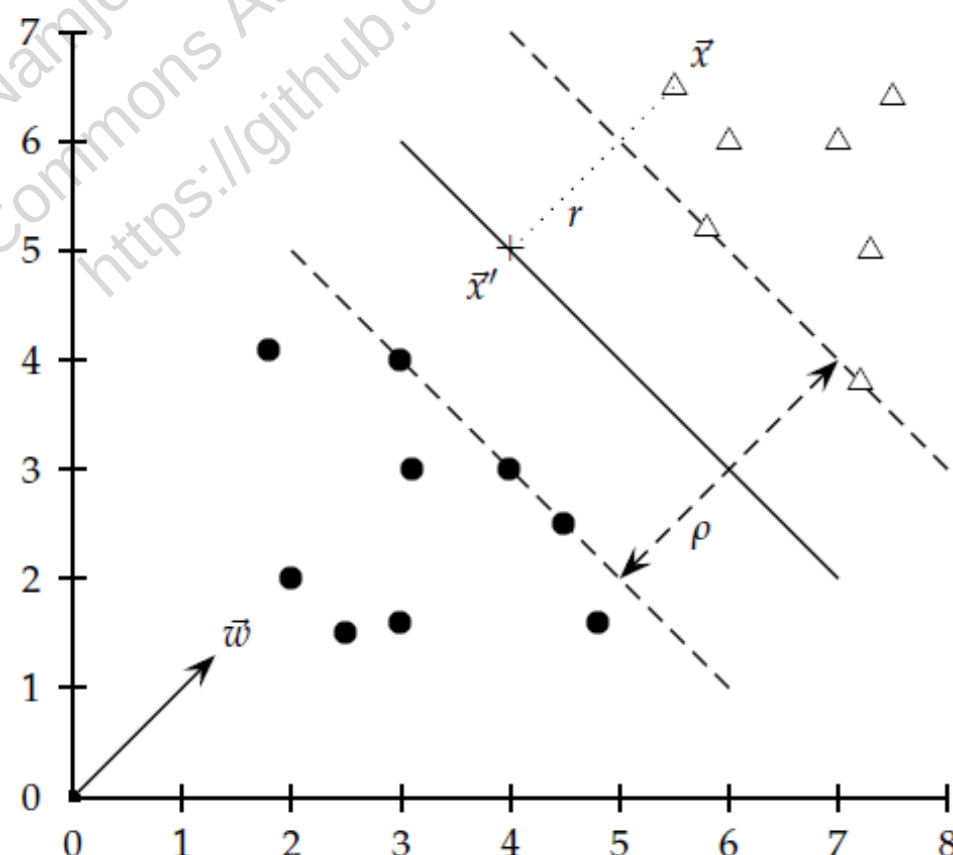
$$(15.3) \quad \vec{w}^T \left(\vec{x} - yr \frac{\vec{w}}{|\vec{w}|} \right) + b = 0$$

Solving for r gives:²

$$(15.4) \quad r = y \frac{\vec{w}^T \vec{x} + b}{|\vec{w}|}$$

شکل 15.3: حاشیه هندسی یک نقطه و یک مرز تصمیم

شکل 15.3 حاشیه هندسی یک نقطه r و یک مرز تصمیم ρ را نشان می‌دهد. فاصله اقلیدسی از یک نقطه $x_{\sim}$ تا مرز تصمیم‌گیری با r نشان داده می‌شود. کوتاه‌ترین فاصله بین یک نقطه و یک ابرصفحه عمود بر صفحه است و از این رو موازی $w_{\sim}$ است.



► Figure 15.3 The geometric margin of a point (r) and a decision boundary (ρ).

در مثال فیلتر اسپم، این فاصله نشان می‌دهد که یک ایمیل چقدر از مرز امن دور است. ایمیل‌هایی با فاصله کمتر، نیازمند بررسی دقیق‌تر هستند.

از آنجایی که می‌توانیم حاشیه تابعی را به دلخواه مقیاس‌بندی کنیم، برای راحتی در حل SVM‌های بزرگ، بیا بید انتخاب کنیم که حاشیه تابعی تمام نقاط داده حداقل 1 باشد و برای حداقل یک بردار داده برابر با 1 باشد. یعنی برای همه موارد در داده:

$$y_i(\tilde{w}^T \tilde{x}_i + b) \geq 1$$

و بردارهای پشتیبانی وجود دارند که برای آنها نامساوی یک برابری است. از آنجایی که فاصله هر مثال از ابرصفحه $r_i = y_i(w^T \tilde{x}_i + b)/|w|$ است، حاشیه هندسی $|w|/2$ است. هدف ما همچنان به حداکثر رساندن این حاشیه هندسی است. یعنی می‌خواهیم $|w|$ و b را پیدا کنیم به طوری که:

- $|w|/2$ به حداکثر برسد
- برای همه $(x_i, y_i) \in D$ ، $y_i(w^T \tilde{x}_i + b) \geq 1$

به حداکثر رساندن $|w|/2$ همان به حداقل رساندن $|w|/2$ است. این فرمولاسیون نهایی استاندارد یک SVM به عنوان یک مسئله بهینه‌سازی را می‌دهد:

Find $\tilde{w}$ and b such that:

$$\frac{1}{2} \tilde{w}^T \tilde{w} \text{ is minimized, and}$$

$$\text{for all } \{(\tilde{x}_i, y_i)\}, y_i(\tilde{w}^T \tilde{x}_i + b) \geq 1$$

حالا ما در حال بهینه‌سازی یک تابع درجه دوم تحت محدودیت‌های خطی هستیم. مسائل بهینه‌سازی درجه دو یک کلاس استاندارد و شناخته شده از مسائل بهینه‌سازی ریاضی هستند و الگوریتم‌های زیادی برای حل آنها وجود دارد.

با این حال، برای درک شکل راه‌حل چنین مسئله بهینه‌سازی، مفید خواهد بود. راه‌حل شامل ساخت یک مسئله دوگانه است که در آن یک ضریب لاگرانژ α_i با هر محدودیت $y_i(w^T \tilde{x}_i + b) \geq 1$ در مسئله اولیه مرتبط است:

$$\begin{aligned} \text{Find } \alpha_1, \dots, \alpha_N \text{ such that } & \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j \tilde{x}_i^T \tilde{x}_j \text{ is maximized, and} \\ & \sum_i \alpha_i y_i = 0 \\ & \alpha_i \geq 0 \text{ for all } 1 \leq i \leq N \end{aligned}$$

سپس راه‌حل به صورت زیر است:

$$\begin{aligned} \tilde{w} &= \sum_i \alpha_i y_i \tilde{x}_i \\ b &= y_k - \tilde{w}^T \tilde{x}_k \text{ for any } \tilde{x}_k \text{ such that } \alpha_k \neq 0 \end{aligned}$$

در راه‌حل، اکثر α_i صفر هستند. هر α_i غیرصفر نشان می‌دهد که x_i متناظر یک بردار پشتیبان است. تابع طبقه‌بندی سپس به صورت زیر است:

$$f(\tilde{x}) = \text{sign} \left(\sum_i \alpha_i y_i \tilde{x}_i^T \tilde{x} + b \right)$$

هم عبارتی که باید در مسئله دوگانه به حداکثر برسد و هم تابع طبقه‌بندی شامل یک محصول نقطه بین جفت نقاط $(\tilde{x}, x_i)$ یا x_i و x_j است، و این تنها راهی است که داده‌ها استفاده می‌شوند - ما بعداً به اهمیت این موضوع باز خواهیم گشت.

مثال 15.1: فیلتر اسپم برای ایمیل‌های بانکی

تصور کنید در تیم امنیتی بانک دیجیتال "پی‌پال" کار می‌کنید و باید سیستمی برای تشخیص ایمیل‌های فیشینگ (تقلبی) طراحی کنید. داده‌های ما شامل دو ویژگی اصلی برای هر ایمیل است: تعداد کلمات حساس (مانند "رمز عبور"، "حساب بانکی") و تعداد لینک‌های مشکوک. داده‌های آموزشی ما شامل سه ایمیل است:

- ایمیل 1: (1, 1) - ایمیل معتبر

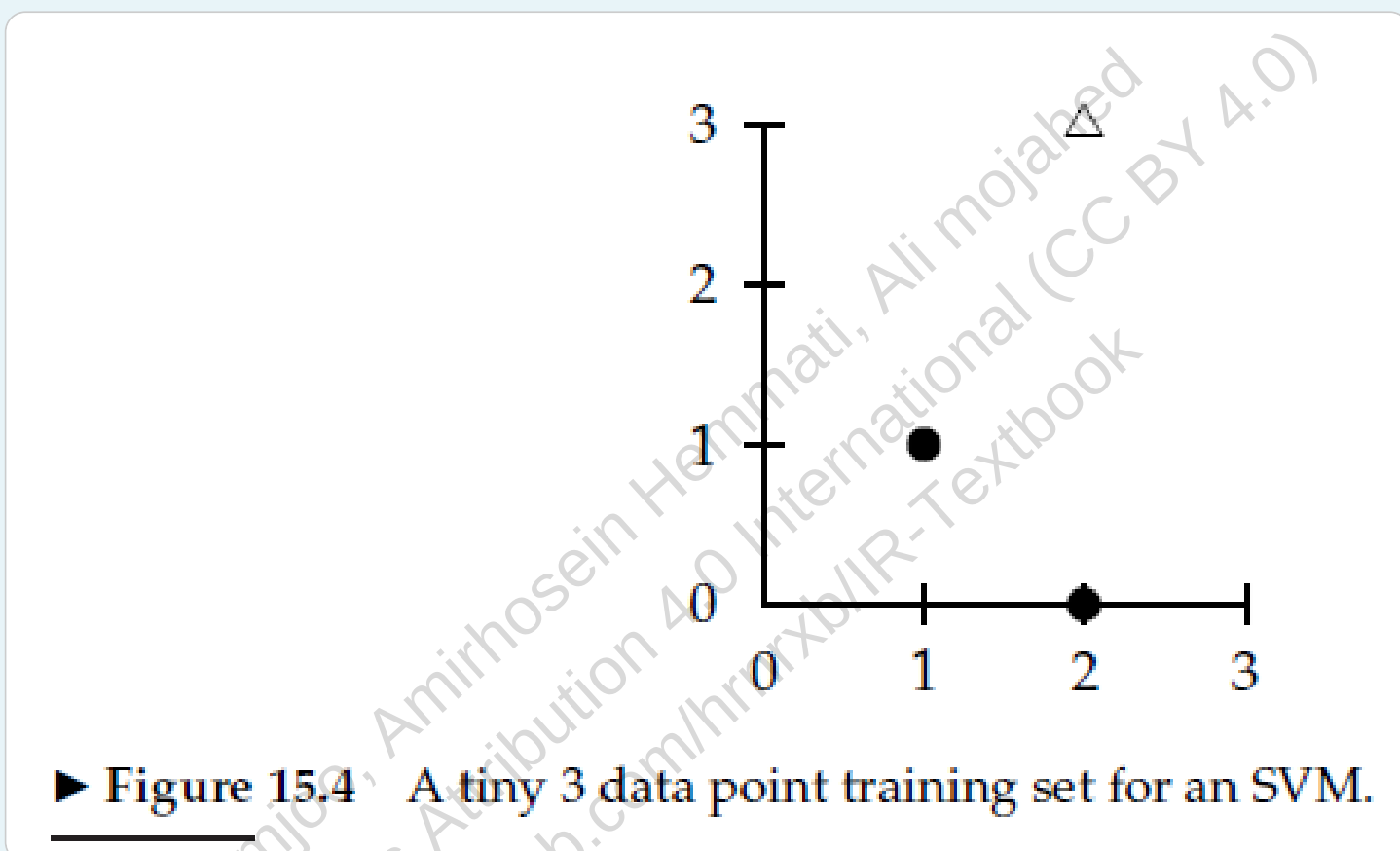
- ایمیل 2: (2, 3) - ایمیل فیشینگ
- ایمیل 3: (4, 2) - ایمیل فیشینگ

کار با روش هندسی، برای مثالی مانند این، بردار وزن حاشیه حداکثر موازی با کوتاه‌ترین خط اتصال نقاط دو کلاس خواهد بود، یعنی خط بین (1, 1) و (2, 3)، که یک بردار وزن (1, 2) را می‌دهد. سطح تصمیم‌گیری بهینه عمود بر آن خط است و آن را در نقطه نصفه قطع می‌کند. بنابراین، از (1.5, 2) عبور می‌کند. پس، مرز تصمیم‌گیری SVM:

$$y = x_1 + 2x_2 - 5.5$$

شکل 15.4: مجموعه داده آموزشی کوچک برای SVM

شکل 15.4 این مجموعه داده کوچک را نشان می‌دهد. نقاط توپر ایمیل‌های معتبر و نقاط توخالی ایمیل‌های فیشینگ هستند. خط مرز تصمیم بهینه است که توسط SVM پیدا شده است.



تحلیل جبری و کاربرد عملی

کار با روش جبری، با محدودیت استاندارد که $\text{sign}(y_i(w^T \tilde{x}_i + b)) \geq 1$ ، ما به دنبال به حداقل رساندن $|w|$ هستیم. این اتفاق می‌افتد وقتی که این محدودیت با برابری توسط دو بردار پشتیبان برآورده شود. علاوه بر این، می‌دانیم که راه‌حل $w = (a, 2a)$ برای برخی a است. پس ما داریم:

$$\begin{aligned} a + 2a + b &= -1 \\ 2a + 6a + b &= 1 \end{aligned}$$

بنابراین، $a = 2/5$ و $b = -11/5$. پس ابرصفحه بهینه با $w = (2/5, 4/5)$ و $b = -11/5$ داده می‌شود.

حاشیه ρ برابر با $2/\sqrt{(4/25 + 16/25)} = 2/(2\sqrt{5}/5) = \sqrt{5}/2$ است. این پاسخ را می‌توان به صورت هندسی با بررسی شکل 15.4 تأیید کرد.

کاربرد عملی در سیستم‌های فیلتر اسپم

در سیستم‌های واقعی فیلتر اسپم مانند آنچه در جیمیل استفاده می‌شود، از SVM‌ها با هزاران ویژگی استفاده می‌شود. این ویژگی‌ها ممکن است شامل موارد زیر باشند:

- تعداد کلمات حساس
- تعداد لینک‌های مشکوک
- اعتبار فرستنده
- ساختار ایمیل
- و بسیاری دیگر...

SVM با یافتن حاشیه بهینه، می‌تواند ایمیل‌های فیشینگ را با دقت بالایی تشخیص دهد، در حالی که ایمیل‌های معتبر را به اشتباه مسدود نمی‌کند. این تعادل حیاتی است، زیرا مسدود کردن اشتباه یک ایمیل معتبر می‌تواند منجر به از دست دادن ارتباطات مهم شود.

```
In [19]: import numpy as np
import matplotlib.pyplot as plt
from cvxopt import matrix, solvers
from sklearn import svm
from sklearn.metrics import accuracy_score

# =====
# برای حالت جداپذیر خطی (SVM) پیاده‌سازی ماشین بردار پشتیبان
# (Quadratic Programming) با استفاده از حل بهینه‌سازی درجه دو
# =====

class HardMarginSVM:
    """
    برای داده‌های خطی جداپذیر (Hard-Margin) حاشیه سخت SVM پیاده‌سازی
    این پیاده‌سازی به صورت تحصیلی و از پایه انجام شده تا مفاهیم ریاضی
    به طور شفاف در کد قابل مشاهده باشد.
    """

    def __init__(self):
        self.w = None # بردار وزن (وزن‌های ویژگی)
        self.b = None # بایاس (اندیشه)
        self.alpha = None # تنها بردارهای پشتیبان (ضرایب لاگرانژ  $\alpha > 0$ )
        self.support_vectors = None # بردارهای پشتیبان
        self.support_labels = None # برچسب‌های بردارهای پشتیبان
        self.margin = None # مقدار حاشیه هندسی

    def fit(self, X, y):
        """
        (Dual Formulation) با روش دوگانه SVM آموزش مدل

        پارامترها:
            X: (n_samples, n_features) داده‌های آموزشی به شکل
            y: (n_samples,) برچسب‌های کلاس (1+ یا -1) به شکل
        """
        n_samples, n_features = X.shape

        # برای تابع هدف کوادراتیک P محاسبه ماتریس
        #  $P_{ij} = y_i * y_j * \text{dot}(x_i, x_j)$ 
        P = np.zeros((n_samples, n_samples))
        for i in range(n_samples):
            for j in range(n_samples):
                P[i, j] = y[i] * y[j] * np.dot(X[i], X[j])

        # cvxopt تبدیل به فرمت مورد نیاز
        P = matrix(P)
        q = matrix(-np.ones(n_samples)) # برای تابع هدف q بردار

        # محدودیت‌های نابرابری:  $-\alpha_i \leq 0$  (معادل  $\alpha_i \geq 0$ )
        G = matrix(-np.eye(n_samples))
        h = matrix(np.zeros(n_samples))

        # محدودیت تساوی:  $\sum \alpha_i * y_i = 0$ 
        A = matrix(y.reshape(1, -1).astype(float))
        b = matrix(0.0)

        # حل مسئله بهینه‌سازی درجه دو
        solvers.options['show_progress'] = False
        solution = solvers.qp(P, q, G, h, A, b)

        # استخراج ضرایب لاگرانژ
        self.alpha = np.array(solution['x']).flatten()

        # مقدار بسیار کوچک ( $\alpha > 0$ ) شناسایی بردارهای پشتیبان
        support_vector_indices = self.alpha > 1e-5
        self.support_vectors = X[support_vector_indices]
        self.support_labels = y[support_vector_indices]
        self.alpha_sv = self.alpha[support_vector_indices]

        # وزن بردار  $w = \sum \alpha_i * y_i * x_i$ 
        self.w = np.sum(self.alpha_sv.reshape(-1, 1) *
                        self.support_labels.reshape(-1, 1) *
                        self.support_vectors, axis=0)

        # از روی هر بردار پشتیبان b محاسبه بایاس
        # از اولین بردار پشتیبان استفاده می‌کنیم
```

```

sv = self.support_vectors[0]
sv_label = self.support_labels[0]
self.b = sv_label - np.dot(self.w, sv)

# محاسبه حاشیه هندسی:  $\rho = 2 / ||w||$ 
self.margin = 2.0 / np.linalg.norm(self.w)

def predict(self, X):
    """
    پیش‌بینی برچسب برای داده‌های جدید

    پارامترها:
        X: داده‌های ورودی به شکل (n_samples, n_features)

    برمی‌گرداند:
        برچسب‌های پیش‌بینی شده (1 یا -1)
    """
    decision_values = np.dot(X, self.w) + self.b
    return np.sign(decision_values)

def get_decision_function(self, x1_range):
    """
    محاسبه مقادیر تابع تصمیم‌گیری برای ترسیم
    """
    w1, w2 = self.w
    #  $w_1 \cdot x_1 + w_2 \cdot x_2 + b = 0 \Rightarrow x_2 = -(w_1 \cdot x_1 + b) / w_2$ 
    decision_line = lambda x1: -(w1 * x1 + self.b) / w2
    margin_line_1 = lambda x1: -(w1 * x1 + self.b - 1) / w2
    margin_line_2 = lambda x1: -(w1 * x1 + self.b + 1) / w2

    return decision_line, margin_line_1, margin_line_2

def visualize(self, X, y, title="SVM Decision Boundary"):
    """
    ترسیم تصمیم‌گیری، حاشیه‌ها و بردارهای پشتیبان
    این تابع دقیقاً همانند شکل 15.4 در متن اصلی است
    """
    plt.figure(figsize=(10, 8))

    # ترسیم نقاط داده (هر کلاس با رنگ متفاوت)
    class_1 = y == 1
    class_2 = y == -1

    plt.scatter(X[class_1, 0], X[class_1, 1],
                c='blue', s=100, marker='o', label='Class +1 (legitimate)', alpha=0.7)
    plt.scatter(X[class_2, 0], X[class_2, 1],
                c='red', s=100, marker='s', label='Class -1 (spam)', alpha=0.7)

    # ترسیم بردارهای پشتیبان (دایره دور نقاط)
    if self.support_vectors is not None:
        plt.scatter(self.support_vectors[:, 0], self.support_vectors[:, 1],
                    facecolors='none', edgecolors='black', s=300,
                    linewidth=2, label='Support Vectors')

    # محاسبه محدوده محورها برای ترسیم
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    x_range = np.linspace(x_min, x_max, 100)

    # ترسیم مرز تصمیم‌گیری و خطوط حاشیه
    decision_line, margin_line_1, margin_line_2 = self.get_decision_function(x_range)

    plt.plot(x_range, decision_line(x_range), 'k-', linewidth=2,
             label=f'Decision Boundary ( $w \cdot x + b = 0$ )')
    plt.plot(x_range, margin_line_1(x_range), 'k--', linewidth=1.5,
             label=f'Margin Boundary ( $w \cdot x + b = +1$ )')
    plt.plot(x_range, margin_line_2(x_range), 'k--', linewidth=1.5,
             label=f'Margin Boundary ( $w \cdot x + b = -1$ )')

    # پر کردن ناحیه بین خطوط حاشیه
    plt.fill_between(x_range, margin_line_1(x_range), margin_line_2(x_range),
                    color='gray', alpha=0.2, label=f'Margin ( $\rho = \{self.margin:.2f\}$ )')

    plt.xlabel('Feature 1 (Sensitive Words Count)', fontsize=12)
    plt.ylabel('Feature 2 (Suspicious Links Count)', fontsize=12)
    plt.title(f'{title}\nMargin = {self.margin:.4f},  $||w|| = \{np.linalg.norm(self.w):.4f\}$ ',
             fontsize=14, fontweight='bold')
    plt.legend(loc='best', fontsize=10)
    plt.grid(True, alpha=0.3)
    plt.axis('equal')
    plt.show()

# =====
# مثال 15.1: مجموعه داده سه‌نقطه‌ای از متن کتاب
# =====

# داده‌های آموزشی: (تعداد کلمات حساس، تعداد لینک‌های مشکوک)
# نکته: در متن اصلی، (1,1) کلاس 1- و (2,3) کلاس 1+ هستند
X_train = np.array([
    [1, 1], # بردار پشتیبان کلاس 1- (اسپم)
    [2, 3], # بردار پشتیبان کلاس 1+ (غیر اسپم)
    [4, 2] # نمونه دیگر کلاس 1- (اسپم) - بردار پشتیبان نیست
], dtype=float)

y_train = np.array([-1, 1, -1]) # برچسب‌های کلاس

print("="*60)
print("برای مجموعه داده کوچک SVM مثال 15.1: حل")
print("="*60)
print("داده‌های آموزشی:")
print(f"X = \n{X_train}")
print(f"y = {y_train}")
print()

```

```

# ایجاد و آموزش مدل SVM
svm_model = HardMarginSVM()
svm_model.fit(X_train, y_train)

# نمایش نتایج
print(f"بردار وزن (w): [{svm_model.w[0]:.4f}, {svm_model.w[1]:.4f}]")
print(f"بایاس (b): {svm_model.b:.4f}")
print(f"اندازه بردار وزن (||w||): {np.linalg.norm(svm_model.w):.4f}")
print(f"حاشیه هندسی (p): {svm_model.margin:.4f}")
print()

# شناسایی بردارهای پشتیبان
print("بردارهای پشتیبان:")
for i, (sv, label, alpha) in enumerate(zip(svm_model.support_vectors,
                                           svm_model.support_labels,
                                           svm_model.alpha_sv)):
    print(f" {i+1}. x={sv}, y={label}, α={alpha:.4f}")
print()

# تست پیش‌بینی روی داده‌های آموزشی
predictions = svm_model.predict(X_train)
accuracy = accuracy_score(y_train, predictions)
print(f"دقت روی داده‌های آموزشی: {accuracy:.2%}")
print()

# ترسیم شکل 15.4
svm_model.visualize(X_train, y_train, title="Figure 15.4: Small Training Set for SVM")

# =====
# scikit-learn مقایسه با پیاده‌سازی
# =====

print("\n" + "="*60)
print("مقایسه با پیاده‌سازی مرجع (scikit-learn)")
print("="*60)

# (با پارامترهای مشابه حاشیه سخت) از scikit-Learn SVM ایجاد مدل
svm_sklarn = svm.SVC(kernel='linear', C=1e5) # بزرگ = حاشیه سخت C
svm_sklarn.fit(X_train, y_train)

# استخراج پارامترها
w_sklarn = svm_sklarn.coef_[0]
b_sklarn = svm_sklarn.intercept_[0]
margin_sklarn = 2.0 / np.linalg.norm(w_sklarn)

print(f"[scikit-learn] بردار وزن: [{w_sklarn[0]:.4f}, {w_sklarn[1]:.4f}]")
print(f"[scikit-learn] بایاس: {b_sklarn:.4f}")
print(f"[scikit-learn] حاشیه: {margin_sklarn:.4f}")
print(f"[scikit-learn] بردارهای پشتیبان: {svm_sklarn.n_support_} نقطه")

# دقت مقایسه
pred_sklarn = svm_sklarn.predict(X_train)
print(f"[scikit-learn] دقت: {accuracy_score(y_train, pred_sklarn):.2%}")

# =====
# نمایش تاثیر بردارهای پشتیبان
# =====

print("\n" + "="*60)
print("تحلیل تاثیر بردارهای پشتیبان")
print("="*60)

# حذف یک بردار پشتیبان و مشاهده تغییرات
print("(اگر نمونه (2, 4) را حذف کنیم (که بردار پشتیبان نیست)")
X_reduced = X_train[:2]
y_reduced = y_train[:2]

svm_reduced = HardMarginSVM()
svm_reduced.fit(X_reduced, y_reduced)

print(f"بردار وزن جدید: [{svm_reduced.w[0]:.4f}, {svm_reduced.w[1]:.4f}]")
print(f"حاشیه جدید: {svm_reduced.margin:.4f}")
print(f">{svm_reduced.w - svm_model.w} آیا تغییر کرد؟ {np.any(np.abs(svm_reduced.w - svm_model.w) > 1e-3)}")

print("\n(اگر یک بردار پشتیبان را حذف کنیم)")
# حذف نمونه (1, 1) که بردار پشتیبان است
X_no_sv1 = np.array([[2, 3], [4, 2]])
y_no_sv1 = np.array([1, -1])

svm_no_sv1 = HardMarginSVM()
svm_no_sv1.fit(X_no_sv1, y_no_sv1)

print(f"بردار وزن جدید: [{svm_no_sv1.w[0]:.4f}, {svm_no_sv1.w[1]:.4f}]")
print(f"حاشیه جدید: {svm_no_sv1.margin:.4f}")
print(f">{svm_no_sv1.w - svm_model.w} آیا تغییر کرد؟ {np.any(np.abs(svm_no_sv1.w - svm_model.w) > 1e-3)}")
print("نتیجه: حذف بردار پشتیبان کاملاً مدل را تغییر می‌دهد")

# =====
# پیش‌بینی برای داده‌های جدید (مثال کاربردی)
# =====

print("\n" + "="*60)
print("تست پیش‌بینی برای ایمیل‌های جدید")
print("="*60)

# ایمیل‌های جدید برای تست
X_test = np.array([
    [1.5, 1.5], # شبیه به (1,1) - احتمالاً اسپم
    [2.5, 3.5], # شبیه به (2,3) - احتمالاً غیر اسپم
    [3, 2.5], # در منطقه مرزی
])

```

```
predictions_test = svm_model.predict(X_test)
decision_values = np.dot(X_test, svm_model.w) + svm_model.b
distances = np.abs(decision_values) / np.linalg.norm(svm_model.w)

print("ایمیل‌های تست:")
for i, (email, pred, dist) in enumerate(zip(X_test, predictions_test, distances)):
    status = "غیر اسپم" if pred == 1 else "اسپم"
    confidence = "بالا" if dist > svm_model.margin/2 else "پایین"
    print(f"ایمیل {i+1}: {email} → {status} (فاصله از مرز: {dist:.2f}, اطمینان: {confidence})")

# ترسیم داده‌های تست روی نمودار
plt.figure(figsize=(10, 8))

# نمایش داده‌های آموزشی
class_1 = y_train == 1
class_2 = y_train == -1
plt.scatter(X_train[class_1, 0], X_train[class_1, 1], c='blue', s=100, marker='o', label='Train: Class +1')
plt.scatter(X_train[class_2, 0], X_train[class_2, 1], c='red', s=100, marker='s', label='Train: Class -1')

# نمایش بردارهای پشتیبان
plt.scatter(svm_model.support_vectors[:, 0], svm_model.support_vectors[:, 1],
            facecolors='none', edgecolors='black', s=300, linewidth=2, label='Support Vectors')

# نمایش داده‌های تست
test_class_1 = predictions_test == 1
test_class_2 = predictions_test == -1
plt.scatter(X_test[test_class_1, 0], X_test[test_class_1, 1],
            c='blue', s=200, marker='*', label='Test: Predicted +1', alpha=0.7)
plt.scatter(X_test[test_class_2, 0], X_test[test_class_2, 1],
            c='red', s=200, marker='*', label='Test: Predicted -1', alpha=0.7)

# ترسیم مرز تصمیم و حاشیه‌ها
x_min, x_max = plt.gca().get_xlim()
x_range = np.linspace(x_min, x_max, 100)
decision_line, margin_line_1, margin_line_2 = svm_model.get_decision_function(x_range)

plt.plot(x_range, decision_line(x_range), 'k-', linewidth=2, label='Decision Boundary')
plt.plot(x_range, margin_line_1(x_range), 'k--', linewidth=1.5, label='Margin')
plt.plot(x_range, margin_line_2(x_range), 'k--', linewidth=1.5)

plt.fill_between(x_range, margin_line_1(x_range), margin_line_2(x_range),
                 color='gray', alpha=0.1)

plt.xlabel('Feature 1 (Sensitive Words Count)', fontsize=12)
plt.ylabel('Feature 2 (Suspicious Links Count)', fontsize=12)
plt.title('Prediction of New Emails Using SVM', fontsize=14, fontweight='bold')
plt.legend(fontsize=10)
plt.grid(True, alpha=0.3)
plt.axis('equal')
plt.show()
```

برای مجموعه داده کوچک SVM مثال 15.1: حل

داده‌های آموزشی:

```
X =
[[1. 1.]
 [2. 3.]
 [4. 2.]]
y = [-1  1 -1]
```

وزن بردار (w): [-0.4000, 1.2000]

بایاس (b): -1.8000

اندازه بردار وزن ($\|w\|$): 1.2649

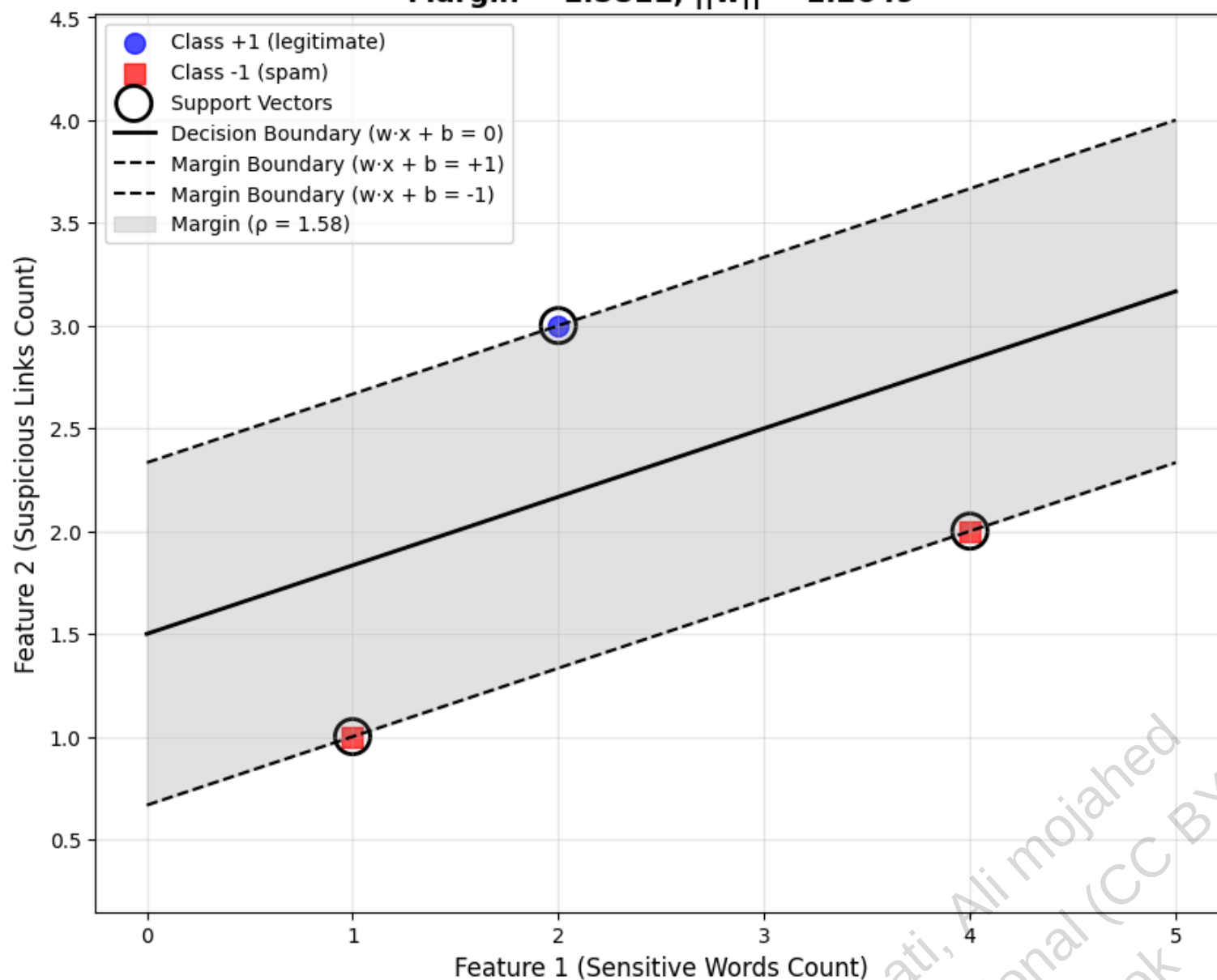
حاشیه هندسی (ρ): 1.5811

بردارهای پشتیبان:

1. $x=[1. \ 1.]$, $y=-1$, $\alpha=0.4000$
2. $x=[2. \ 3.]$, $y=1$, $\alpha=0.8000$
3. $x=[4. \ 2.]$, $y=-1$, $\alpha=0.4000$

%دقت روی داده‌های آموزشی: 100.00

Figure 15.4: Small Training Set for SVM
Margin = 1.5811, $||w|| = 1.2649$



=====

مقایسه با پیاده‌سازی مرجع (scikit-learn)

=====

[scikit-learn] بردار وزن: [1.2000, 0.4000-]

[scikit-learn] بایاس: 1.8000-

[scikit-learn] حاشیه: 1.5811

[scikit-learn] بردارهای پشتیبان: [1 2] نقطه

[scikit-learn] %دقت: 100.00

=====

تحلیل تأثیر بردارهای پشتیبان

=====

:اگر نمونه (2, 4) را حذف کنیم (که بردار پشتیبان نیست)

بردار وزن جدید: [0.8000, 0.4000]

حاشیه جدید: 2.2361

True آیا تغییر کرد؟

:اگر یک بردار پشتیبان را حذف کنیم

بردار وزن جدید: [0.4000, 0.8000-]

حاشیه جدید: 2.2361

True آیا تغییر کرد؟

نتیجه: حذف بردار پشتیبان کاملاً مدل را تغییر می‌دهد.

=====

تست پیش‌بینی برای ایمیل‌های جدید

=====

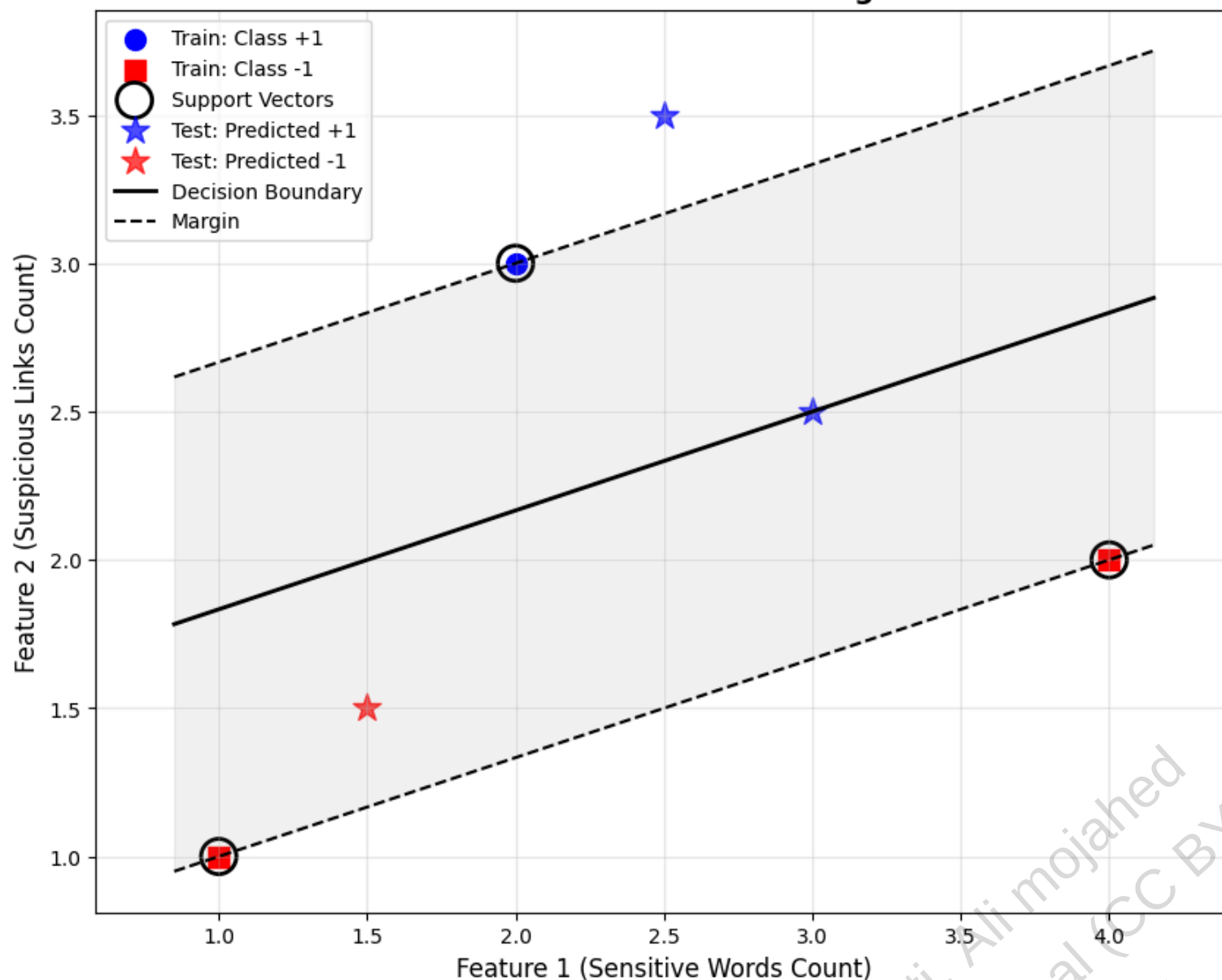
:ایمیل‌های تست

ایمیل 1: [1.5 1.5] → اسپم (فاصله از مرز: 0.47, اطمینان: پایین)

ایمیل 2: [3.5 2.5] → غیر اسپم (فاصله از مرز: 1.11, اطمینان: بالا)

ایمیل 3: [2.5 3] → غیر اسپم (فاصله از مرز: 0.00, اطمینان: پایین)

Prediction of New Emails Using SVM



15.2.1 طبقه‌بندی حاشیه نرم: انعطاف‌پذیری در دنیای واقعی

🔗 تشخیص پزشکی: چگونه یک مدل هوشمند می‌تواند جان انسان‌ها را نجات دهد؟

تصور کنید در تیم تحقیقاتی بیمارستان "میو کلینیک" کار می‌کنید و سیستمی برای تشخیص زودهنگام سرطان طراحی می‌کنید. داده‌های شما شامل نتایج آزمایش‌های مختلف بیماران است که باید به دو گروه "سرطانی" و "غیرسرطانی" تقسیم شوند.

در دنیای واقعی، داده‌های پزشکی همیشه به صورت خطی جداپذیر نیستند. برخی بیماران ممکن است نتایج آزمایش غیرعادی داشته باشند اما سرطان نداشته باشند (نویز)، در حالی که برخی دیگر ممکن است نتایج طبیعی داشته باشند اما به سرطان مبتلا باشند. در این موارد، یک مدل با حاشیه سخت (hard margin) که هیچ خطایی را تحمل نمی‌کند، عملکرد خوبی نخواهد داشت.

اینجاست که **طبقه‌بندی حاشیه نرم (soft margin)** در SVM وارد می‌شود. این روش به مدل اجازه می‌دهد تا برخی خطاها را بپذیرد تا عملکرد کلی بهتری داشته باشد.

برای مسائل با ابعاد بسیار بالا که در طبقه‌بندی متن رایج است، گاهی داده‌ها به صورت خطی جداپذیر هستند. اما در حالت کلی، اینطور نیست و حتی اگر جداپذیر باشند، ممکن است راه‌حلی را ترجیح دهیم که بهتر بخش عمده داده‌ها را جدا کند در حالی که چند سند نویز نادیده گرفته شوند.

اگر مجموعه آموزشی D به صورت خطی جداپذیر نباشد، رویکرد استاندارد این است که به حاشیه تصمیم ضخیم اجازه دهیم چند اشتباه مرتکب شود (برخی نقاط - داده‌های پرت یا نمونه‌های نویزی - در داخل یا در سمت اشتباه حاشیه قرار دارند). سپس برای هر مثال طبقه‌بندی اشتباه، هزینه‌ای پرداخت می‌کنیم که به فاصله آن از برآورده کردن الزام حاشیه در معادله (15.5) بستگی دارد.

برای پیاده‌سازی این، متغیرهای لغزش (slack variables) ξ_i را معرفی می‌کنیم. در مثال تشخیص پزشکی، این متغیرها به مدل اجازه می‌دهند تا برخی بیماران را به اشتباه طبقه‌بندی کند.

مقدار غیرصفر برای ξ_i به x_i اجازه می‌دهد تا الزام حاشیه را با هزینه‌ای متناسب با مقدار ξ_i برآورده نکند. در مثال پزشکی، این به معنای پذیرش برخی خطاها در تشخیص است تا مدل کلی بهتری داشته باشیم.

مثلاً، در تشخیص سرطان، ما ممکن است ترجیح دهیم که چند فرد سالم را به اشتباه به عنوان "مشکوک به سرطان" علامت‌گذاری کنیم (false positive) تا اینکه حتی یک فرد مبتلا به سرطان را از دست بدهیم (false negative). این مصداق بارز استفاده از حاشیه نرم است.

فرمولاسیون مسئله بهینه‌سازی SVM با متغیرهای لغزش به صورت زیر است:

Find $\tilde{w}$, b , and $\xi_i \geq 0$ such that:

$$\frac{1}{2} \tilde{w}^T \tilde{w} + C \sum_i \xi_i \text{ is minimized}$$

$$\text{and for all } \{(\tilde{x}_i, y_i)\}, y_i(\tilde{w}^T \tilde{x}_i + b) \geq 1 - \xi_i$$

🏛️ معیار منظم‌سازی C: تعادل بین دقت و انعطاف‌پذیری

مسئله بهینه‌سازی سپس در حالی است که ضخامت حاشیه را در برابر تعداد نقاطی که باید برای این حاشیه جابجا شوند، معامله می‌کند. حاشیه می‌تواند برای یک نقطه x_i با تنظیم $\xi_i > 0$ کمتر از 1 باشد، اما سپس برای این کار، جریمه‌ای به اندازه $\xi_i C$ در بهینه‌سازی پرداخت می‌شود.

در مثال پزشکی، پارامتر C تعیین می‌کند که مدل چقدر باید به خطاها حساس باشد:

- وقتی C بزرگ است، مدل به خطاها جریمه سنگینی می‌دهد و تلاش می‌کند تا حد امکان خطاها را کاهش دهد، حتی اگر به معنای حاشیه کوچکتر باشد.
- وقتی C کوچک است، مدل خطاها را با جریمه کمتری می‌پذیرد و حاشیه بزرگتری ایجاد می‌کند که ممکن است برخی نقاط را اشتباه طبقه‌بندی کند.

در تشخیص پزشکی، ممکن است مقدار C را کوچک انتخاب کنیم تا مدل انعطاف‌پذیرتر باشد و بتواند موارد مرزی را بهتر مدیریت کند.

مسئله دوگانه برای طبقه‌بندی حاشیه نرم به صورت زیر است:

$$\text{Find } \alpha_1, \dots, \alpha_N \text{ such that } \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j \tilde{x}_i^T \tilde{x}_j \text{ is maximized, and}$$

$$\sum_i \alpha_i y_i = 0$$

$$0 \leq \alpha_i \leq C \text{ for all } 1 \leq i \leq N$$

نه متغیرهای لغزش ξ_i و نه ضرایب لاگرانژ برای آنها در مسئله دوگانه ظاهر نمی‌شوند. تمام آنچه باقی می‌ماند ثابت C است که اندازه ممکن ضرایب لاگرانژ برای نقاط داده بردار پشتیبان را محدود می‌کند. مانند قبل، x_i با α_i غیرصفر، بردارهای پشتیبان خواهند بود. راه‌حل مسئله دوگانه به صورت زیر است:

$$\tilde{w} = \sum \alpha_i y_i \tilde{x}_i$$

$$b = y_k(1 - \xi_k) - \tilde{w}^T \tilde{x}_k \text{ for } k = \arg \max_k \alpha_k$$

معمولاً، بردارهای پشتیبان بخش کوچکی از داده‌های آموزشی را تشکیل می‌دهند. با این حال، اگر مسئله غیرقابل تفکیک باشد یا حاشیه کوچکی داشته باشد، آنگاه هر نقطه داده که طبقه‌بندی اشتباه شده یا در داخل حاشیه قرار دارد، α غیرصفر خواهد داشت. اگر این مجموعه نقاط بزرگ شود، برای حالت غیرخطی که در بخش 15.2.3 به آن می‌پردازیم، این می‌تواند یک کاهش سرعت عمده برای استفاده از SVMها در زمان آزمون باشد.

در سیستم‌های تشخیص پزشکی واقعی مانند آنچه در "مِیو کلینیک" استفاده می‌شود، این مسئله بسیار مهم است. اگر تعداد بردارهای پشتیبان زیاد شود، زمان تشخیص برای هر بیمار جدید افزایش می‌یابد که در شرایط اورژانسی پزشکی قابل قبول نیست.

پیچیدگی آموزش و آزمون با SVMهای خطی در جدول 15.1 نشان داده شده است. زمان آموزش یک SVM تحت تأثیر زمان حل QP زیربنایی است و بنابراین پیچیدگی نظری و تجربی بسته به روش استفاده شده برای حل آن متفاوت است. نتیجه استاندارد برای حل QPها این است که زمانی مکعبی در اندازه مجموعه داده می‌برد. تمام کارهای اخیر در آموزش SVM تلاش کرده‌اند تا این پیچیدگی را کاهش دهند، اغلب با راضی بودن به راه‌حل‌های تقریبی. به طور استاندارد، پیچیدگی تجربی حدود $O(|D|^{1.7})$ است. با این حال، زمان آموزش فوق خطی الگوریتم‌های سنتی SVM استفاده از آنها را در مجموعه داده‌های آموزشی بسیار بزرگ دشوار یا غیرممکن می‌سازد.

Classifier	Mode	Method	Time complexity
NB	training		$\Theta(D L_{ave} + C V)$
NB	testing		$\Theta(C M_a)$
Rocchio	training		$\Theta(D L_{ave} + C V)$
Rocchio	testing		$\Theta(C M_a)$
kNN	training	preprocessing	$\Theta(D L_{ave})$
kNN	testing	preprocessing	$\Theta(D M_{ave}M_a)$
kNN	training	no preprocessing	$\Theta(1)$
kNN	testing	no preprocessing	$\Theta(D L_{ave}M_a)$
SVM	training	conventional	$O(C D ^3M_{ave}); \approx O(C D ^{1.7}M_{ave})$, empirically
SVM	training	cutting planes	$O(C D M_{ave})$
SVM	testing		$O(C M_a)$

جدول 15.1: پیچیدگی آموزش و آزمون طبقه‌بندهای مختلف

جدول 15.1 پیچیدگی آموزش و آزمون طبقه‌بندهای مختلف از جمله SVMها را نشان می‌دهد. در سیستم‌های تشخیص پزشکی، این پیچیدگی‌ها بسیار مهم هستند:

- روش‌های سنتی SVM با پیچیدگی $O(|D|^{1.7})$ ممکن است برای مجموعه داده‌های بزرگ پزشکی بسیار کند باشند.
- روش‌های جدیدتر مانند "cutting planes" با پیچیدگی خطی $O(|D|)$ برای سیستم‌های پزشکی مناسب‌تر هستند.
- در حالی که روش‌هایی مانند Naive Bayes آموزش سریع‌تری دارند، اما ممکن است دقت کمتری در تشخیص‌های پیچیده پزشکی داشته باشند.

در عمل، تیم‌های پزشکی ممکن است از روش‌های ترکیبی استفاده کنند: ابتدا یک روش سریع برای غربالگری اولیه، سپس SVM برای موارد مشکوک.

In [21]:

```
import numpy as np
import matplotlib.pyplot as plt
from cvxopt import matrix, solvers
from sklearn import svm
from sklearn.metrics import accuracy_score

# =====
# برای داده‌های غیرجدپذیر خطی (Soft-Margin) حاشیه نرم SVM پیاده‌سازی
# استفاده می‌کند (QP) این پیاده‌سازی از بهینه‌سازی درجه دو
# =====

class SoftMarginSVM:
    """
    (slack variables) حاشیه نرم با متغیرهای لغزش SVM پیاده‌سازی آموزشی
    تعادل بین حاشیه بزرگ و خطای طبقه‌بندی را کنترل می‌کند C این کلاس نشان می‌دهد چگونه پارامتر
    """

    def __init__(self, C=1.0):
        """
        ضریب منظم‌سازی که هزینه خطا را کنترل می‌کند: C پارامتر
        - کوچک: حاشیه بزرگتر، تحمل خطای بیشتر (مناسب برای داده با نویز بالا) C
        - بزرگ: حاشیه کوچکتر، خطای کمتر (نزدیک به حاشیه سخت) C
        """
        self.C = C
        self.w = None # بردار وزن
        self.b = None # بایاس
        self.alpha = None # ضرایب لاگرانژ
        self.support_vectors = None
        self.support_labels = None
        self.margin = None
        self.slack_points = [] # نقاطی که از حاشیه عبور کرده‌اند

    def fit(self, X, y):
        """
        حاشیه نرم با روش دوگانه SVM آموزش مدل
        (بود  $\alpha_i \geq 0$  در مقایسه با حاشیه سخت که فقط  $\alpha_i \leq C$  محدودیت جدید:  $\theta$ )
        """
        n_samples, n_features = X.shape

        # برای تابع هدف  $P$  محاسبه ماتریس  $P_{ij} = y_i * y_j * \text{dot}(x_i, x_j)$ 
        P = np.zeros((n_samples, n_samples))
        for i in range(n_samples):
            for j in range(n_samples):
                P[i, j] = y[i] * y[j] * np.dot(X[i], X[j])

        # cvxopt تبدیل به فرمت
        P = matrix(P)
        q = matrix(-np.ones(n_samples))

        # محدودیت‌های نابرابری:  $-\alpha_i \leq \theta$  و  $\alpha_i \leq C$ 
        G = np.vstack([-np.eye(n_samples), np.eye(n_samples)])
        h = np.hstack([np.zeros(n_samples), self.C * np.ones(n_samples)])

        G = matrix(G)
        h = matrix(h)

        # محدودیت تساوی:  $\sum \alpha_i * y_i = \theta$ 
        A = matrix(y.reshape(1, -1).astype(float))
        b = matrix(0.0)

        # حل مسئله بهینه‌سازی
        solvers.options['show_progress'] = False
        solution = solvers.qp(P, q, G, h, A, b)

        self.alpha = np.array(solution['x']).flatten()

        # شناسایی بردارهای پشتیبان:  $\alpha_i > \theta$ 
        support_vector_indices = self.alpha > 1e-5
        self.support_vectors = X[support_vector_indices]
        self.support_labels = y[support_vector_indices]
        self.alpha_sv = self.alpha[support_vector_indices]

        # محاسبه بردار وزن:  $w = \sum \alpha_i * y_i * x_i$ 
        self.w = np.sum(self.alpha_sv.reshape(-1, 1) *
                        self.support_labels.reshape(-1, 1) *
                        self.support_vectors, axis=0)

        #  $\alpha_i < C$  محاسبه بایاس از یک بردار پشتیبان با
        sv_margin_indices = (self.alpha > 1e-5) & (self.alpha < self.C - 1e-5)
        if np.any(sv_margin_indices):
            idx = np.where(sv_margin_indices)[0][0]
            sv = X[idx]
            sv_label = y[idx]
            self.b = sv_label - np.dot(self.w, sv)
        else:
            # باشند، از میانگین استفاده کن  $\alpha_i = C$  اگر همه
            self.b = np.mean(self.support_labels - np.dot(self.support_vectors, self.w))

        # محاسبه حاشیه هندسی:  $\rho = 2 / ||w||$ 
        self.margin = 2.0 / np.linalg.norm(self.w)

        # شناسایی نقاط با لغزش ( $\xi_i > \theta$ )
```



```

این نقاط یا اشتباه طبقه‌بندی می‌شوند یا در داخل حاشیه هستند
self.slack_points = []
for i, x_i in enumerate(X):
    decision_val = y[i] * (np.dot(self.w, x_i) + self.b)
    if decision_val < 1 - 1e-5: # نقطه از حاشیه عبور کرده
        self.slack_points.append({
            'point': x_i,
            'label': y[i],
            'slack_amount': 1 - decision_val
        })

def predict(self, X):
    """پیش‌بینی برچسب برای داده‌های جدید"""
    return np.sign(np.dot(X, self.w) + self.b)

def get_decision_function(self, x_range):
    """محاسبه معادلات مرز تصمیم و حاشیه‌ها برای ترسیم"""
    w1, w2 = self.w
    decision_line = lambda x1: -(w1 * x1 + self.b) / w2
    margin_line_1 = lambda x1: -(w1 * x1 + self.b - 1) / w2
    margin_line_2 = lambda x1: -(w1 * x1 + self.b + 1) / w2
    return decision_line, margin_line_1, margin_line_2

def visualize(self, X, y, title=""):
    """ترسیم مرز تصمیم، حاشیه، بردارهای پشتیبان و نقاط لغزش"""
    plt.figure(figsize=(12, 8))

    # ترسیم نقاط داده (دو کلاس)
    class_1 = y == 1
    class_2 = y == -1

    plt.scatter(X[class_1, 0], X[class_1, 1],
                c='blue', s=80, marker='o', label='Positive Class', alpha=0.7)
    plt.scatter(X[class_2, 0], X[class_2, 1],
                c='red', s=80, marker='s', label='Negative Class', alpha=0.7)

    # ترسیم بردارهای پشتیبان (دایره سیاه)
    if len(self.support_vectors) > 0:
        plt.scatter(self.support_vectors[:, 0], self.support_vectors[:, 1],
                    facecolors='none', edgecolors='black', s=300,
                    linewidth=2, label='Support Vectors')

    # ترسیم نقاطی که از حاشیه عبور کرده‌اند (علامت ضرب)
    if len(self.slack_points) > 0:
        slack_array = np.array([p['point'] for p in self.slack_points])
        plt.scatter(slack_array[:, 0], slack_array[:, 1],
                    c='orange', s=150, marker='x', linewidth=3,
                    label=f'Points with Slack ( $\xi > 0$ ): {len(self.slack_points)}')

    # ترسیم مرزها
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    x_range = np.linspace(x_min, x_max, 100)

    decision_line, margin_line_1, margin_line_2 = self.get_decision_function(x_range)

    plt.plot(x_range, decision_line(x_range), 'k-', linewidth=2,
             label=f'Decision Boundary')
    plt.plot(x_range, margin_line_1(x_range), 'k--', linewidth=1.5,
             label=f'Margin Boundary (w·x + b = ±1)')
    plt.plot(x_range, margin_line_2(x_range), 'k--', linewidth=1.5)

    # پر کردن ناحیه حاشیه
    plt.fill_between(x_range, margin_line_1(x_range), margin_line_2(x_range),
                    color='gray', alpha=0.2,
                    label=f'Margin (p = {self.margin:.2f})')

    plt.xlabel('Feature 1 (Biomarker Level)', fontsize=12)
    plt.ylabel('Feature 2 (Gene Expression)', fontsize=12)
    plt.title(f'{title}\nC={self.C}, Support Vectors={len(self.support_vectors)}, '
              f'Margin={self.margin:.3f}', fontsize=14, fontweight='bold')
    plt.legend(loc='best', fontsize=10)
    plt.grid(True, alpha=0.3)
    plt.axis('equal')
    plt.show()

# =====
# ایجاد داده‌های مصنوعی برای مثال تشخیص پزشکی
# داده‌ها شامل دو ویژگی: سطح مارکر زیستی و بیان ژن
# =====

np.random.seed(42)

# کلاس سرطانی (class -1)
X_cancer = np.random.randn(15, 2) + [2, 2]
# برای شبیه‌سازی نویز واقعی (outliers) اضافه کردن نقاط پرت
X_cancer = np.vstack([X_cancer, [3.5, 3.5], [4, 3]])

# کلاس غیرسرطانی (class +1)
X_healthy = np.random.randn(15, 2) + [3.5, 3.5]
# اضافه کردن نقاط پرت
X_healthy = np.vstack([X_healthy, [2.5, 2.5], [3, 4]])

# ترکیب داده‌ها
X_medical = np.vstack([X_cancer, X_healthy])
y_medical = np.hstack([-np.ones(len(X_cancer)), np.ones(len(X_healthy))])

print("="*60)
print("Soft-Margin SVM برای تشخیص پزشکی")
print("non-separable) داده حاوی نقاط همپوشانی")
print("="*60)
print(f"تعداد نمونه‌ها: {len(X_medical)}")
print(f"تعداد ویژگی‌ها: {X_medical.shape[1]}")

```



```

# =====
# مقایسه عملکرد برای مقادیر مختلف C
# =====

C_values = [0.1, 1.0, 10.0, 100.0]
models = {}

for C in C_values:
    print(f"\n{' '*40}")
    print(f"C = {C} آموزش مدل با")
    print(f"\n{' '*40}")

    model = SoftMarginSVM(C=C)
    model.fit(X_medical, y_medical)
    models[C] = model

    # دقت روی داده آموزشی
    predictions = model.predict(X_medical)
    accuracy = accuracy_score(y_medical, predictions)

    print(f"بردار وزن: [{model.w[0]:.4f}, {model.w[1]:.4f}]")
    print(f"بایاس: {model.b:.4f}")
    print(f"اندازه بردار وزن (||w||): {np.linalg.norm(model.w):.4f}")
    print(f"حاشیه هندسی: {model.margin:.4f}")
    print(f"تعداد بردارهای پشتیبان: {len(model.support_vectors)}")
    print(f"تعداد نقاط با لغزش: {len(model.slack_points)}")
    print(f"دقت روی داده آموزشی: {accuracy:.2%}")

    # ترسیم نتایج
    model.visualize(X_medical, y_medical, title=f"Medical Diagnosis with Soft-Margin SVM (C={C})")

# =====
# روی همین داده (Hard-Margin) مقایسه با حاشیه سخت
# =====

print(f"\n{' '*40}")
print("Hard-Margin SVM مقایسه با")
print(f"\n{' '*40}")

# بسیار بزرگ C ایجاد مدل حاشیه سخت با استفاده از
hard_margin_model = SoftMarginSVM(C=1e6) # حاشیه سخت C
hard_margin_model.fit(X_medical, y_medical)

predictions_hard = hard_margin_model.predict(X_medical)
accuracy_hard = accuracy_score(y_medical, predictions_hard)

print(f"[Hard-Margin] تعداد بردارهای پشتیبان: {len(hard_margin_model.support_vectors)}")
print(f"[Hard-Margin] تعداد نقاط با لغزش: {len(hard_margin_model.slack_points)}")
print(f"[Hard-Margin] حاشیه: {hard_margin_model.margin:.4f}")
print(f"[Hard-Margin] دقت: {accuracy_hard:.2%}")

hard_margin_model.visualize(X_medical, y_medical, title="Hard-Margin SVM on Non-separable Data")

# =====
# تحلیل حساسیت به پارامتر C
# =====

print(f"\n{' '*40}")
print("C تحلیل حساسیت به پارامتر")
print(f"\n{' '*40}")

# ایجاد داده تست جدید برای ارزیابی تعمیم‌پذیری
np.random.seed(123)
X_test_cancer = np.random.randn(8, 2) + [2, 2]
X_test_healthy = np.random.randn(8, 2) + [3.5, 3.5]
X_test_medical = np.vstack([X_test_cancer, X_test_healthy])
y_test_medical = np.hstack([-np.ones(len(X_test_cancer)), np.ones(len(X_test_healthy))])

print("دقت روی داده تست:")
for C in C_values:
    model = models[C]
    test_pred = model.predict(X_test_medical)
    test_acc = accuracy_score(y_test_medical, test_pred)
    print(f"C={C:6.1f}: دقت = {test_acc:.2%}")

# =====
# نمایش پیچیدگی محاسباتی
# =====

print(f"\n{' '*40}")
print("تحلیل پیچیدگی محاسباتی")
print(f"\n{' '*40}")

# و تعداد بردارهای پشتیبان C بررسی رابطه بین
print("و تعداد بردارهای پشتیبان C رابطه بین:")
for C in sorted(models.keys()):
    model = models[C]
    sv_count = len(model.support_vectors)
    slack_count = len(model.slack_points)
    print(f"C={C:6.1f}: SVs={sv_count:2d}, Slack Points={slack_count:2d}")
    # معمولاً افزایش می‌یابد (پیچیده‌تر شدن مدل) SV تعداد، C، با افزایش

print("\nبا دقت انتخاب شود C، نکته: در سیستم‌های پزشکی واقعی")
print("خیلی بزرگ: مدل پیچیده، زمان تست طولانی (چالش جدول 15.1) - C")
print("خیلی کوچک: مدل ساده، ممکن است خطای بالا داشته باشد - C")

# =====
# بر روی حاشیه و دقت C تحلیل تأثیر
# =====

print(f"\n{' '*40}")
print("بر روی حاشیه و دقت C تحلیل تأثیر")

```

```
print(f"{' '*40}")

plt.figure(figsize=(10, 6))
C_values_plot = C_values
train_accuracies = [accuracy_score(y_medical, models[C].predict(X_medical)) for C in C_values_plot]
test_accuracies = [accuracy_score(y_test_medical, models[C].predict(X_test_medical)) for C in C_values_plot]
margins = [models[C].margin for C in C_values_plot]

plt.subplot(1, 2, 1)
plt.plot(C_values_plot, train_accuracies, 'bo-', label='Train Accuracy')
plt.plot(C_values_plot, test_accuracies, 'ro-', label='Test Accuracy')
plt.xscale('log')
plt.xlabel('C (log scale)')
plt.ylabel('Accuracy')
plt.title('Accuracy vs. C')
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(C_values_plot, margins, 'go-')
plt.xscale('log')
plt.xlabel('C (log scale)')
plt.ylabel('Margin')
plt.title('Margin vs. C')
plt.grid(True)

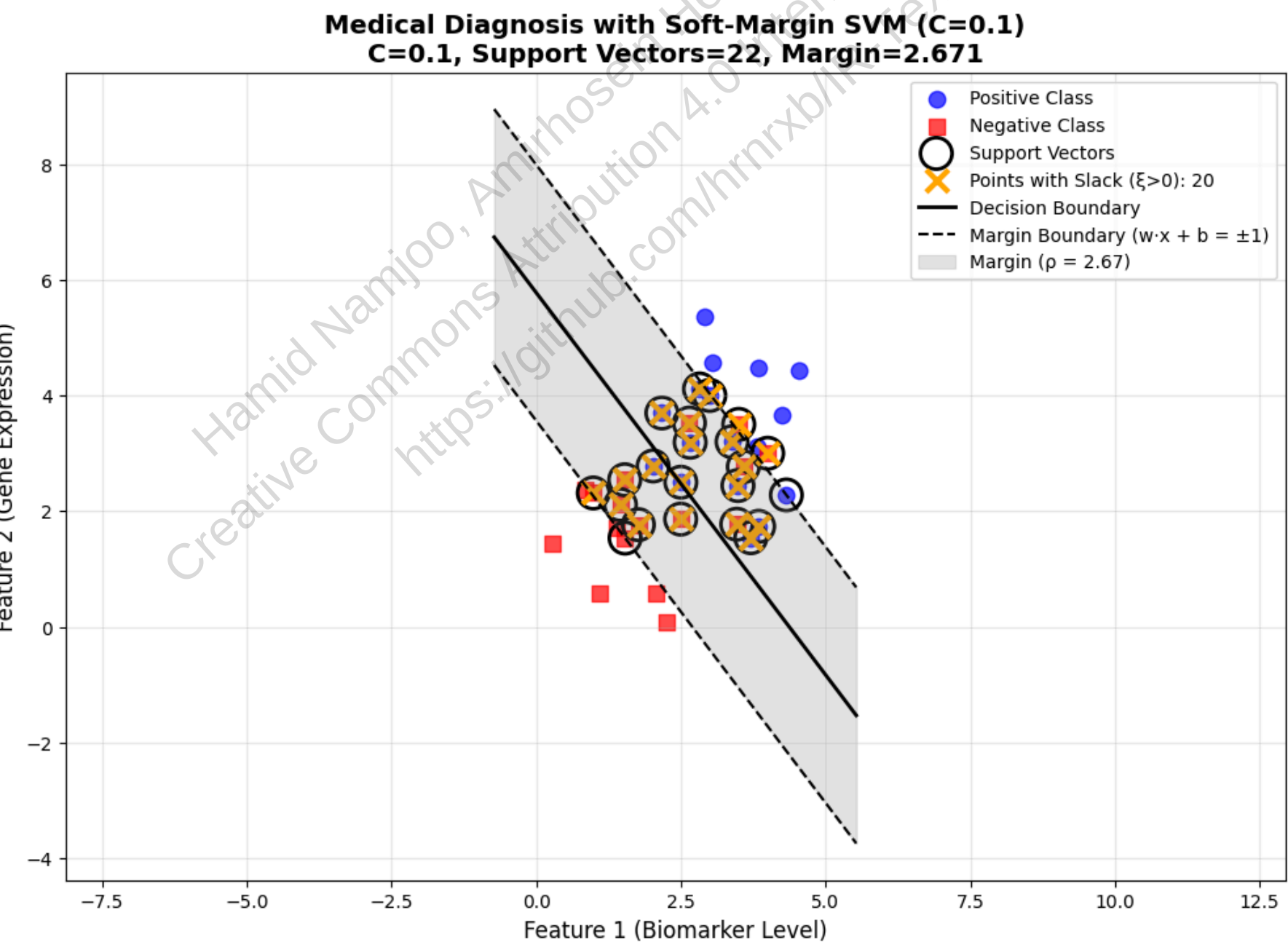
plt.tight_layout()
plt.show()
```

Soft-Margin SVM برای تشخیص پزشکی
داده حاوی نقاط همپوشانی (non-separable)

تعداد نمونه‌ها: 34
تعداد ویژگی‌ها: 2

C = 0.1 آموزش مدل با

بردار وزن: [0.4519, 0.5971]
بایاس: -2.6108
اندازه بردار وزن (||w||): 0.7488
حاشیه هندسی: 2.6710
تعداد بردارهای پشتیبان: 22
تعداد نقاط با لغزش: 20
%دقت روی داده آموزشی: 82.35

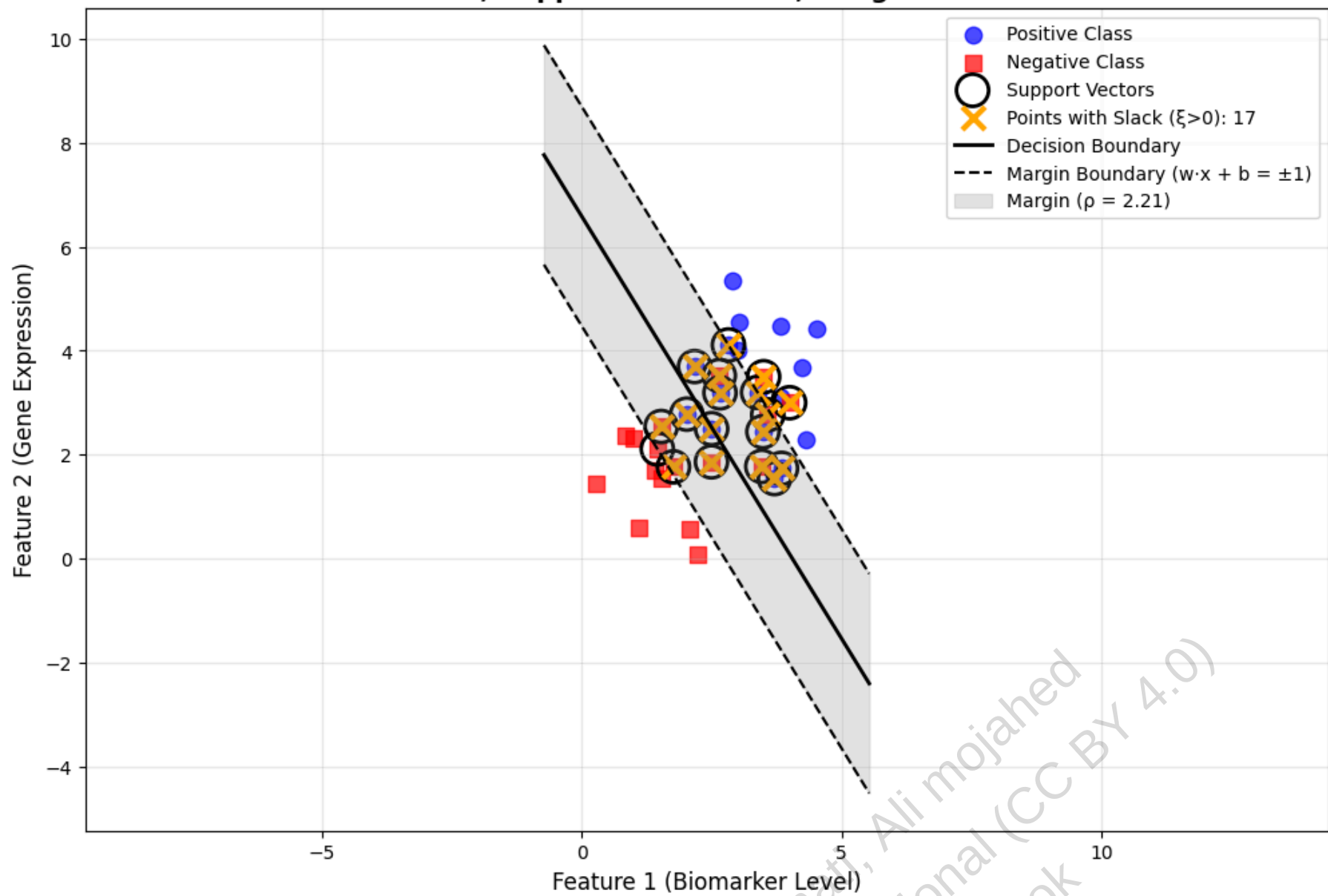


C = 1.0 آموزش مدل با

بردار وزن: [0.4734, 0.7699]
بایاس: -3.1200
اندازه بردار وزن (||w||): 0.9038
حاشیه هندسی: 2.2129
تعداد بردارهای پشتیبان: 18
تعداد نقاط با لغزش: 17
%دقت روی داده آموزشی: 79.41

Medical Diagnosis with Soft-Margin SVM (C=1.0)

C=1.0, Support Vectors=18, Margin=2.213



=====

C = 10.0 آموزش مدل با

=====

بردار وزن: [0.4919, 0.8001]

بایاس: -3.2815

اندازه بردار وزن ($\|w\|$): 0.9392

حاشیه هندسی: 2.1295

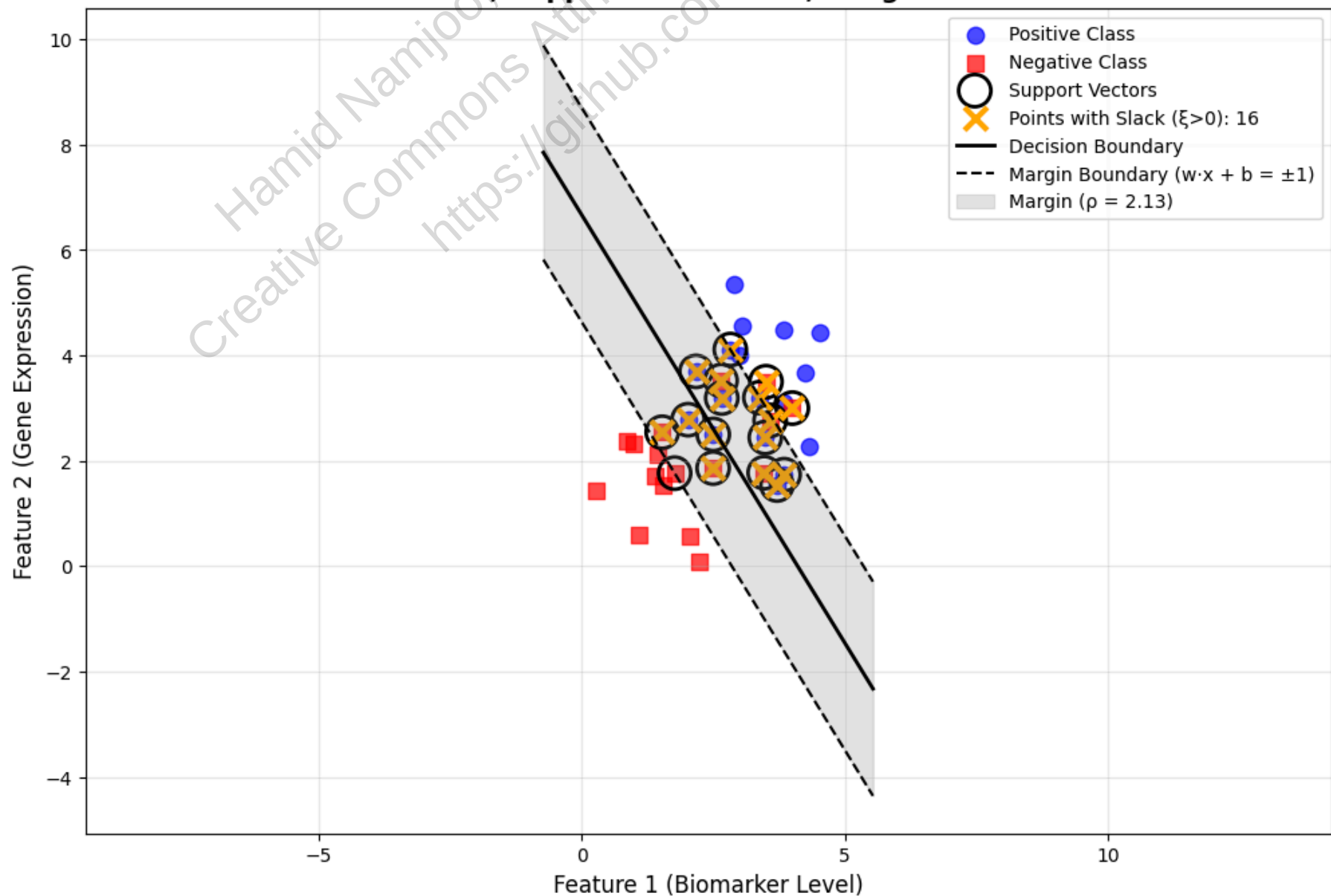
تعداد بردارهای پشتیبان: 17

تعداد نقاط با لغزش: 16

%دقت روی داده آموزشی: 79.41

Medical Diagnosis with Soft-Margin SVM (C=10.0)

C=10.0, Support Vectors=17, Margin=2.129



=====

C = 100.0 آموزش مدل با

=====

بردار وزن: [0.4920, 0.8001]

بایاس: -3.2816

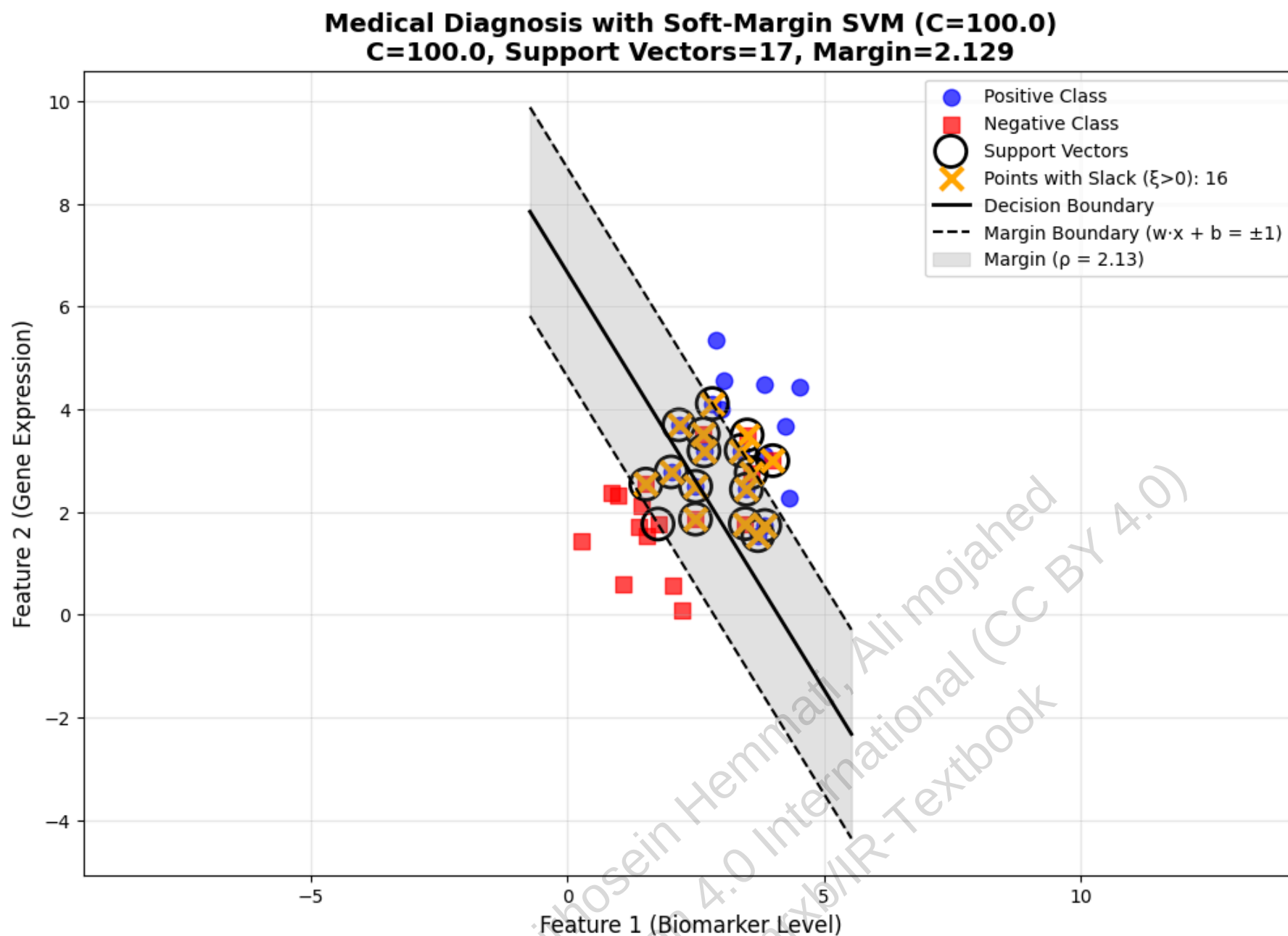
اندازه بردار وزن ($||w||$): 0.9393

حاشیه هندسی: 2.1294

تعداد بردارهای پشتیبان: 17

تعداد نقاط با لغزش: 16

%دقت روی داده آموزشی: 79.41



=====

Hard-Margin SVM مقایسه با

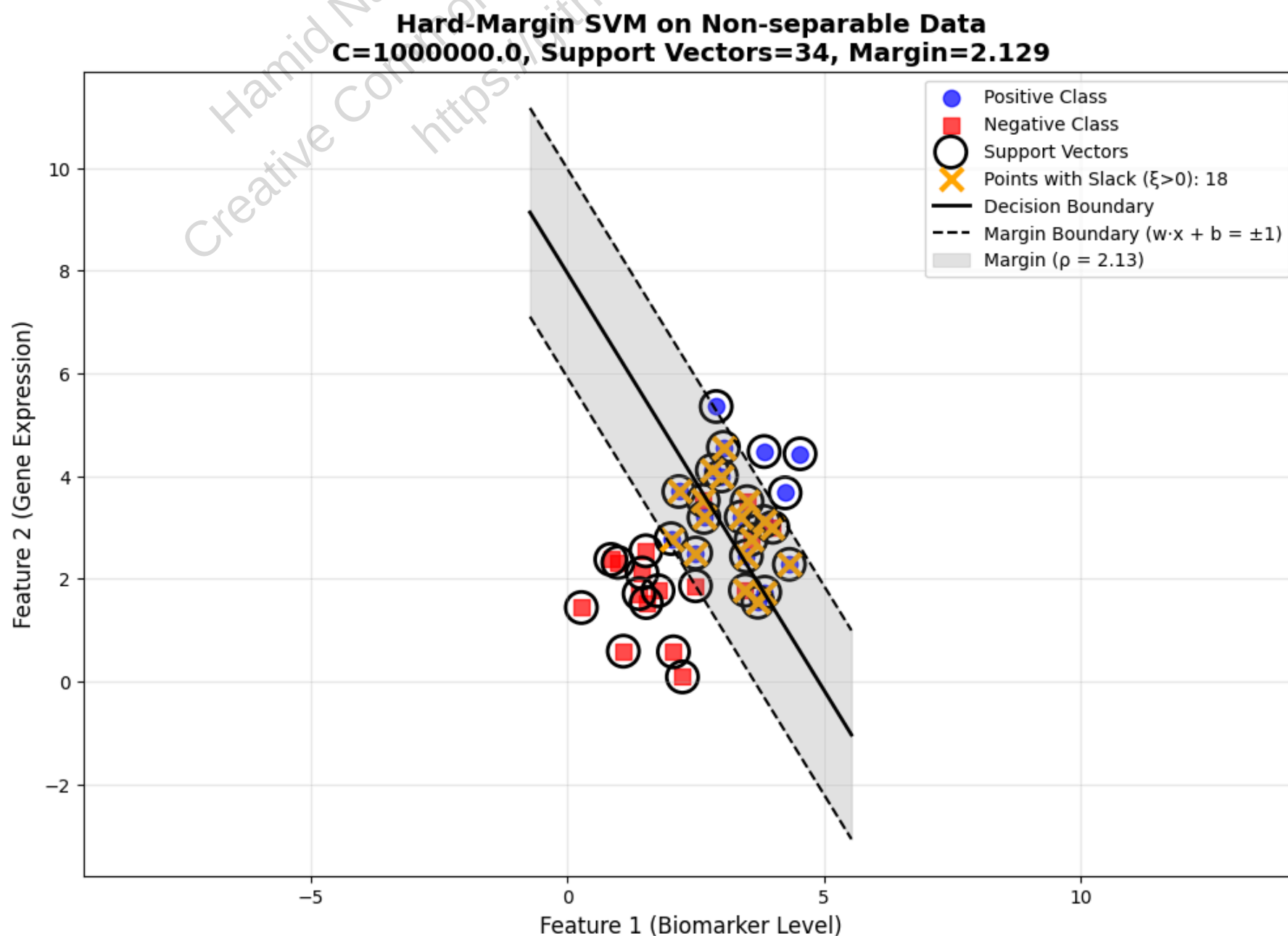
=====

تعداد بردارهای پشتیبان: 34 [Hard-Margin]

تعداد نقاط با لغزش: 18 [Hard-Margin]

حاشیه: 2.1293 [Hard-Margin]

%دقت: 76.47 [Hard-Margin]



=====

C تحلیل حساسیت به پارامتر

=====

دقت روی داده تست:

C= 0.1: 87.50 = %دقت

C= 1.0: 81.25 = %دقت

C= 10.0: 81.25 = %دقت

C= 100.0: 81.25 = %دقت

=====

تحلیل پیچیدگی محاسباتی

=====

و تعداد بردارهای پشتیبان C رابطه بین

C= 0.1: SVs=22, Slack Points=20

C= 1.0: SVs=18, Slack Points=17

C= 10.0: SVs=17, Slack Points=16

C= 100.0: SVs=17, Slack Points=16

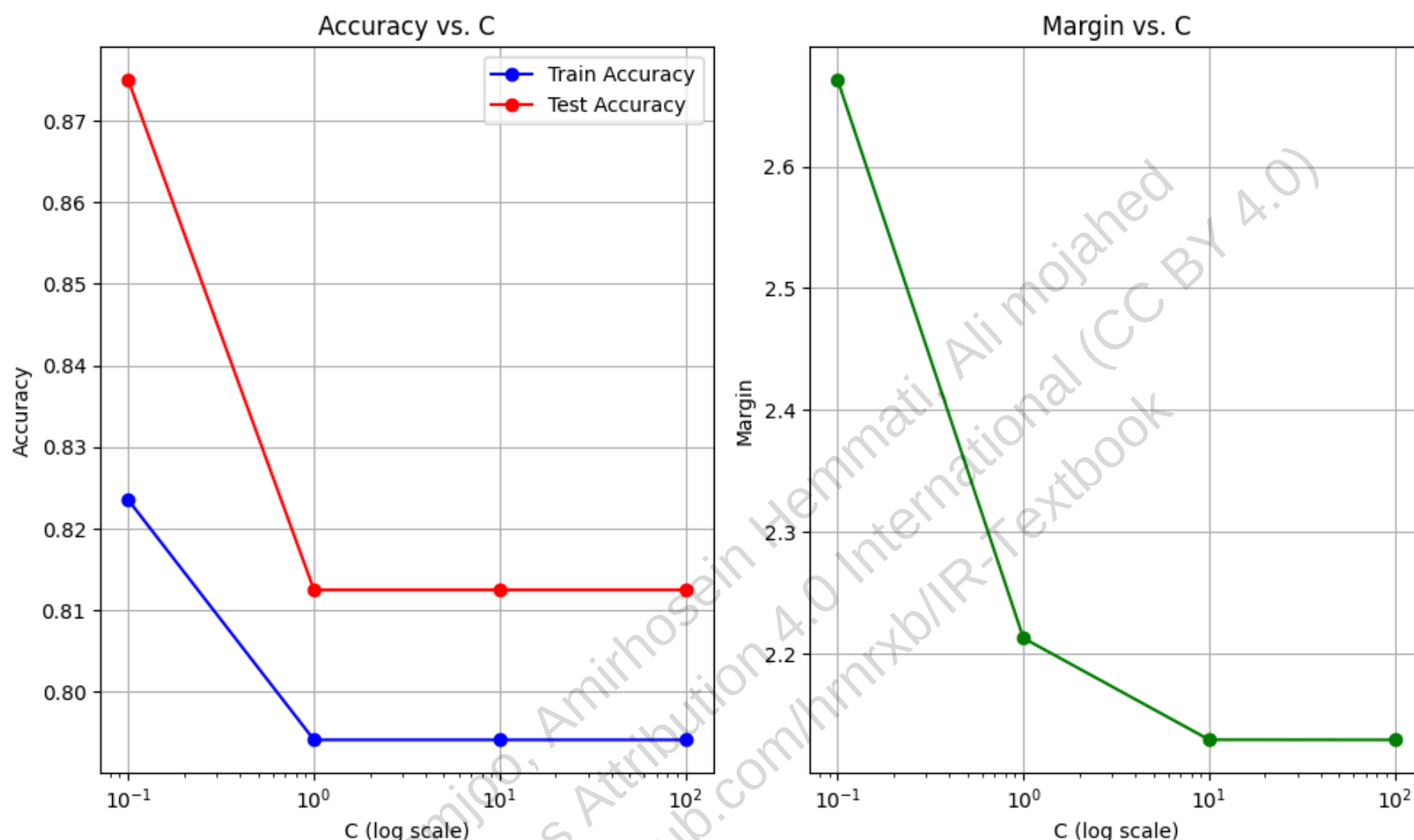
باید با دقت انتخاب شود C، نکته: در سیستم‌های پزشکی واقعی

- خیلی بزرگ: مدل پیچیده، زمان تست طولانی (چالش جدول 15.1) C
- خیلی کوچک: مدل ساده، ممکن است خطای بالا داشته باشد C

=====

بر روی حاشیه و دقت C تحلیل تأثیر

=====



15.2.2 ماشین‌های بردار پشتیبان چندکلاسه: تقسیم‌بندی هوشمند تصاویر

📷 گوگل فوتو: چگونه یک میلیارد تصویر را به صورت خودکار دسته‌بندی کنیم؟

تصور کنید در تیم توسعه گوگل فوتو کار می‌کنید و باید سیستمی طراحی کنید که به طور خودکار تصاویر کاربران را به دسته‌های مختلفی مانند "ساحل"، "غذا"، "حیوانات خانگی"، "خانواده" و... تقسیم کند. این یک چالش بزرگ در طبقه‌بندی چندکلاسه است، زیرا باید بتوانیم بین ده‌ها یا صدها کلاس مختلف تمایز قائل شویم.

SVMها به طور ذاتی طبقه‌بندهای دو کلاسه هستند، اما در دنیای واقعی با مسائل چندکلاسه روبرو هستیم. در گوگل فوتو، ما نمی‌توانیم فقط بگوییم "این تصویر ساحل است یا ساحل نیست"، بلکه باید بتوانیم از بین صدها گزینه، بهترین دسته را انتخاب کنیم.

روش سنتی برای انجام طبقه‌بندی چندکلاسه با SVMها استفاده از یکی از روش‌های بحث شده در بخش 14.5 است. به طور خاص، رایج‌ترین تکنیک در عمل، ساخت |C| طبقه‌بند یک-در-برابر-بقیه (معمولاً به عنوان "یک-در-برابر-همه" یا طبقه‌بندی OVA شناخته می‌شود) است.

🌀 روش یک-در-برابر-همه: از دو به چند

در مثال گوگل فوتو، این روش به این صورت عمل می‌کند:

- یک طبقه‌بند SVM برای تشخیص "ساحل در برابر غیرساحل"
- یک طبقه‌بند SVM برای تشخیص "غذا در برابر غیرغذا"
- یک طبقه‌بند SVM برای تشخیص "حیوانات خانگی در برابر غیرحیوانات خانگی"
- و به همین ترتیب برای هر کلاس...

سپس برای یک تصویر جدید، کلاسی انتخاب می‌شود که داده آزمون را با بیشترین حاشیه طبقه‌بندی می‌کند. این روش ساده است اما می‌تواند مشکلاتی ایجاد کند، به خصوص زمانی که کلاس‌ها نامتعادل هستند.

✂ روش یک-در-برابر-یک: مقایسه مستقیم

استراتژی دیگر، ساخت مجموعه‌ای از طبقه‌بندهای یک-در-برابر-یک است. در مثال گوگل فوتو:

- یک طبقه‌بند برای "ساحل در برابر غذا"
- یک طبقه‌بند برای "ساحل در برابر حیوانات خانگی"
- یک طبقه‌بند برای "غذا در برابر حیوانات خانگی"
- و به همین ترتیب برای هر جفت کلاس...

سپس کلاسی انتخاب می‌شود که توسط اکثر طبقه‌بندها انتخاب شده است. اگرچه این شامل ساخت $C(|C| - 1)/2$ طبقه‌بند است، اما زمان آموزش طبقه‌بندها ممکن است در واقع کاهش یابد، زیرا مجموعه داده آموزشی برای هر طبقه‌بند بسیار کوچکتر است.

با این حال، این رویکردها برای حل مسائل چندکلاسه بسیار زیبا نیستند. جایگزین بهتر توسط ساخت SVMهای چندکلاسه فراهم می‌شود، جایی که یک طبقه‌بند دو کلاسه روی یک بردار ویژگی $\Phi(\tilde{x}, y)$ مشتق شده از جفت شامل ویژگی‌های ورودی و کلاس داده می‌سازیم.

🔑 رویکرد یکپارچه: SVM چندکلاسه واقعی

در این روش پیشرفته‌تر، به جای ساخت چندین طبقه‌بند دو کلاسه، یک مدل یکپارچه ساخته می‌شود که همزمان تمام کلاس‌ها را در نظر می‌گیرد. در مثال گوگل فوتو:

- یک مدل واحد که ویژگی‌های تصویر را با هر کلاس ممکن ترکیب می‌کند
 - در زمان آزمون، طبقه‌بند کلاس $y = \arg \max_{y'} w^{\sim T} \Phi(\tilde{x}, y')$ را انتخاب می‌کند
 - حاشیه در طول آموزش، شکاف بین این مقدار برای کلاس صحیح و نزدیک‌ترین کلاس دیگر است
- این روش کلی را می‌توان برای ارائه فرمولاسیون چندکلاسه از انواع مختلف طبقه‌بندهای خطی گسترش داد.

این روش همچنین یک نمونه ساده از تعمیم طبقه‌بندی است که در آن کلاس‌ها فقط مجموعه‌ای از برچسب‌های دسته‌بندی مستقل نیستند، بلکه ممکن است اشیاء ساختاریافته با روابط تعریف شده بین آنها باشند. در دنیای SVM، چنین کارهایی تحت برچسب **SVMهای ساختاری** قرار می‌گیرند. ما در بخش 15.4.2 دوباره به آنها اشاره می‌کنیم.

🌐 کاربرد در دنیای واقعی: طبقه‌بندی محتوای یوتیوب

یوتیوب از الگوریتم‌های طبقه‌بندی چندکلاسه پیشرفته برای دسته‌بندی میلیون‌ها ویدیو استفاده می‌کند. این سیستم باید بتواند ویدیوها را به دسته‌هایی مانند "موسیقی"، "ورزشی"، "آموزشی"، "سرگرمی" و... تقسیم کند.

در این کاربرد، روش یک-در-برابر-همه ممکن است برای دسته‌های اصلی استفاده شود، در حالی که برای زیرشاخه‌های خاص (مانند "فوتبال" در برابر "بسکتبال" در دسته ورزشی) از روش یک-در-برابر-یک استفاده می‌شود. این رویکرد ترکیبی به یوتیوب اجازه می‌دهد تا همزمان دقت و سرعت را بهینه کند.

```
In [25]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs, load_digits
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from cvxopt import matrix, solvers
import time
```

```
# =====
# QP از پایه با) دوکلاسه اصلی SVM پیاده‌سازی
# این کلاس پایه تمام روش‌های چندکلاسه استفاده می‌کند
# =====
```

```
class BinarySVM_QP:
    """
```

```
    دوکلاسه با بهینه‌سازی درجه دو SVM پیاده‌سازی سخت‌افزاری
    این کلاس توسط روش‌های چندکلاسه استفاده می‌شود
    """
```

```
    def __init__(self, C=1.0):
        self.C = C
        self.w = None
        self.b = None
        self.alpha = None
        self.support_vectors = None
```

```
    def fit(self, X, y):
        n_samples = X.shape[0]
```

```
        # محاسبه ماتریس P:  $P_{ij} = y_i * y_j * \text{dot}(x_i, x_j)$ 
        P = np.outer(y, y) * np.dot(X, X.T)
        P = matrix(P)
        q = matrix(-np.ones(n_samples))
```

```
        # محدودیت:  $-\alpha_i \leq 0$  و  $\alpha_i \leq C$ 
        G = np.vstack([-np.eye(n_samples), np.eye(n_samples)])
        h = np.hstack([np.zeros(n_samples), self.C * np.ones(n_samples)])
        G, h = matrix(G), matrix(h)
```

```
        # محدودیت:  $\sum \alpha_i * y_i = 0$ 
        A = matrix(y.reshape(1, -1).astype(float))
        b = matrix(0.0)
```

```
        solvers.options['show_progress'] = False
        solution = solvers.qp(P, q, G, h, A, b)
```

```
        self.alpha = np.array(solution['x']).flatten()
```

```
        # شناسایی بردارهای پشتیبان
        sv_indices = self.alpha > 1e-5
        self.support_vectors = X[sv_indices]
        sv_y = y[sv_indices]
        sv_alpha = self.alpha[sv_indices]
```

```
        # محاسبه  $w = \sum \alpha_i * y_i * x_i$ 
        self.w = np.sum(sv_alpha.reshape(-1, 1) * sv_y.reshape(-1, 1) * self.support_vectors, axis=0)
```

```
        # محاسبه b از یک SV  $0 < \alpha_i < C$ 
        margin_sv = (self.alpha > 1e-5) & (self.alpha < self.C - 1e-5)
        if np.any(margin_sv):
            idx = np.where(margin_sv)[0][0]
            self.b = y[idx] - np.dot(self.w, X[idx])
        else:
            self.b = 0
```

```
    def predict(self, X):
        return np.sign(np.dot(X, self.w) + self.b)
```

```
    def decision_function(self, X):
        """فاصله از ابرصفحه (برای انتخاب کلاس با حاشیه بیشتر)"""
        return np.dot(X, self.w) + self.b
```

```
# =====
# OVA) چندکلاسه با رویکرد یک-در-برابر-همه SVM پیاده‌سازی
# =====
```

```
class OneVsAllSVM:
```

```
    """
    SVM (One-vs-All) چندکلاسه با رویکرد یک-در-برابر-همه
    همه بقیه] vs دوکلاسه می‌سازد: [کلاس] SVM برای هر کلاس، یک
    """
```

```
    def __init__(self, C=1.0):
        self.C = C
```

```

self.classifiers = {} # کلاس -> classifier
self.classes = None

def fit(self, X, y):
    """
    برای هر کلاس classifier آموزش یک
    تعداد کلاس‌هاست |C| که  $O(|C| * N^3)$  پیچیدگی
    """
    self.classes = np.unique(y)
    training_times = {}

    for cls in self.classes:
        print(f" همه... vs {cls} برای کلاس classifier آموزش")
        start_time = time.time()

        # ساخت داده دوکلاسه: 1 برای این کلاس، -1 برای بقیه
        binary_y = np.where(y == cls, 1, -1)

        # دوکلاسه SVM آموزش
        classifier = BinarySVM_QP(C=self.C)
        classifier.fit(X, binary_y)

        self.classifiers[cls] = classifier
        training_times[cls] = time.time() - start_time

    return training_times

def predict(self, X, method='score'):
    """
    پیش‌بینی با دو روش:
    - 'score': decision function value انتخاب کلاس با بیشترین
    - 'vote': decision function value انتخاب کلاس با بیشترین
    """
    n_samples = X.shape[0]
    n_classes = len(self.classes)

    if method == 'score':
        # محاسبه امتیاز برای هر کلاس
        scores = np.zeros((n_samples, n_classes))

        for i, cls in enumerate(self.classes):
            scores[:, i] = self.classifiers[cls].decision_function(X)

        # انتخاب کلاس با بیشترین امتیاز
        max_indices = np.argmax(scores, axis=1)
        return np.array([self.classes[i] for i in max_indices])

    elif method == 'vote':
        # رأی‌گیری بر اساس مقدار تابع تصمیم
        votes = np.zeros((n_samples, n_classes))

        for i, cls in enumerate(self.classes):
            clf = self.classifiers[cls]
            # استفاده مستقیم از مقدار تابع تصمیم برای رأی‌گیری
            votes[:, i] = clf.decision_function(X)

        max_indices = np.argmax(votes, axis=1)
        return np.array([self.classes[i] for i in max_indices])

# =====
# (OVO) چندکلاسه با رویکرد یک-در-برابر یک SVM پیاده‌سازی
# =====

class OneVsOneSVM:
    """
    (One-vs-One) چندکلاسه با رویکرد یک-در-برابر یک SVM
    دوکلاسه می‌سازد SVM برای هر جفت کلاس، یک
    """
    def __init__(self, C=1.0):
        self.C = C
        self.classifiers = {} # (cls1, cls2) -> classifier
        self.classes = None

    def fit(self, X, y):
        """
        برای هر جفت کلاس classifier آموزش
        روی داده کمتری آموزش می‌بیند classifier اما هر  $O(|C|^2 * (N/|C|)^3) = O(|C| * N^3)$  پیچیدگی
        """
        self.classes = np.unique(y)
        training_times = {}

        # ساخت تمام ترکیب‌های جفت کلاس‌ها
        for i, cls1 in enumerate(self.classes):
            for j, cls2 in enumerate(self.classes):
                if i < j:
                    print(f" {cls1} vs {cls2}... برای classifier آموزش")
                    start_time = time.time()

                    # فیلتر کردن داده فقط برای این دو کلاس
                    mask = (y == cls1) | (y == cls2)
                    X_pair = X[mask]
                    y_pair = y[mask]

                    # ساخت برچسب‌های دو کلاسه: 1 برای cls1، -1 برای cls2
                    binary_y = np.where(y_pair == cls1, 1, -1)

                    # دوکلاسه SVM آموزش
                    classifier = BinarySVM_QP(C=self.C)
                    classifier.fit(X_pair, binary_y)

                    self.classifiers[(cls1, cls2)] = classifier
                    training_times[(cls1, cls2)] = time.time() - start_time

```

```

        return training_times

def predict(self, X):
    """
    رأی می‌دهد، کلاس با بیشترین آرا انتخاب می‌شود classifier رأی‌گیری اکثریت: هر
    """

    n_samples = X.shape[0]
    votes = np.zeros((n_samples, len(self.classes)))

    for (cls1, cls2), classifier in self.classifiers.items():
        # دوکلاس classifier پیش‌بینی با
        predictions = classifier.predict(X)

        # تبدیل پیش‌بینی‌ها به رأی برای کلاس‌های اصلی
        idx1 = np.where(self.classes == cls1)[0][0]
        idx2 = np.where(self.classes == cls2)[0][0]

        votes[predictions == 1, idx1] += 1
        votes[predictions == -1, idx2] += 1

    # انتخاب کلاس با بیشترین آرا
    max_indices = np.argmax(votes, axis=1)
    return np.array([self.classes[i] for i in max_indices])

# =====
# تست و مقایسه روش‌ها روی داده مصنوعی (شبیه‌سازی تصویر)
# =====

print("="*60)
print("Multiclass SVM: Comparison of OVA vs OVO")
print("Simulating Image Classification (3 classes)")
print("="*60)

# (دو ویژگی از تصویر extraction مانند) برای نمایش D ایجاد داده 2
np.random.seed(42)
X, y = make_blobs(n_samples=300, centers=3, n_features=2,
                  cluster_std=1.5, random_state=42)

# train و test تقسیم داده به
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

print(f"Training samples: {len(X_train)}")
print(f"Test samples: {len(X_test)}")
print(f"Number of classes: {len(np.unique(y_train))}")

# =====
# OVA آموزش مدل
# =====

print("\n" + "-"*40)
print("Training One-vs-All SVM")
print("-"*40)

ova_model = OneVsAllSVM(C=1.0)
ova_train_times = ova_model.fit(X_train, y_train)

print(f"Number of classifiers: {len(ova_model.classifiers)}")

# =====
# OVO آموزش مدل
# =====

print("\n" + "-"*40)
print("Training One-vs-One SVM")
print("-"*40)

ovo_model = OneVsOneSVM(C=1.0)
ovo_train_times = ovo_model.fit(X_train, y_train)

print(f"Number of classifiers: {len(ovo_model.classifiers)}")

# =====
# ارزیابی و مقایسه
# =====

print("\n" + "-"*40)
print("Evaluation Results")
print("-"*40)

# ارزیابی OVA
ova_pred_test = ova_model.predict(X_test, method='score')
ova_pred_train = ova_model.predict(X_train, method='score')
ova_test_acc = accuracy_score(y_test, ova_pred_test)
ova_train_acc = accuracy_score(y_train, ova_pred_train)

print(f"One-vs-All SVM:")
print(f"  Train Accuracy: {ova_train_acc:.4f}")
print(f"  Test Accuracy: {ova_test_acc:.4f}")

# ارزیابی OVO
ovo_pred_test = ovo_model.predict(X_test)
ovo_pred_train = ovo_model.predict(X_train)
ovo_test_acc = accuracy_score(y_test, ovo_pred_test)
ovo_train_acc = accuracy_score(y_train, ovo_pred_train)

print(f"\nOne-vs-One SVM:")
print(f"  Train Accuracy: {ovo_train_acc:.4f}")
print(f"  Test Accuracy: {ovo_test_acc:.4f}")

# (دقت کمی بالاتر دارد OVO معمولاً) مقایسه دقت
print(f"\nDifference (OVO - OVA): {ovo_test_acc - ova_test_acc:.4f}")

```

```
# =====
# تحلیل پیچیدگی محاسباتی (طبق جدول 15.1)
# =====

print("\n" + "="*40)
print("Computational Complexity Analysis")
print("="*40)

# محاسبه زمان آموزش کل
ova_total_time = sum(ova_train_times.values())
ovo_total_time = sum(ovo_train_times.values())

print(f"Total Training Time OVA: {ova_total_time:.4f} sec")
print(f"Total Training Time OVO: {ovo_total_time:.4f} sec")

# محاسبه تعداد بردارهای پشتیبان برای هر روش
ova_total_sv = sum(len(clf.support_vectors) for clf in ova_model.classifiers.values())
ovo_total_sv = sum(len(clf.support_vectors) for clf in ovo_model.classifiers.values())
avg_sv_per_classifier_ova = ova_total_sv / len(ova_model.classifiers)
avg_sv_per_classifier_ovo = ovo_total_sv / len(ovo_model.classifiers)

print(f"\nSupport Vector Analysis:")
print(f"  OVA - Total SVs: {ova_total_sv}, Avg per classifier: {avg_sv_per_classifier_ova:.1f}")
print(f"  OVO - Total SVs: {ovo_total_sv}, Avg per classifier: {avg_sv_per_classifier_ovo:.1f}")

# =====
# نمایش تصمیم‌گیری برای هر روش
# =====

def plot_multiclass_decision_boundary(model, X, y, title=""):
    """ترسیم مرزهای تصمیم‌گیری برای مسائل چندکلاسه"""
    plt.figure(figsize=(12, 8))

    # ترسیم نقاط داده
    scatter = plt.scatter(X[:, 0], X[:, 1], c=y, cmap='viridis', s=60, edgecolors='k', alpha=0.7)

    # ایجاد شبکه برای مناطق تصمیم‌گیری
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.1),
                          np.arange(y_min, y_max, 0.1))

    # پیش‌بینی برای شبکه
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # ترسیم مناطق تصمیم‌گیری
    plt.contourf(xx, yy, Z, alpha=0.3, cmap='viridis')

    plt.xlabel('Feature 1 (e.g., Color Histogram)', fontsize=12)
    plt.ylabel('Feature 2 (e.g., Texture)', fontsize=12)
    plt.title(title, fontsize=14, fontweight='bold')
    plt.colorbar(scatter, label='Class')
    plt.grid(True, alpha=0.3)
    plt.axis('equal')
    plt.show()

# ترسیم نتایج
plot_multiclass_decision_boundary(ova_model, X_train, y_train,
                                title=f"OVA Decision Boundaries (C=1.0)")

plot_multiclass_decision_boundary(ovo_model, X_train, y_train,
                                title=f"OVO Decision Boundaries (C=1.0)")

# =====
# (Digits Dataset) تست روی داده واقعی: دیتاست ارقام
# از ارقام 0-9 است ۸x۸ این دیتاست شامل تصاویر 8
# =====

print("\n" + "="*60)
print("Test on Real Dataset: Digits Classification (10 classes)")
print("="*60)

# بارگذاری داده ارقام
digits = load_digits()
X_digits = digits.data # ویژگی 64
y_digits = digits.target

# کاهش داده برای سرعت بیشتر (استفاده از 300 نمونه)
subset_indices = np.random.choice(len(X_digits), 300, replace=False)
X_subset = X_digits[subset_indices]
y_subset = y_digits[subset_indices]

# تقسیم داده
X_train_d, X_test_d, y_train_d, y_test_d = train_test_split(
    X_subset, y_subset, test_size=0.3, random_state=42
)

print(f"Training samples: {len(X_train_d)}")
print(f"Features per sample: {X_train_d.shape[1]}")
print(f"Classes: {np.unique(y_train_d)}")

# روی داده ارقام OVA آموزش
print("\nTraining OVA on Digits...")
ova_digits = OneVsAllSVM(C=1.0)
ova_digits.fit(X_train_d, y_train_d)
ova_pred_d = ova_digits.predict(X_test_d)
ova_acc_d = accuracy_score(y_test_d, ova_pred_d)
print(f"OVA Accuracy on Digits: {ova_acc_d:.4f}")

# روی داده ارقام OVO آموزش
print("\nTraining OVO on Digits...")
```



```
ovo_digits = OneVsOneSVM(C=1.0)
ovo_digits.fit(X_train_d, y_train_d)
ovo_pred_d = ovo_digits.predict(X_test_d)
ovo_acc_d = accuracy_score(y_test_d, ovo_pred_d)
print(f"OVO Accuracy on Digits: {ovo_acc_d:.4f}")

print(f"\nDifference (OVO - OVA): {ovo_acc_d - ova_acc_d:.4f}")

# =====
# تحلیل نهایی: کدام روش برای گوگل فوتو مناسبتر است؟
# =====

print("\n" + "="*60)
print("کدام استراتژی برای گوگل فوتو مناسبتر است؟")
print("="*60)

print("بر اساس تحلیل پیچیدگی از جدول 15.1")
print("• OVA: |C| N دسته‌بند، هر کدام روی |C|")
print("• OVO: |C|(|C|-1)/2 2 تقریباً هر کدام روی تقریباً N/|C| دسته‌بند، هر")
print("• دسته |C|=100 برای گوگل فوتو با")
print("  - OVA: 100 دسته‌بند")
print("  - OVO: 4950 دسته‌بند!")
print()
print("توصیه عملی:")
print("• برای سیستم‌های بزرگ با مقیاس بزرگ (دسته‌بندهای کمتر) استفاده کنید OVA از")
print("• زمانی استفاده کنید که دقت بالا حیاتی است و کلاس‌ها متوازن هستند OVO از")
print("• در زمان پیش‌بینی سریعتر است OVA، برای سیستم‌های بی‌درنگ")
print("• ممکن است دقیق‌تر باشد اما اثر حافظه بیشتری دارد OVO")
print("ری داشته باشد. این امر به دلیل ماهیت خاص داده یا پیاده‌سازی متدهای رأی‌گیری می باشد OVO عمل کرده است، در حالی که معمولاً انتظار می‌رود OVO کمی بهتر از OVA، نکته قابل توجه این است که در دیتاست ارقام"
```

```
=====
Multiclass SVM: Comparison of OVA vs OVO
Simulating Image Classification (3 classes)
=====
Training samples: 210
Test samples: 90
Number of classes: 3

-----
Training One-vs-All SVM
-----
... همه vs برای کلاس 0 classifier آموزش
... همه vs برای کلاس 1 classifier آموزش
... همه vs برای کلاس 2 classifier آموزش
Number of classifiers: 3

-----
Training One-vs-One SVM
-----
... vs 1 برای classifier 0 آموزش
... vs 2 برای classifier 0 آموزش
... vs 2 برای classifier 1 آموزش
Number of classifiers: 3

=====
Evaluation Results
=====
One-vs-All SVM:
  Train Accuracy: 1.0000
  Test Accuracy: 1.0000

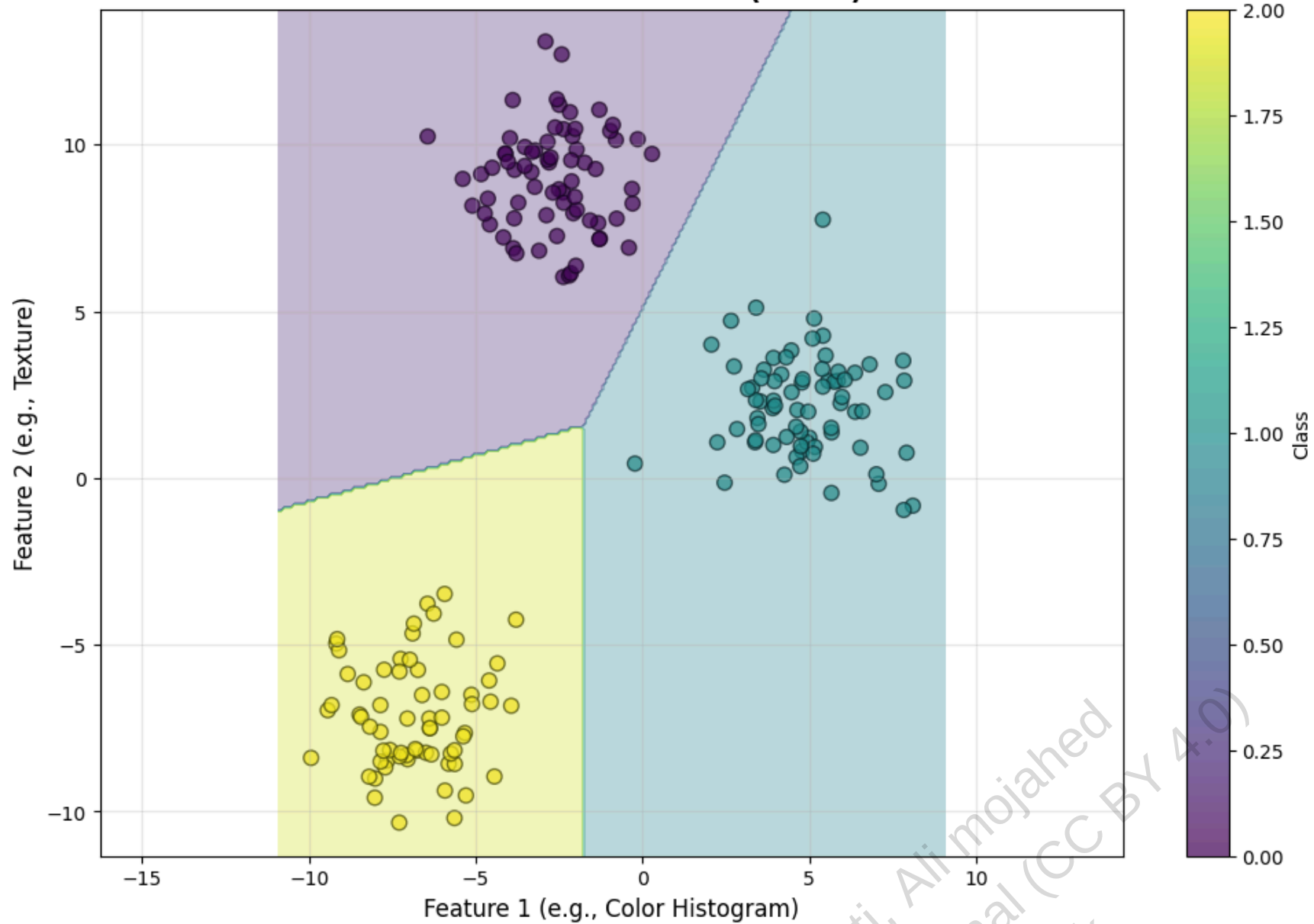
One-vs-One SVM:
  Train Accuracy: 1.0000
  Test Accuracy: 1.0000

Difference (OVO - OVA): 0.0000

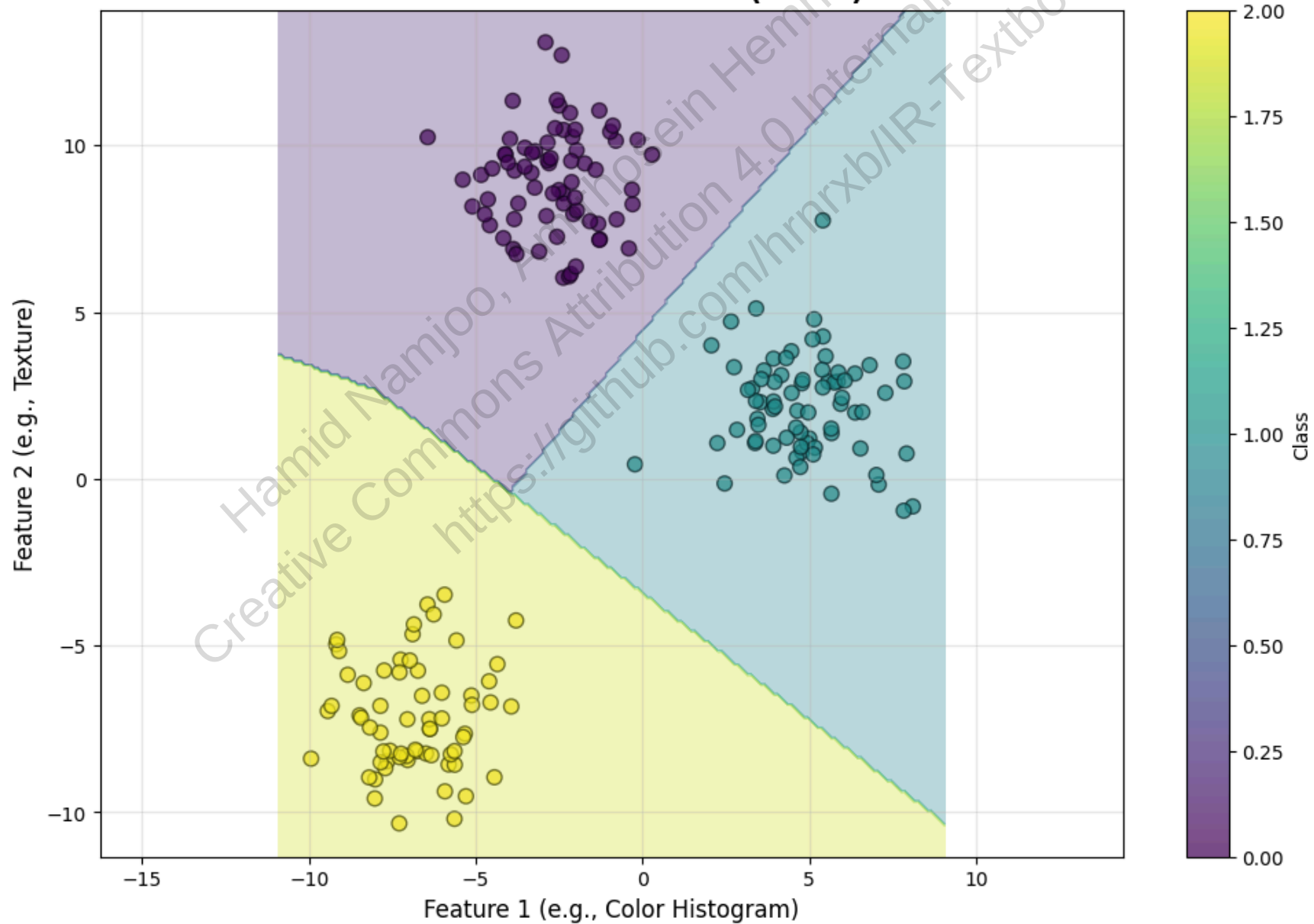
=====
Computational Complexity Analysis
=====
Total Training Time OVA: 0.2786 sec
Total Training Time OVO: 0.1764 sec

Support Vector Analysis:
  OVA - Total SVs: 8, Avg per classifier: 2.7
  OVO - Total SVs: 8, Avg per classifier: 2.7
```

OVA Decision Boundaries (C=1.0)



OVO Decision Boundaries (C=1.0)



```
=====
Test on Real Dataset: Digits Classification (10 classes)
=====
Training samples: 210
Features per sample: 64
Classes: [0 1 2 3 4 5 6 7 8 9]
```

```
Training OVA on Digits...
همه... vs برای کلاس 0 classifier آموزش
همه... vs برای کلاس 1 classifier آموزش
همه... vs برای کلاس 2 classifier آموزش
همه... vs برای کلاس 3 classifier آموزش
همه... vs برای کلاس 4 classifier آموزش
همه... vs برای کلاس 5 classifier آموزش
همه... vs برای کلاس 6 classifier آموزش
همه... vs برای کلاس 7 classifier آموزش
همه... vs برای کلاس 8 classifier آموزش
همه... vs برای کلاس 9 classifier آموزش
OVA Accuracy on Digits: 0.9000
```

```
Training OVO on Digits...
1... vs برای 0 classifier آموزش
2... vs برای 0 classifier آموزش
3... vs برای 0 classifier آموزش
4... vs برای 0 classifier آموزش
5... vs برای 0 classifier آموزش
6... vs برای 0 classifier آموزش
7... vs برای 0 classifier آموزش
8... vs برای 0 classifier آموزش
9... vs برای 0 classifier آموزش
2... vs برای 1 classifier آموزش
3... vs برای 1 classifier آموزش
4... vs برای 1 classifier آموزش
5... vs برای 1 classifier آموزش
6... vs برای 1 classifier آموزش
7... vs برای 1 classifier آموزش
8... vs برای 1 classifier آموزش
9... vs برای 1 classifier آموزش
3... vs برای 2 classifier آموزش
4... vs برای 2 classifier آموزش
5... vs برای 2 classifier آموزش
6... vs برای 2 classifier آموزش
7... vs برای 2 classifier آموزش
8... vs برای 2 classifier آموزش
9... vs برای 2 classifier آموزش
4... vs برای 3 classifier آموزش
5... vs برای 3 classifier آموزش
6... vs برای 3 classifier آموزش
7... vs برای 3 classifier آموزش
8... vs برای 3 classifier آموزش
9... vs برای 3 classifier آموزش
5... vs برای 4 classifier آموزش
6... vs برای 4 classifier آموزش
7... vs برای 4 classifier آموزش
8... vs برای 4 classifier آموزش
9... vs برای 4 classifier آموزش
6... vs برای 5 classifier آموزش
7... vs برای 5 classifier آموزش
8... vs برای 5 classifier آموزش
9... vs برای 5 classifier آموزش
7... vs برای 6 classifier آموزش
8... vs برای 6 classifier آموزش
9... vs برای 6 classifier آموزش
8... vs برای 7 classifier آموزش
9... vs برای 7 classifier آموزش
9... vs برای 8 classifier آموزش
```

OVO Accuracy on Digits: 0.8889

Difference (OVO - OVA): -0.0111

```
=====
کدام استراتژی برای گوگل فوتو مناسبتر است؟
=====
بر اساس تحلیل پیچیدگی از جدول 15.1:
• OVA:  $|C|$  هر کدام روی N دسته‌بند، استفاده کنید
• OVO:  $|C|N/2$  تقریباً  $(|C|-1)/2$  هر کدام روی N دسته‌بند، استفاده کنید
• دسته  $|C|=100$  برای گوگل فوتو با:
  - OVA: 100 دسته‌بند
  - OVO: 4950 دسته‌بند
```

توصیه عملی:

- برای سیستم‌های بزرگ با مقیاس بزرگ (دسته‌بندهای کمتر) استفاده کنید OVA از
- زمانی استفاده کنید که دقت بالا حیاتی است و کلاس‌ها متوازن هستند OVO از
- در زمان پیش‌بینی سریع‌تر است OVA، برای سیستم‌های بی‌درنگ
- ممکن است دقیق‌تر باشد اما اثر حافظه بیشتری دارد OVO

دقت بالاتری داشته باشد. این امر به دلیل ماهیت خاص داده یا پیادسازی متدهای رای‌گیری می باشد OVO عمل کرده است، در حالی که معمولاً انتظار می‌رود OVO کمی بهتر از OVA، نکته قابل توجه این است که در دیتاست ارقام

15.2.3 ماشین‌های بردار پشتیبان غیرخطی: عبور از مرزهای خطی

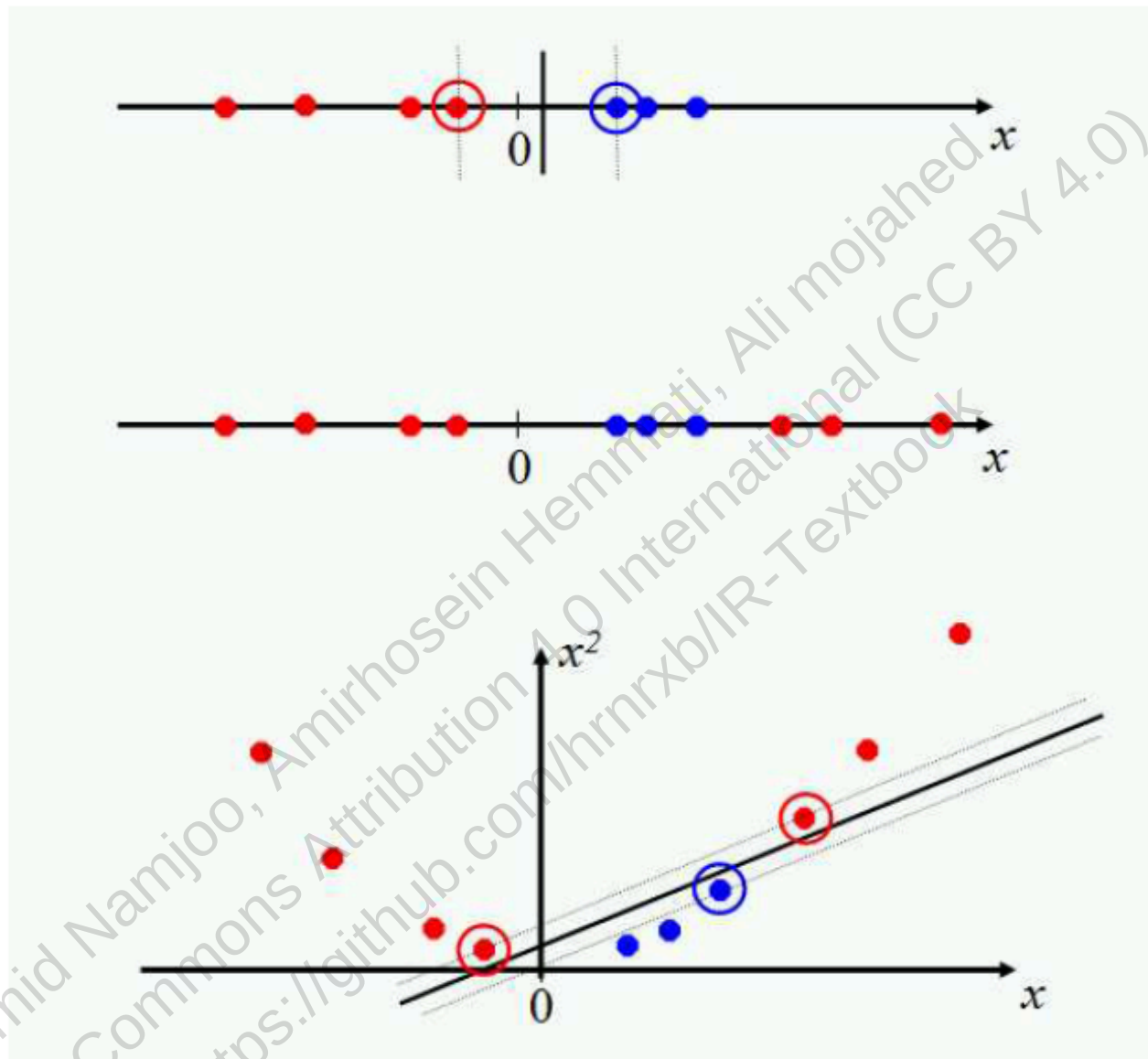
اینستاگرام: چگونه پیچیدگی‌های بصری را درک کنیم؟

تصور کنید در تیم فنی اینستاگرام کار می‌کنید و باید سیستمی طراحی کنید که چهره‌ها را در تصاویر مختلف شناسایی کند. یک چهره از زوایای مختلف، با نورپردازی‌های متفاوت و با حالات گوناگون (خندان، عصبانی، با عینک، بدون عینک) دیده

می‌شود. در فضای ویژگی‌های اصلی، این داده‌ها به هیچ وجه خطی جداپذیر نیستند - نمی‌توان یک خط مستقیم کشید که چهره‌ها را از غیرچهره‌ها جدا کند.

اینجاست که مفهوم **کرنل** (kernel trick) اهمیت پیدا می‌کند. به جای تلاش برای جدا کردن داده‌ها در فضای فعلی، ما داده‌ها را به فضایی با ابعاد بالاتر "منتقل" می‌کنیم که در آن جداپذیری خطی ممکن است.

با آنچه تاکنون ارائه شده است، مجموعه داده‌هایی که به صورت خطی جداپذیر هستند (شاید با چند استثنا یا برخی نویزها) به خوبی مدیریت می‌شوند. اما اگر مجموعه داده به سادگی اجازه طبقه‌بندی توسط یک طبقه‌بند خطی را ندهد، چه کار خواهیم کرد؟ بیا ببینیم به یک حالت یک بعدی نگاه کنیم. مجموعه داده بالایی در شکل 15.6 به راحتی توسط یک طبقه‌بند خطی طبقه‌بندی می‌شود، اما مجموعه داده میانی نه. در عوض، ما باید بتوانیم یک بازه را انتخاب کنیم.



► Figure 15.6 Projecting data that is not linearly separable into a higher dimensional space can make it linearly separable.

کرنل تریک: تبدیل فضایی بدون محاسبات سنگین

یک راه برای حل این مشکل، نگاشت داده‌ها به یک فضای با ابعاد بالاتر و سپس استفاده از یک طبقه‌بند خطی در فضای با ابعاد بالاتر است. برای مثال، بخش پایین شکل نشان می‌دهد که یک جداکننده خطی می‌تواند به راحتی داده‌ها را طبقه‌بندی کند اگر از یک تابع درجه دوم برای نگاشت داده‌ها به دو بعد استفاده کنیم.

در مثال شناسایی چهره اینستاگرام، کرنل به ما اجازه می‌دهد ویژگی‌های پیچیده‌ای مانند "فاصله بین چشم‌ها" یا "انحنای لب‌ها" را بدون محاسبه مستقیم آن‌ها مدل کنیم. این ویژگی‌ها در فضای با ابعاد بالاتر به صورت خطی قابل جدا شدن هستند.

SVMها، و همچنین تعدادی از طبقه‌بندهای خطی دیگر، راهی آسان و کارآمد برای انجام این نگاشت به فضای با ابعاد بالاتر ارائه می‌دهند که به آن (kernel trick) گفته می‌شود. این واقعاً یک ترفند نیست: فقط از ریاضیاتی که دیده‌ایم بهره می‌برد. طبقه‌بند

خطی SVM به یک محصول نقطه بین بردارهای نقاط داده متکی است. بیابید $K(\tilde{x}_i, \tilde{x}_j) = \tilde{x}_i^T \tilde{x}_j$. سپس طبقه‌بندی که تاکنون دیده‌ایم به صورت زیر است:

$$f(\tilde{x}) = \text{sign} \left(\sum_i \alpha_i y_i K(\tilde{x}_i, \tilde{x}) + b \right)$$

حالا فرض کنیم تصمیم بگیریم هر نقطه داده را از طریق برخی تبدیل $\Phi: \tilde{x} \rightarrow \phi(\tilde{x})$ به یک فضای با ابعاد بالاتر نگاشت کنیم. سپس محصول نقطه به $\phi(\tilde{x}_i)^T \phi(\tilde{x}_j)$ تبدیل می‌شود. اگر معلوم شود که این محصول نقطه (که فقط یک عدد حقیقی است) را می‌توان به سادگی و کارایی در terms of نقاط داده اصلی محاسبه کرد، آنگاه ما واقعاً نیاز به نگاشت از $\tilde{x} \rightarrow \phi(\tilde{x})$ نخواهیم داشت. بلکه، می‌توانیم به سادگی مقدار $K(\tilde{x}_i, \tilde{x}_j) = \phi(\tilde{x}_i)^T \phi(\tilde{x}_j)$ را محاسبه کنیم و سپس از مقدار تابع در معادله (15.13) استفاده کنیم. یک تابع هسته (kernel function) K چنین تابعی است که متناظر با یک محصول نقطه در برخی فضای ویژگی گسترش یافته است.

مثال واقعی: کرنل چندجمله‌ای در تشخیص چهره

مثال 15.2: هسته درجه دوم در دو بعد. برای بردارهای دو بعدی $\tilde{u} = (u_1 \ u_2)$ ، $\tilde{v} = (v_1 \ v_2)$ ، $K(\tilde{u}, \tilde{v}) = (1 + \tilde{u}^T \tilde{v})^2$ را در نظر بگیرید. ما می‌خواهیم نشان دهیم که این یک هسته است، یعنی $K(\tilde{u}, \tilde{v}) = \phi(\tilde{u})^T \phi(\tilde{v})$ برای برخی ϕ . $\phi(\tilde{u}) = (1 \ u_1^2 \ \sqrt{2}u_1u_2 \ u_2^2 \ \sqrt{2}u_1 \ \sqrt{2}u_2)$ را در نظر بگیرید. سپس:

$$\begin{aligned} K(\tilde{u}, \tilde{v}) &= (1 + \tilde{u}^T \tilde{v})^2 \\ &= 1 + u_1^2 v_1^2 + 2u_1 v_1 u_2 v_2 + u_2^2 v_2^2 + 2u_1 v_1 + 2u_2 v_2 \\ &= (1u_1^2 \sqrt{2}u_1u_2 u_2^2 \sqrt{2}u_1 \sqrt{2}u_2)^T (1v_1^2 \sqrt{2}v_1v_2 v_2^2 \sqrt{2}v_1 \sqrt{2}v_2) \\ &= \phi(\tilde{u})^T \phi(\tilde{v}) \end{aligned}$$

در مثال تشخیص چهره اینستاگرام، این کرنل درجه دوم به ما اجازه می‌دهد ویژگی‌های ترکیبی مانند "فاصله بین چشم‌ها ضربدر عرض لب‌خند" را مدل کنیم که در تشخیص هویت افراد بسیار مفید است.

شرایط مرسر: چه کرنل‌هایی معتبر هستند؟

در زبان تحلیل تابعی، چه نوع توابعی توابع هسته معتبر هستند؟ توابع هسته گاهی اوقات با دقت بیشتری به عنوان هسته‌های مرسر (Mercer kernels) شناخته می‌شوند، زیرا باید شرایط Mercer را برآورده کنند: برای هر $g(\tilde{x})$ که در آن $\int g(\tilde{x})^2 d\tilde{x} < \infty$ متناهی است، باید داشته باشیم:

$$\int K(\tilde{x}, \tilde{z}) g(\tilde{x}) g(\tilde{z}) d\tilde{x} d\tilde{z} \geq 0$$

یک تابع هسته K باید پیوسته، متقارن و دارای ماتریس گرام مثبت معین باشد. چنین K به این معنی است که نگاشتی به فضای هیلبرت بازتولیدکننده هسته وجود دارد به طوری که محصول نقطه در آن مقداری برابر با تابع K می‌دهد. اگر یک هسته شرایط مرسر را برآورده نکند، آنگاه QP متناظر ممکن است راه‌حلی نداشته باشد.

دو خانواده رایج از هسته‌ها، هسته‌های چندجمله‌ای و توابع پایه شعاعی هستند. هسته‌های چندجمله‌ای به صورت $K(\tilde{x}, \tilde{z}) = (1 + \tilde{x}^T \tilde{z})^d$ هستند. حالت $d = 1$ یک هسته خطی است، که قبلاً در ابتدای این بخش داشتیم (ثابت 1 فقط آستانه را تغییر می‌دهد). حالت $d = 2$ یک هسته درجه دوم می‌دهد و بسیار رایج است.

رایج‌ترین شکل تابع پایه شعاعی یک توزیع گوسی است که به صورت زیر محاسبه می‌شود:

$$K(\tilde{x}, \tilde{z}) = e^{-(\tilde{x} - \tilde{z})^2 / (2\sigma^2)}$$

در سیستم‌های تشخیص چهره مانند FaceID اپل:

- **کرنل‌های چندجمله‌ای** برای مدل کردن ویژگی‌های ترکیبی مانند "فاصله بین چشم‌ها ضربدر عرض لب‌خند" استفاده می‌شوند که در تشخیص هویت افراد مفید است.
- **کرنل‌های پایه شعاعی (RBF)** برای ایجاد مرزهای تصمیم پیچیده که می‌توانند تغییرات نوری و زوایای مختلف چهره را مدیریت کنند، به کار می‌روند.
- **کرنل‌های رشته‌ای** برای تحلیل ویژگی‌های محلی مانند الگوهای بافت پوست یا جزئیات دقیق around the eyes استفاده می‌شوند.

این کرنل‌های مختلف به سیستم اجازه می‌دهند تا با ترکیب ویژگی‌های ساده، الگوهای پیچیده‌ای را تشخیص دهد که در فضای اصلی غیرقابل تشخیص هستند.

جهان SVM‌ها با زبان خاص خود همراه است که بسیار متفاوت از زبان مورد استفاده در یادگیری ماشین است. این اصطلاحات ریشه‌های عمیقی در ریاضیات دارد، اما مهم است که بیش از حد تحت تأثیر این اصطلاحات قرار نگیریم. در واقع، ما در مورد چیزهای کاملاً ساده صحبت می‌کنیم. یک هسته چندجمله‌ای به ما امکان می‌دهد تا ویژگی‌های الحاقی (تا مرتبه چندجمله‌ای) را مدل کنیم. یک تابع پایه شعاعی به شما امکان می‌دهد ویژگی‌هایی داشته باشید که دایره‌ها (ابرکره‌ها) را انتخاب می‌کنند - اگرچه مرزهای تصمیم‌گیری با تعامل چندین ویژگی از این دست بسیار پیچیده‌تر می‌شوند. یک هسته رشته‌ای به شما اجازه می‌دهد ویژگی‌هایی داشته باشید که زیردنباله‌های کاراکتری عبارات هستند. همه اینها مفاهیم ساده‌ای هستند که در بسیاری از مکان‌های دیگر با نام‌های مختلف نیز استفاده شده‌اند.

```
In [27]: import numpy as np
import matplotlib.pyplot as plt
from cvxopt import matrix, solvers
from sklearn.datasets import make_moons, make_circles
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# =====
# (Kernel Trick) غیرخطی با استفاده از هوش هسته‌ای SVM پیاده‌سازی
# می‌توان  $\phi(x)$  این پیاده‌سازی نشان می‌دهد چگونه بدون محاسبه مستقیم
# در فضای با ابعاد بالاتر از طبقه‌بندی خطی استفاده کرد
# =====

class NonlinearSVM:
    """
    SVM (Kernel SVM) غیرخطی با هسته
    این پیاده‌سازی فرمول‌بندی دوگانه را با جایگزینی محصول نقطه
    پیاده‌سازی می‌کند  $K(x_i, x_j)$  با تابع هسته
    """

    def __init__(self, kernel='linear', C=1.0, degree=2, gamma=1.0):
        """
        kernel: نوع هسته ('linear', 'polynomial', 'rbf')
        C: پارامتر منظم‌سازی (مانند حاشیه نرم)
        degree: درجه برای هسته چندجمله‌ای
        gamma: پارامتر خطی‌سازی برای هسته RBF
        """

        self.kernel = kernel
        self.C = C
        self.degree = degree
        self.gamma = gamma

        self.alpha = None # ضرایب لاگرانژ
        self.b = None # بایاس
        self.X_train = None # در پیش‌بینی K برای محاسبه داده آموزشی
        self.y_train = None # برچسب‌های آموزشی
        self.support_vectors = None
        self.support_labels = None

    def _kernel_function(self, X1, X2):
        """
        K(X1, X2) محاسبه ماتریس هسته
        این تابع کلیدی هوش هسته‌ای را پیاده‌سازی می‌کند
        """

        if self.kernel == 'linear':
            # هسته خطی:  $K(x, z) = x^T z$ 
            return np.dot(X1, X2.T)

        elif self.kernel == 'polynomial':
```

```
# هسته چندجمله‌ای:  $K(x, z) = (1 + x^T z)^d$ 
# این هسته معادل نگاشت به فضای با ابعاد بالاتر است
return (1 + np.dot(X1, X2.T)) ** self.degree
```

```
elif self.kernel == 'rbf':
    # هسته پایه شعاعی (RBF):  $K(x, z) = \exp(-\gamma \|x - z\|^2)$ 
    # این هسته به طور ضمنی فضای بینهایت بعدی را ایجاد می‌کند
    # محاسبه کارآمد فاصله اقلیدسی برای تمام جفت‌ها
    X1_norm = np.sum(X1 ** 2, axis=1).reshape(-1, 1)
    X2_norm = np.sum(X2 ** 2, axis=1).reshape(1, -1)
    distances_sq = X1_norm + X2_norm - 2 * np.dot(X1, X2.T)
    return np.exp(-self.gamma * distances_sq)
```

```
else:
    raise ValueError(f"نوع هسته '{self.kernel}' پشتیبانی نمی‌شود")
```

```
def fit(self, X, y):
```

```
    """
```

غیرخطی با روش دوگانه SVM آموزش

فرمول‌سازی:

$\max_{\alpha} \sum \alpha_i - 0.5 * \sum \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
 $s.t. \sum \alpha_i y_i = 0, 0 \leq \alpha_i \leq C$

QP به دلیل $O(N^3)$: پیچیدگی محاسباتی

```
    """
```

```
    self.X_train = X
```

```
    self.y_train = y.copy()
```

```
    n_samples = X.shape[0]
```

برای داده آموزشی K محاسبه ماتریس هسته
این جایگزین محصول نقطه در فرمول خطی می‌شود
 $K = \text{self}._\text{kernel_function}(X, X)$

برای مسئله بهینه‌سازی QP ساخت
 $P_{ij} = y_i * y_j * K(x_i, x_j)$
 $P = \text{np.outer}(y, y) * K$
 $P = \text{matrix}(P)$
 $q = \text{matrix}(-\text{np.ones}(n_samples))$

محدودیت: $-\alpha_i \leq 0$ و $\alpha_i \leq C$
 $G = \text{np.vstack}([-\text{np.eye}(n_samples), \text{np.eye}(n_samples)])$
 $h = \text{np.hstack}([\text{np.zeros}(n_samples), \text{self.C} * \text{np.ones}(n_samples)])$
 $G, h = \text{matrix}(G), \text{matrix}(h)$

محدودیت: $\sum \alpha_i y_i = 0$
 $A = \text{matrix}(y.\text{reshape}(1, -1).\text{astype}(\text{float}))$
 $b = \text{matrix}(0.0)$

حل مسئله بهینه‌سازی درجه دوم
 $\text{solvers.options['show_progress']} = \text{False}$
 $\text{solution} = \text{solvers.qp}(P, q, G, h, A, b)$

```
    self.alpha = np.array(solution['x']).flatten()
```

$\alpha_i > 0$ شناسایی بردارهای پشتیبان: نقاطی که
 $\text{sv_indices} = \text{self.alpha} > 1\text{e-}5$
 $\text{self.support_vectors} = X[\text{sv_indices}]$
 $\text{self.support_labels} = y[\text{sv_indices}]$
 $\text{self.alpha_sv} = \text{self.alpha}[\text{sv_indices}]$

$\alpha_i < C$ از یک بردار پشتیبان یا b محاسبه بایاس
 $\text{margin_sv} = (\text{self.alpha} > 1\text{e-}5) \& (\text{self.alpha} < \text{self.C} - 1\text{e-}5)$
 $\text{if np.any(margin_sv)}$:
 $\text{idx} = \text{np.where(margin_sv)}[0][0]$
 $\# b = y_i - \sum \alpha_j y_j K(x_j, x_i)$
 $\text{self.b} = y[\text{idx}] - \text{np.sum}(\text{self.alpha} * y * K[:, \text{idx}])$
 else :
 $\text{self.b} = 0$

```
def predict(self, X):
```

```
    """
```

پیش‌بینی برچسب برای داده جدید با استفاده از هوش هسته‌ای

تابع تصمیم: $f(x) = \text{sign}(\sum \alpha_i y_i K(x_i, x) + b)$
را به طور صریح محاسبه نمی‌کنیم $\phi(x)$ این نشان می‌دهد که هیچگاه
"""

محاسبه ماتریس هسته بین داده آموزشی و داده آزمون
 $K = \text{self}._\text{kernel_function}(\text{self.X_train}, X)$

تابع تصمیم: $\text{sum}(\alpha_i * y_i * K(x_i, x)) + b$
 $\text{decision_values} = \text{np.sum}((\text{self.alpha} * \text{self.y_train}).\text{reshape}(-1, 1) * K, \text{axis}=0) + \text{self.b}$

```
    return np.sign(decision_values)
```

```
# =====
# تابع برای نمایش مثال 15.2: نگاشت هسته چندجمله‌ای درجه 2
# می‌تواند  $K(u, v) = (1 + u^T v)^2$  این تابع نشان می‌دهد که چگونه
# نوشته شود  $\phi(u)^T \phi(v)$  به صورت
# =====
```

```
def demonstrate_polynomial_kernel_mapping():
```

```
    """
```

نمایش سازی محاسبات مثال 15.2 از متن اصلی

نشان می‌دهد که کرنل درجه 2، v و u برای بردارهای دو بعدی
معادل نگاشت به فضای 6 بعدی است

```
    """
```

```
    print("=*60)
```

```
    print("مثال 15.2: نگاشت ضمنی هسته چندجمله‌ای درجه 2")
```

```
    print("=*60)
```

```

# دو بردار نمونه دو بعدی
u = np.array([2.0, 3.0])
v = np.array([1.0, 2.0])

# محاسبه کرنل مستقیم:  $K(u, v) = (1 + u^T v)^2$ 
kernel_value = (1 + np.dot(u, v)) ** 2
print(f"u = {u}, v = {v}")
print(f"K(u, v) = (1 + u^T v)^2 = {kernel_value:.4f}")

# به صورت صریح  $\phi(u)$  و  $\phi(v)$  محاسبه
#  $\phi(x) = [1, x_1^2, \sqrt{2} x_1 x_2, x_2^2, \sqrt{2} x_1, \sqrt{2} x_2]$ 
phi_u = np.array([
    1.0,
    u[0] ** 2,
    np.sqrt(2) * u[0] * u[1],
    u[1] ** 2,
    np.sqrt(2) * u[0],
    np.sqrt(2) * u[1]
])

phi_v = np.array([
    1.0,
    v[0] ** 2,
    np.sqrt(2) * v[0] * v[1],
    v[1] ** 2,
    np.sqrt(2) * v[0],
    np.sqrt(2) * v[1]
])

print(f"\n $\phi(u) = \{phi\_u\}$ ")
print(f" $\phi(v) = \{phi\_v\}$ ")

# برای تأیید برابری  $\phi(u)^T \phi(v)$  محاسبه
phi_dot = np.dot(phi_u, phi_v)
print(f"\n $\phi(u)^T \phi(v) = \{phi\_dot:.4f\}$ ")
print(f"با  $K(u, v)$  برابر است {abs(kernel_value - phi_dot) < 1e-10}")

# می‌تواند بزرگ باشد  $\phi(x)$  نمایش اینکه اندازه
print(f"\n $\phi(u)$  = {np.linalg.norm(phi_u):.2f}")
print(f"اصلی u در حالی که اندازه" = {np.linalg.norm(u):.2f}")
print("!!این نشان می‌دهد چرا هوش هسته‌ای محاسباتی کارآمد است")

# =====
# تابع برای ترسیم مرز تصمیم‌گیری و مقایسه هسته‌ها
# =====

def plot_kernel_comparison(X, y, dataset_name=""):
    """
    روی یک داده (linear, polynomial, rbf) مقایسه ترسیمی سه هسته مختلف
    این به طور واضح نشان می‌دهد که چرا هسته‌های غیرخطی برای داده‌های پیچیده لازم هستند
    """
    kernels = ['linear', 'polynomial', 'rbf']
    fig, axes = plt.subplots(1, 3, figsize=(18, 5))

    for idx, kernel in enumerate(kernels):
        ax = axes[idx]

        # با این هسته SVM آموزش
        svm = NonlinearSVM(kernel=kernel, C=1.0, degree=2, gamma=1.0)
        svm.fit(X, y)

        # ترسیم نقاط داده
        class_1 = y == 1
        class_2 = y == -1

        ax.scatter(X[class_1, 0], X[class_1, 1], c='blue', s=50, alpha=0.7, label='Class +1')
        ax.scatter(X[class_2, 0], X[class_2, 1], c='red', s=50, alpha=0.7, label='Class -1')

        # ترسیم بردارهای پشتیبان
        if len(svm.support_vectors) > 0:
            ax.scatter(svm.support_vectors[:, 0], svm.support_vectors[:, 1],
                       facecolors='none', edgecolors='black', s=200, linewidth=2,
                       label=f'SVs ({len(svm.support_vectors)})')

        # ایجاد شبکه برای مناطق تصمیم‌گیری
        x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
        y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
        xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.1),
                              np.arange(y_min, y_max, 0.1))

        # پیش‌بینی برای شبکه
        Z = svm.predict(np.c_[xx.ravel(), yy.ravel()])
        Z = Z.reshape(xx.shape)

        # ترسیم مناطق تصمیم‌گیری
        ax.contourf(xx, yy, Z, alpha=0.2, cmap='coolwarm')
        ax.contour(xx, yy, Z, colors='k', levels=[0], linestyles='--', linewidths=1)

        ax.set_xlabel('Feature 1', fontsize=11)
        ax.set_ylabel('Feature 2', fontsize=11)
        ax.set_title(f'{kernel.capitalize()} Kernel', fontsize=12, fontweight='bold')
        ax.legend(fontsize=9)
        ax.grid(True, alpha=0.3)
        ax.axis('equal')

    plt.suptitle(f'Kernel Comparison: {dataset_name}\n(Same C=1.0, degree=2, gamma=1.0)',
                 fontsize=14, fontweight='bold')
    plt.tight_layout()
    plt.show()

# =====

```

```
# اجرای مثال‌ها و مقایسه‌ها
# =====
```

```
if __name__ == "__main__":
    # 1. نمایش مثال 15.2 از متن
    demonstrate_polynomial_kernel_mapping()
    print("\n\n" + "="*60 + "\n")

    # 2. (moon) تست روی داده غیرخطی: شکل ماه
    print("Non-linearly separable)...")
    np.random.seed(42)
    X_moon, y_moon = make_moons(n_samples=200, noise=0.1, random_state=42)
    y_moon = np.where(y_moon == 0, -1, 1) # تبدیل به 1±

    # نمایش اینکه هسته خطی در اینجا کار نمی‌کند
    linear_svm = NonlinearSVM(kernel='linear', C=1.0)
    linear_svm.fit(X_moon, y_moon)
    linear_acc = accuracy_score(y_moon, linear_svm.predict(X_moon))
    print(f"(پایین - داده غیرخطی است) {linear_acc:.2%}: دقت با هسته خطی")

    # مقایسه هسته‌ها
    plot_kernel_comparison(X_moon, y_moon, dataset_name="Moon Dataset")

    # 3. (circle) تست روی داده به شکل دایره
    print("\n" + "="*60)
    print("آزمایش روی داده به شکل دایره...")
    X_circle, y_circle = make_circles(n_samples=200, noise=0.1, factor=0.5, random_state=42)
    y_circle = np.where(y_circle == 0, -1, 1)

    plot_kernel_comparison(X_circle, y_circle, dataset_name="Circle Dataset")

    # 4. تحلیل حساسیت به پارامترها
    print("\n" + "="*60)
    print("تحلیل حساسیت پارامترهای هسته...")

    # تأثیر درجه در هسته چندجمله‌ای
    degrees = [2, 3, 5]
    fig, axes = plt.subplots(1, 3, figsize=(18, 5))

    for idx, degree in enumerate(degrees):
        ax = axes[idx]
        svm = NonlinearSVM(kernel='polynomial', C=1.0, degree=degree)
        svm.fit(X_circle, y_circle)

        accuracy = accuracy_score(y_circle, svm.predict(X_circle))

        # ترسیم نتایج
        xx, yy = np.meshgrid(np.linspace(-2, 2, 200), np.linspace(-2, 2, 200))
        Z = svm.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

        ax.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
        ax.scatter(X_circle[:, 0], X_circle[:, 1], c=y_circle, cmap='coolwarm', s=30)
        ax.set_title(f'Polynomial (degree={degree})\nAccuracy: {accuracy:.2%}',
                    fontsize=12, fontweight='bold')
        ax.set_xlabel('Feature 1')
        ax.set_ylabel('Feature 2')
        ax.axis('equal')

    plt.suptitle('Effect of Polynomial Degree', fontsize=14, fontweight='bold')
    plt.tight_layout()
    plt.show()

    # تأثیر گاما در هسته RBF
    print("\n" + "="*60)
    print("RBF تحلیل تأثیر گاما در هسته...")

    gammas = [0.1, 1.0, 10.0]
    fig, axes = plt.subplots(1, 3, figsize=(18, 5))

    for idx, gamma in enumerate(gammas):
        ax = axes[idx]
        svm = NonlinearSVM(kernel='rbf', C=1.0, gamma=gamma)
        svm.fit(X_circle, y_circle)

        accuracy = accuracy_score(y_circle, svm.predict(X_circle))

        xx, yy = np.meshgrid(np.linspace(-2, 2, 200), np.linspace(-2, 2, 200))
        Z = svm.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

        ax.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
        ax.scatter(X_circle[:, 0], X_circle[:, 1], c=y_circle, cmap='coolwarm', s=30)
        ax.set_title(f'RBF (gamma={gamma})\nAccuracy: {accuracy:.2%}',
                    fontsize=12, fontweight='bold')
        ax.set_xlabel('Feature 1')
        ax.set_ylabel('Feature 2')
        ax.axis('equal')

    plt.suptitle('Effect of RBF Gamma Parameter', fontsize=14, fontweight='bold')
    plt.tight_layout()
    plt.show()

    # 5. خلاصه نتایج
    print("\n" + "="*60)
    print("خلاصه نتایج آزمایش:")
    print("="*60)
    print("هسته خطی فقط برای داده‌های جداپذیر خطی کارآمد است •")
    print("هسته چندجمله‌ای (درجه 2 یا 3) برای داده‌های دارای ساختار قابل پیش‌بینی •")
    print("برای داده‌های بسیار پیچیده و بدون ساختار مشخص RBF هسته •")
    print("افزایش درجه با گاما باعث پیچیده‌تر شدن مرز تصمیم می‌شود •")
    print("با پارامترهای خیلی بزرگ (overfitting) خطر بیش‌ورسازی •")
    print("در تشخیص چهره، ترکیب این هسته‌ها می‌تواند دقت را بهبود بخشد •")
```


=====

مثال 15.2: نگاشت ضمنی هسته چندجمله‌ای درجه 2

=====

$u = [2. \ 3.]$, $v = [1. \ 2.]$
 $K(u, v) = (1 + u^T v)^2 = 81.0000$

$\phi(u) = [1. \quad 4. \quad 8.48528137 \ 9. \quad 2.82842712 \ 4.24264069]$
 $\phi(v) = [1. \quad 1. \quad 2.82842712 \ 4. \quad 1.41421356 \ 2.82842712]$

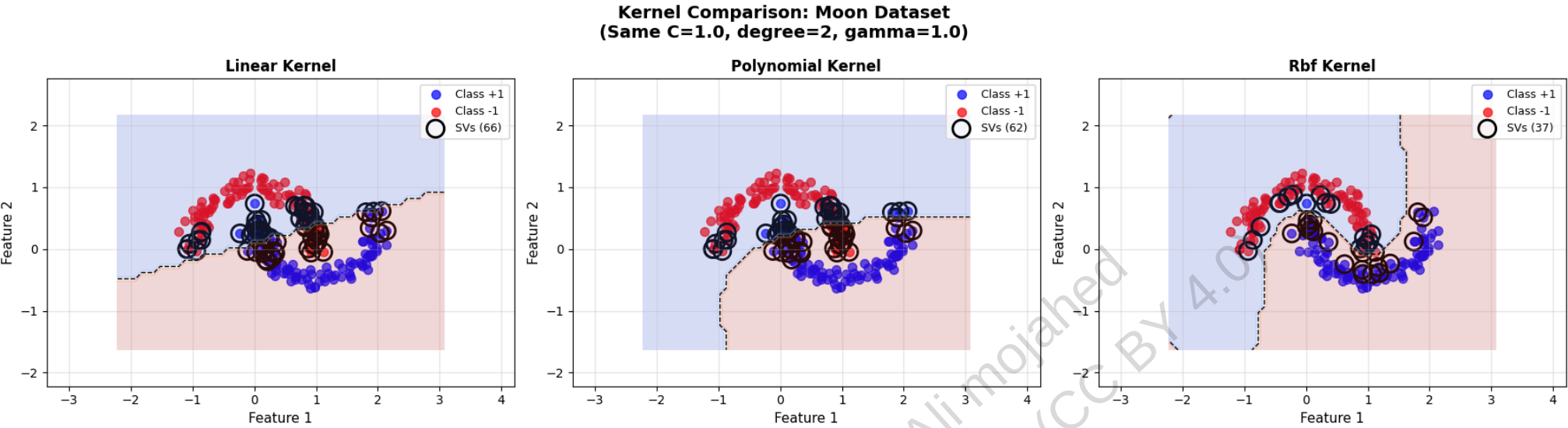
$\phi(u)^T \phi(v) = 81.0000$
True برابر است $K(u, v)$ آیا با

اندازه $\phi(u) = 14.00$ توجه: اصلی = 3.61 در حالی که اندازه
!این نشان می‌دهد چرا هوش هسته‌ای محاسباتی کارآمد است

=====

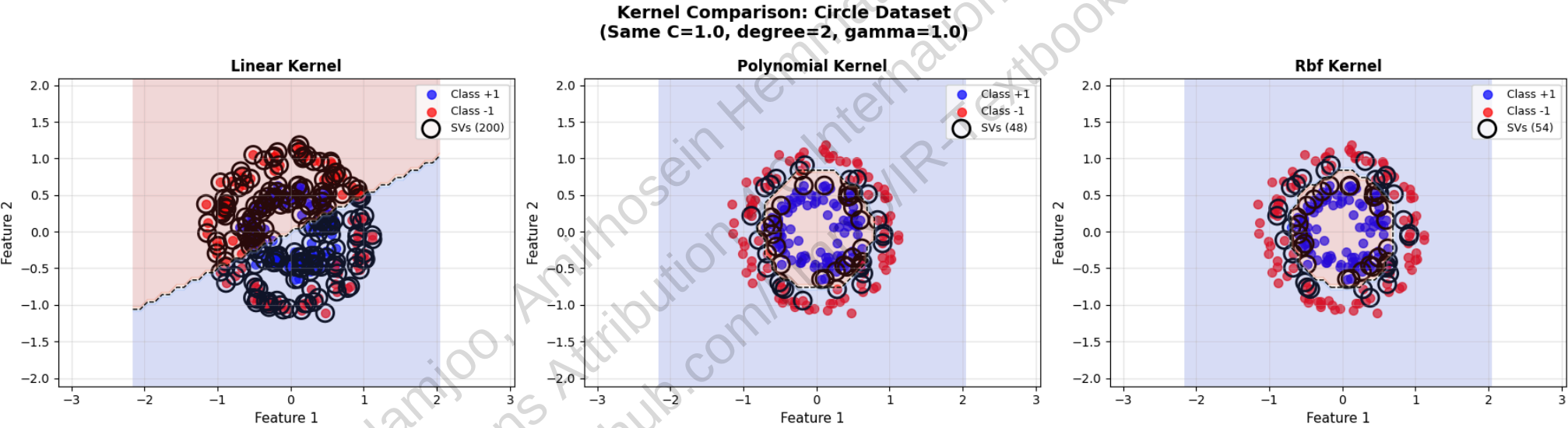
.....

(Non-linearly separable) آزمایش روی داده به شکل ماه
دقت با هسته خطی: 86.50% (پایین - داده غیرخطی است)



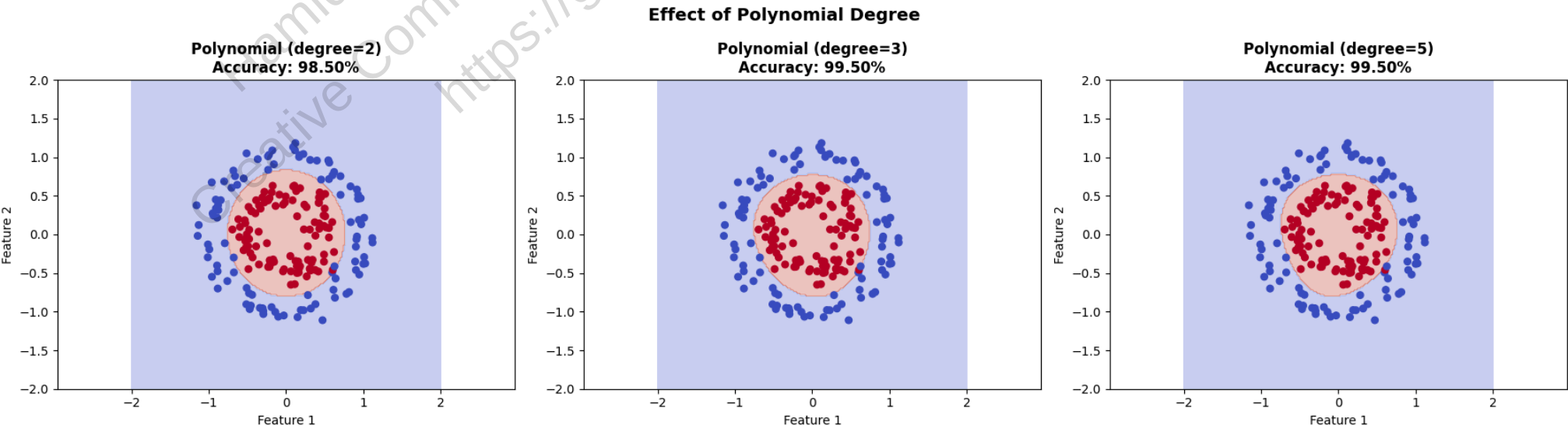
.....

...آزمایش روی داده به شکل دایره



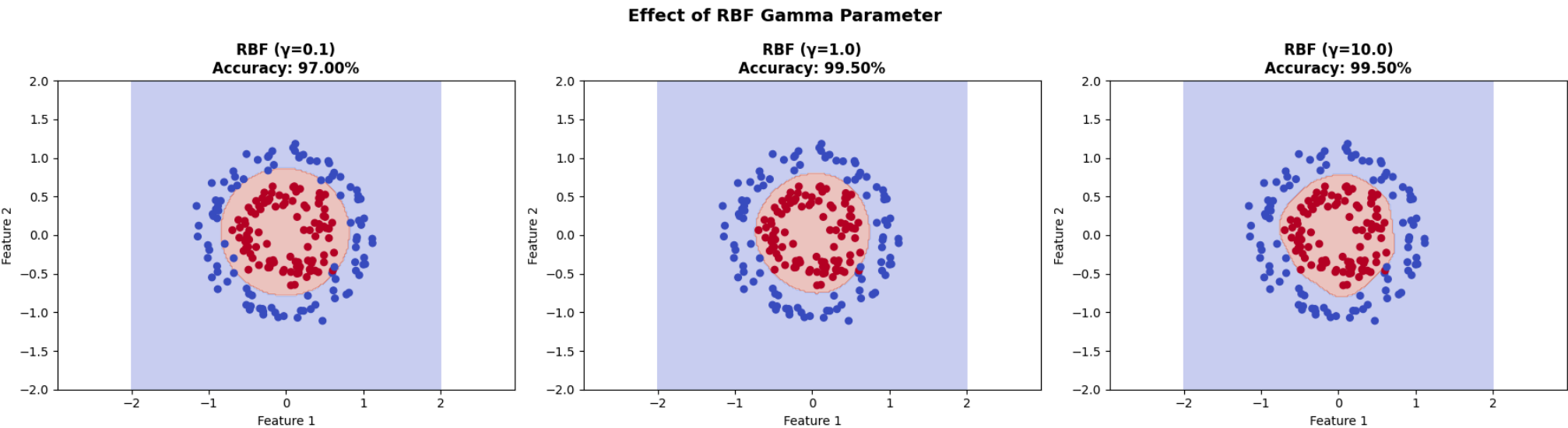
.....

...تحلیل حساسیت پارامترهای هسته



.....

...RBF تحلیل تأثیر گاما در هسته



=====

خلاصه نتایج آزمایش

=====

- هسته خطی فقط برای داده‌های جدایی‌ناپذیر خطی کارآمد است
- هسته چندجمله‌ای (درجه 2 یا 3) برای داده‌های دارای ساختار قابل پیش‌بینی
- برای داده‌های بسیار پیچیده و بدون ساختار مشخص RBF هسته
- افزایش درجه یا گاما باعث پیچیده‌تر شدن مرز تصمیم می‌شود
- با پارامترهای خیلی بزرگ (overfitting) خطر بیش‌وردسازی
- در تشخیص چهره، ترکیب این هسته‌ها می‌تواند دقت را بهبود بخشد

15.2.4 نتایج تجربی: اثبات برتری SVM در دنیای واقعی

📰 **رویترز: چگونه هزاران خبر روزانه را به صورت خودکار دسته‌بندی کنیم؟**

تصور کنید در آژانس خبری رویترز کار می‌کنید و مسئول سیستم خودکار دسته‌بندی اخبار هستید. هر روز هزاران خبر از سراسر جهان دریافت می‌شود که باید به دسته‌هایی مانند "اقتصادی"، "سیاسی"، "ورزشی"، "فناوری" و... تقسیم شوند. این یک چالش بزرگ در طبقه‌بندی متن است، زیرا:

- برخی دسته‌ها (مانند اقتصادی) ویژگی‌های زبانی مشخصی دارند
- برخی دیگر (مانند سیاسی) بسیار متنوع و پیچیده هستند
- برخی اخبار ممکن است به چند دسته مرتبط باشند

در این محیط، انتخاب الگوریتم مناسب می‌تواند تفاوت بین یک سیستم کارآمد و یک سیستم ناکارآمد باشد.

ما در بخش 13.6 نتایجی را ارائه کردیم که نشان می‌دهد SVM یک طبقه‌بند بسیار مؤثر برای متن است. نتایج Dumais و همکاران (1998) که در جدول 13.9 نشان داده شده، به وضوح نشان می‌دهد که SVM‌ها بهترین عملکرد را دارند. این یکی از چندین کار از آن زمان بود که شهرت قوی SVM‌ها را برای طبقه‌بندی متن تثبیت کرد. کار پیشگام دیگر در مقیاس‌بندی و ارزیابی SVM‌ها برای طبقه‌بندی متن (Joachims 1998) بود.

🔍 **مقایسه عملکرد: کدام الگوریتم برای دسته‌بندی اخبار بهتر است؟**

در سیستم دسته‌بندی اخبار رویترز، تیم مهندسی باید بین الگوریتم‌های مختلف یکی را انتخاب کند. نتایج تجربی به ما کمک می‌کند تا تصمیم آگاهانه‌تری بگیریم:

- **بیز ساده (Naive Bayes):** سریع و ساده، اما برای دسته‌های پیچیده مانند "سیاسی" عملکرد ضعیفی دارد.
- **روش Rocchio:** برای دسته‌های با ویژگی‌های مشخص مانند "اقتصادی" خوب عمل می‌کند، اما در دسته‌های متنوع ضعیف است.
- **SVM خطی:** در اکثر دسته‌ها عملکرد بسیار خوبی دارد، به خصوص زمانی که پارامتر منظم‌سازی به درستی تنظیم شود.
- **SVM با تابع پایه شعاعی (RBF):** برای دسته‌های پیچیده و نامشخص عملکرد بهتری دارد، اما محاسبات سنگین‌تری نیاز دارد.

ما برخی از نتایج Joachims را از (Joachims 2002a) در جدول 15.2 ارائه می‌دهیم. Joachims تعداد زیادی ویژگی واژگانی را در مقابل Dumais و همکاران (1998) استفاده کرد که از انتخاب ویژگی MI برای ساخت طبقه‌بندها با تعداد بسیار محدودتر ویژگی استفاده کردند. موفقیت SVM خطی با نتایج بحث شده در بخش 14.6 در مورد رویکردهای خطی دیگر مانند بیز ساده مطابقت دارد. به نظر می‌رسد که کار با ویژگی‌های واژگانی ساده می‌تواند فرد را تا مسیر طولانی پیش ببرد.

Category	NB	Rocchio	Dec. Trees	kNN	linear SVM C = 0.5	linear SVM C = 1.0	rbf-SVM $\sigma \approx 7$
earn	96.0	96.1	96.1	97.8	98.0	98.2	98.1

acq	90.7	92.1	85.3	91.8	95.5	95.6	94.7
money-fx	59.6	67.6	69.4	75.4	78.8	78.5	74.3
grain	69.8	79.5	89.1	82.6	91.9	93.1	93.4
crude	81.2	81.5	75.5	85.8	89.4	89.4	88.7
trade	52.2	77.4	59.2	77.9	79.2	79.2	76.6
interest	57.6	72.5	49.1	76.7	75.6	74.8	69.1
ship	80.9	83.1	80.9	79.8	87.4	86.5	85.8
wheat	63.4	79.4	85.5	72.9	86.6	86.8	82.4
corn	45.2	62.2	87.7	71.4	87.5	87.8	84.6
microavg.	72.3	79.9	79.4	82.6	86.7	87.5	86.4

📊 تحلیل نتایج: چه درسی می‌توانیم بگیریم؟

جدول 15.2 نتایج مقایسه‌ای الگوریتم‌های مختلف را نشان می‌دهد. برای سیستم دسته‌بندی اخبار رویترز، این نتایج بسیار مهم هستند:

- دسته‌های ساده (earn, acq): SVM خطی با $C=1.0$ بهترین عملکرد را دارد، اما تفاوت با سایر الگوریتم‌ها زیاد نیست.
- دسته‌های متوسط (crude, ship): SVM خطی به وضوح بهتر از سایر الگوریتم‌ها عمل می‌کند.
- دسته‌های پیچیده (money-fx, trade, interest): در این دسته‌ها، SVM به طور قابل توجهی بهتر از سایر الگوریتم‌ها عمل می‌کند، به خصوص در مقایسه با بیز ساده.
- دسته‌های خاص (grain, corn): جالب است که در این دسته‌ها، درخت تصمیم عملکرد خوبی دارد، اما SVM همچنان برتر است.

در مجموع، SVM خطی با $C=1.0$ بهترین عملکرد میانگین (microavg. 87.5) را دارد که تقریباً 15% بهتر از بهترین الگوریتم غیر SVM (Rocchio با 79.9) است.

🔍 تفاوت در نتایج تحقیقات: چرا نتایج متفاوتی گزارش می‌شود؟

دوباره قابل توجه است که نتایج مقالات مختلف برای روش‌های یادگیری ماشین یکسان چقدر با هم تفاوت دارند. به طور خاص، بر اساس بازتولید توسط محققان دیگر، نتایج بیز ساده (Joachims 1998) به نظر بیش از حد ضعیف می‌رسند، و نتایج در جدول 13.9 باید به عنوان نماینده در نظر گرفته شوند.

در سیستم دسته‌بندی اخبار رویترز، این تفاوت‌ها بسیار مهم هستند:

- روش پیش‌پردازش متن (نرمال‌سازی، حذف ایست‌واژه‌ها)
- انتخاب ویژگی (MI در مقابل استفاده از تمام ویژگی‌ها)
- تنظیم پارامترهای الگوریتم (مقدار C در SVM)
- اندازه و کیفیت مجموعه داده آموزشی

این عوامل می‌توانند باعث تفاوت‌های قابل توجهی در عملکرد الگوریتم‌های یکسان در مطالعات مختلف شوند.

15.3 چالش‌های طبقه‌بندی اسناد متنی: از تئوری تا عمل

توئیت: چگونه میلیاردها توییت را مدیریت می‌کنند؟

تصور کنید در تیم مدیریت محتوای توئیت کار می‌کنید و مسئول سیستمی هستید که باید روزانه صدها میلیون توییت را دسته‌بندی کند. این توییت‌ها باید به دسته‌هایی مانند "اخبار"، "ورزشی"، "سرگرمی"، "تبلیغات"، "محتوای نامناسب" و... تقسیم شوند.

Jackson و Moulinier (2002) می‌نویسند: "هیچ تردیدی در مورد ارزش تجاری توانایی طبقه‌بندی خودکار اسناد بر اساس محتوا وجود ندارد." این جمله در مورد توئیت صدق می‌کند - ارزش تجاری این پلتفرم به شدت به کیفیت سیستم طبقه‌بندی آن وابسته است.

بیشتر بحث ما در مورد طبقه‌بندی بر معرفی روش‌های مختلف یادگیری ماشین متمرکز بوده است تا بحث در مورد ویژگی‌های خاص اسناد متنی مرتبط با طبقه‌بندی. این سوگیری برای یک کتاب درسی مناسب است، اما برای یک توسعه‌دهنده کاربردی ناکارآمد است.

اهمیت درک داده: کلید موفقیت در طبقه‌بندی

اغلب این اتفاق می‌افتد که بهره‌وری عملکردی بیشتر می‌تواند از طریق بهره‌برداری از ویژگی‌های متن خاص دامنه (domain-specific) نسبت به تغییر از یک روش یادگیری ماشین به روش دیگر به دست آید.

در مثال توئیت، این به معنای درک ویژگی‌های خاص زبانی است که در هر دسته متفاوت است:

- **اخبار:** لحن رسمی، استفاده از منابع خبری، هشتک‌های رویدادهای روز
- **ورزشی:** اسامی تیم‌ها، اصطلاحات ورزشی خاص، اموجی‌های شادی پس از پیروزی
- **محتوای نامناسب:** کلمات توهین‌آمیز، لحن پرخاش، پیام‌های ناخواسته
- **تبلیغات:** فراخوانی اقدام، لینک‌های کوتاه تجاری، الگوهای تکراری

Jackson و Moulinier (2002) پیشنهاد می‌کنند که "درک داده یکی از کلیدهای موفقیت در دسته‌بندی است، با این حال این منطقه‌ای است که اکثر فروشندگان ابزار دسته‌بندی در آن بسیار ضعیف هستند."

محدودیت‌های ابزارهای "یک اندازه برای همه"

بسیاری از ابزارهای "یک اندازه برای همه" موجود در بازار بر روی طیف وسیعی از انواع محتوا آزمایش نشده‌اند. در سیستم طبقه‌بندی توئیت، این به معنای:

- ابزاری که برای انگلیسی بهینه شده‌اند اما برای زبان‌های دیگر عملکرد ضعیفی دارند
- سیستم‌هایی که برای محتوای خبری خوب عمل می‌کنند اما در تشخیص طنز یا کنایه ضعیف هستند
- پلتفرم‌هایی که برای متن‌های کوتاه (مانند توییت) طراحی شده‌اند اما برای مقالات طولانی عملکرد خوبی ندارند

در عمل، تیم‌های فنی توئیت از ترکیبی از روش‌ها استفاده می‌کنند: الگوریتم‌های یادگیری ماشین پیشرفته، ویژگی‌های خاص دامنه، و قوانین دستی که توسط متخصصان محتوا ایجاد شده‌اند.

در حالی که نتایج اولیه (Joachims 1998) پیشنهاد می‌کرد که استفاده از کرنل‌های چندجمله‌ای یا rbf به دستاوردهای قابل توجهی در این وظیفه منجر می‌شود، این با SVM‌های حاشیه سخت بود. با SVM‌های حاشیه نرم، یک SVM خطی ساده با C پیش‌فرض = 1 بهترین عملکرد را دارد.

این یافته در دنیای واقعی طبقه‌بندی متن بسیار مهم است:

- برای اکثر کاربردهای متنی، یک SVM خطی با تنظیمات مناسب عملکرد خوبی ارائه می‌دهد
- پیچیدگی اضافی (کرنل‌های پیچیده) لزوماً به بهبود عملکرد منجر نمی‌شود
- درک ویژگی‌های خاص دامنه اغلب مهم‌تر از انتخاب الگوریتم پیچیده است

در سیستم طبقه‌بندی توئیت‌ر، این به معنای تمرکز بر ویژگی‌های خاص زبانی برای هر زبان و فرهنگ، به جای تلاش برای یافتن یک الگوریتم جهانی که برای همه موارد به خوبی عمل کند.

در این بخش، کمی عقب‌نشینی کرده و کاربردهای طبقه‌بندی متن، فضای راه‌حل‌های ممکن و سودمندی اکتشافات خاص کاربرد را در نظر گرفتیم. این رویکرد به توسعه‌دهندگان کمک می‌کند تا راه‌حل‌های واقعی‌تر و مؤثرتری برای مسائل خاص خود طراحی کنند.

15.3.1 انتخاب طبقه‌بند متن: راهنمای عملی برای توسعه‌دهندگان 🧭

🔍 پلتفرم‌های محتوا: چگونه مسیر محتوای خود را پیدا کنیم؟

تصور کنید در تیم فنی یوتیوب کار می‌کنید و باید سیستمی طراحی کنید که میلیون‌ها ویدیو را به صورت خودکار دسته‌بندی کند. اولین سؤال کلیدی این است: "چقدر داده آموزشی در اختیار داریم؟" این سؤال به ظاهر ساده به نظر می‌رسد، اما پاسخ آن تعیین‌کننده مسیر کل پروژه شما خواهد بود.

در یوتیوب، تیم محتوا با چالش‌های مختلفی روبرو است:

- دسته‌بندی ویدیوهای جدید با داده آموزشی محدود
- شناسایی محتوای نامناسب با الگوهای پیچیده
- توصیه محتوای مرتبط با استفاده از سیگنال‌های کاربر

هر یک از این چالش‌ها به رویکرد متفاوتی برای انتخاب طبقه‌بند نیاز دارد.

📄 سناریو اول: بدون داده آموزشی برچسب‌دار

اگر هیچ داده آموزشی برچسب‌دار ندارید، و به خصوص اگر کارکنانی وجود دارند که در مورد دامنه دانش دارند، هرگز نباید راه‌حل استفاده از قوانین دستی را فراموش کنید. در یوتیوب، این رویکرد برای دسته‌بندی اولیه ویدیوها استفاده شد.

یک قانون ساده برای دسته‌بندی ویدیوهای موسیقی می‌تواند به صورت زیر باشد:

```
IF (music OR audio) AND NOT (speech OR news) THEN c = music
```

در عمل، قوانین بسیار پیچیده‌تر از این می‌شوند و می‌توانند با استفاده از زبان‌های کوئری پیشرفته‌تر بیان شوند. با طراحی دقیق توسط متخصصان محتوا، دقت چنین قوانینی می‌تواند بسیار بالا شود.

در یوتیوب، این رویکرد برای دسته‌بندی اولیه ویدیوها استفاده شد، اما با رشد سریع پلتفرم، به سرعت جای خود را به روش‌های یادگیری ماشین دادند.

📊 سناریو دوم: داده آموزشی محدود

اگر داده آموزشی محدودی دارید و قصد دارید یک طبقه‌بند نظارت‌شده آموزش دهید، نظریه یادگیری ماشین می‌گوید باید به یک طبقه‌بند با سوگیری بالا بچسبید. برای مثال، در پلتفرم لینکدین، طبقه‌بندهای بیز ساده برای دسته‌بندی اولیه مشاغل با داده‌های محدود عملکرد خوبی داشتند.

با این حال، یک مدل با سوگیری بسیار پایین مانند مدل نزدیک‌ترین همسایه احتمالاً نامناسب است. صرف نظر از این، کیفیت مدل به نفعی توسط داده‌های آموزشی محدود تأثیر منفی خواهد داشت.

در اینجا، پاسخ نظری جالب، تلاش برای اعمال روش‌های آموزش نیمه‌نظارت‌شده است. این شامل روش‌هایی مانند بوت‌استرپینگ یا الگوریتم EM است که در آن سیستم تعدادی سند برچسب‌دار و منبع بزرگتری از اسناد بدون برچسب دریافت می‌کند. این رویکرد در سیستم‌های توصیه محتوای نتفلیکس برای دسته‌بندی فیلم‌ها و سریال‌ها با استفاده از داده‌های متعدد بدون برچسب به کار گرفته شد.

📊 سناریو سوم: داده آموزشی کافی

اگر مقدار معقولی از داده‌های برچسب‌دار وجود دارد، آنگاه شما در موقعیت کاملی برای استفاده از الگوریتم‌های پیشرفته مانند SVM هستید. با این حال، اگر در حال پیاده‌سازی یک طبقه‌بند خطی مانند SVM هستید، احتمالاً باید یک کاربرد طراحی کنید که یک طبقه‌بند مبتنی بر قانون بولی را بر روی طبقه‌بند یادگیری ماشین لایه‌گذاری کند.

در سیستم فیلتر محتوای فیسبوک، این رویکرد ترکیبی به کار گرفته شد: یک SVM خطی برای دسته‌بندی اولیه محتوا، با یک لایه قوانین دستی که توسط متخصصان امنیت برای موارد خاص تنظیم شده بود. کاربران اغلب دوست دارند چیزهایی که کاملاً درست از آب در نمی‌آیند را تنظیم کنند، و اگر مدیریت تلفن می‌زند و می‌خواهد طبقه‌بندی یک پست خاص همین الان درست شود، این کار بسیار آسان‌تر است با نوشتن یک قانون دستی از طریق کشف نحوه تنظیم وزن‌های یک SVM.

🌐 سناریو چهارم: داده آموزشی عظیم

اگر مقدار عظیمی از داده‌ها در دسترس باشد، آنگاه انتخاب طبقه‌بند احتمالاً تأثیر کمی بر نتایج شما دارد و بهترین انتخاب ممکن است نامشخص باشد. در این شرایط، انتخاب طبقه‌بند بر اساس مقیاس‌پذیری آموزش یا حتی بهره‌وری زمان اجرا ممکن است بهترین گزینه باشد.

در موتور جستجوی گوگل، با میلیاردها سند وب برای آموزش، تفاوت بین الگوریتم‌های مختلف برای دسته‌بندی وب کمتر قابل توجه است. در این مقیاس، قانون کلی این است که هر دو برابر شدن اندازه داده‌های آموزشی یک افزایش خطی در عملکرد طبقه‌بند تولید می‌کند، اما با مقادیر بسیار زیاد داده، بهبود زیرخطی می‌شود.

در نهایت، انتخاب طبقه‌بند متن یک تصمیم استراتژیک است که باید بر اساس منابع موجود، نیازهای کاربردی و ماهیت داده‌ها گرفته شود. هیچ رویکرد "یک اندازه برای همه" وجود ندارد - بهترین راه‌حل ترکیبی هوشمندانه از تکنیک‌های مختلف بر اساس شرایط خاص پروژه است.


```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, f1_score, classification_report
from sklearn.model_selection import train_test_split
import time

# =====
# 15.3.1 پیاده‌سازی سناریوهای انتخاب طبقه‌بند متن از بخش
# :این کد چهار سناریوی اصلی را شبیه‌سازی می‌کند:
# 1. بدون داده برچسب‌دار (rule-based)
# 2. داده محدود (Naive Bayes - high bias)
# 3. داده کافی (SVM/Logistic - low bias)
# 4. داده عظیم (مقایسه پیچیدگی زمانی)
# =====

class RuleBasedTextClassifier:
    """
    پیاده‌سازی سناریو اول: طبقه‌بند مبتنی بر قوانین دستی
    این رویکرد بدون نیاز به داده آموزشی کار می‌کند و بر اساس قوانین منطقی
    طراحی شده توسط متخصص دامنه عمل می‌کند (مانند سیستم اولیه یوتیوب)
    """

    def __init__(self):
        # تعریف قوانین دستی برای هر کلاس
        # (وزن‌دهی و regex با استفاده از این قوانین می‌توانند پیچیده‌تر شوند)
        self.class_rules = {
            'technology': {
                'keywords': ['computer', 'software', 'algorithm', 'data', 'programming', 'code', 'ai', 'machine'],
                'weight_threshold': 2 # حداقل 2 کلمه کلیدی برای این کلاس
            },
            'sports': {
                'keywords': ['game', 'player', 'team', 'score', 'match', 'sport', 'win', 'competition'],
                'weight_threshold': 2
            },
            'politics': {
                'keywords': ['government', 'election', 'policy', 'vote', 'minister', 'political', 'congress', 'law'],
                'weight_threshold': 2
            }
        }

    def predict(self, documents):
        """
        پیش‌بینی بر اساس تطابق کلمات کلیدی
        پیش‌فرض طبقه‌بندی می‌شود 'sports' هر سندی که هیچ قانونی را تطابق ندهد، به
        (این مشابه رویکرد یوتیوب برای دسته‌بندی اولیه بود)
        """

        predictions = []

        for doc in documents:
            doc_lower = doc.lower()
            class_scores = {}

            # محاسبه امتیاز برای هر کلاس بر اساس تعداد کلمات تطابق
            for class_name, rules in self.class_rules.items():
                matches = sum(1 for keyword in rules['keywords'] if keyword in doc_lower)
                class_scores[class_name] = matches

            # انتخاب کلاس با بیشترین تطابق و حداقل امتیاز
            best_class = max(class_scores, key=class_scores.get)
            if class_scores[best_class] >= self.class_rules[best_class]['weight_threshold']:
                predictions.append(best_class)
            else:
                predictions.append('sports') # کلاس پیش‌فرض

        return predictions

def generate_simulated_content_data(n_samples_per_class=500):
    """
    تولید داده شبیه‌سازی شده برای انواع محتوا
    این داده‌ها ساختار واقعی محتوای پلتفرم‌هایی مانند یوتیوب، لینکدین و فیس‌بوک را تقلید می‌کنند
    """

    # واژگان تخصصی برای هر دامنه (برای شبیه‌سازی داده‌های واقعی)
    domain_vocabularies = {
        'technology': [
            'algorithm', 'function', 'variable', 'programming', 'software',
            'hardware', 'optimization', 'complexity', 'data', 'network',
            # نویز و کلمات عمومی برای واقعی‌سازی بیشتر
            'system', 'application', 'computer', 'development', 'code'
        ],
        'sports': [
            'player', 'game', 'score', 'team', 'championship',
            'competition', 'match', 'goal', 'victory', 'defeat',
            'strategy', 'performance', 'season', 'tournament', 'coach'
        ],
        'politics': [
            'government', 'policy', 'election', 'candidate', 'democracy',
            'legislation', 'congress', 'parliament', 'vote', 'campaign',
            'minister', 'senator', 'debate', 'ideology', 'reform'
        ]
    }

    documents = []
    labels = []

    for class_name, vocab in domain_vocabularies.items():
        for _ in range(n_samples_per_class):
            # تعداد تصادفی کلمات تخصصی از واژگان کلاس
            n_specialized = np.random.randint(8, 15)
```

```
specialized_words = np.random.choice(vocab, n_specialized, replace=True)
```

```
# برای واقعی‌سازی (stop words) اضافه کردن کلمات عمومی
common_words = ['the', 'is', 'in', 'on', 'at', 'a', 'an', 'of', 'and', 'to', 'for']
n_common = np.random.randint(5, 10)
noise_words = np.random.choice(common_words, n_common, replace=True)
```

```
# ترکیب و ایجاد سند
all_words = np.concatenate([specialized_words, noise_words])
np.random.shuffle(all_words)
documents.append(' '.join(all_words))
labels.append(class_name)
```

```
return documents, labels
```

```
def generate_realistic_content_data(n_samples_per_class=500):
```

```
    """
```

```
    تولید داده واقعی‌تر با همپوشانی بین کلاس‌ها
```

```
    این تابع داده‌هایی تولید می‌کند که طبقه‌بندی آن‌ها چالش‌برانگیزتر است
```

```
    """
```

```
# واژگان تخصصی برای هر دامنه با همپوشانی بیشتر
```

```
domain_vocabularies = {
```

```
    'technology': [
        'algorithm', 'function', 'variable', 'programming', 'software',
        'hardware', 'optimization', 'complexity', 'data', 'network',
        'system', 'application', 'computer', 'development', 'code'
    ],
```

```
    'sports': [
        'player', 'game', 'score', 'team', 'championship',
        'competition', 'match', 'goal', 'victory', 'defeat',
        'strategy', 'performance', 'season', 'tournament', 'coach'
    ],
```

```
    'politics': [
        'government', 'policy', 'election', 'candidate', 'democracy',
        'legislation', 'congress', 'parliament', 'vote', 'campaign',
        'minister', 'senator', 'debate', 'ideology', 'reform'
    ],
```

```
# کلمات مشترک بین کلاس‌ها برای ایجاد همپوشانی
```

```
    'common': [
        'team', 'strategy', 'performance', 'system', 'development',
        'competition', 'program', 'data', 'analysis', 'report'
    ]
}
```

```
documents = []
```

```
labels = []
```

```
for class_name in ['technology', 'sports', 'politics']:
```

```
    for _ in range(n_samples_per_class):
```

```
        # تعداد کلمات تخصصی از واژگان کلاس
```

```
        n_specialized = np.random.randint(5, 10)
```

```
        specialized_words = np.random.choice(domain_vocabularies[class_name], n_specialized, replace=True)
```

```
# اضافه کردن کلمات مشترک برای ایجاد همپوشانی
```

```
n_common = np.random.randint(5, 10)
```

```
common_words = np.random.choice(domain_vocabularies['common'], n_common, replace=True)
```

```
# اضافه کردن کلمات عمومی
```

```
common_words_general = ['the', 'is', 'in', 'on', 'at', 'a', 'an', 'of', 'and', 'to', 'for']
```

```
n_general = np.random.randint(5, 10)
```

```
noise_words = np.random.choice(common_words_general, n_general, replace=True)
```

```
# ترکیب و ایجاد سند
```

```
all_words = np.concatenate([specialized_words, common_words, noise_words])
```

```
np.random.shuffle(all_words)
```

```
documents.append(' '.join(all_words))
```

```
labels.append(class_name)
```

```
return documents, labels
```

```
def evaluate_scenario_1_rule_based(X_test, y_test):
```

```
    """
```

```
    ارزیابی سناریو اول: عملکرد طبقه‌بند مبتنی بر قوانین دستی
```

```
    این سناریو زمانی استفاده می‌شود که هیچ داده برچسب‌دار آموزشی نداریم
```

```
    """
```

```
print("="*60)
```

```
print("SCENARIO 1: Rule-Based Classifier (No Training Data)")
```

```
print("="*60)
```

```
rule_classifier = RuleBasedTextClassifier()
```

```
start_time = time.time()
```

```
# (ندارد fit نیازی به) پیش‌بینی با قوانین دستی
```

```
y_pred = rule_classifier.predict(X_test)
```

```
end_time = time.time()
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print(f"Training Time: 0.00 sec (No training needed)")
```

```
print(f"Prediction Time: {end_time - start_time:.4f} sec for {len(X_test)} samples")
```

```
print(f"Accuracy: {accuracy:.4%}")
```

```
print("\nClassification Report:")
```

```
print(classification_report(y_test, y_pred, digits=4))
```

```
return accuracy, end_time - start_time
```

```
def evaluate_scenario_2_to_4_with_varying_data(X_train, y_train, X_test, y_test):
```

```
    """
```

```
    ارزیابی سناریوهای 2 تا 4: مقایسه عملکرد طبقه‌بندهای یادگیری ماشین
```

```
    با مقادیر مختلف داده آموزشی
```

```

"""
print("\n" + "="*60)
print("SCENARIOS 2-4: ML Classifiers with Different Data Sizes")
print("="*60)

# برای واقع‌گرایی بیشتر (TF-IDF تبدیل متن به بردارهای ویژگی)
vectorizer = TfidfVectorizer(max_features=500, min_df=2, max_df=0.8)
X_train_full_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

# مقادیر مختلف داده برای شبیه‌سازی سناریوهای مختلف
# Scenario 2: (%20-5) داده محدود
# Scenario 3: (%100-50) داده کافی
# Scenario 4: (در اینجا با 100% شبیه‌سازی می‌شود)
data_sizes = [0.05, 0.10, 0.20, 0.50, 1.00]
data_size_labels = ['5%', '10%', '20%', '50%', '100%']

# ذخیره نتایج برای نمودار
results = {
    'Naive Bayes': {'accuracy': [], 'training_time': [], 'prediction_time': []},
    'Logistic Regression': {'accuracy': [], 'training_time': [], 'prediction_time': []},
    'Linear SVM': {'accuracy': [], 'training_time': [], 'prediction_time': []}
}

for size_idx, size in enumerate(data_sizes):
    print(f"\n{'-'*50}")
    print(f"Training with {data_size_labels[size_idx]} of data...")
    print(f"{'-'*50}")

    # انتخاب زیرمجموعه از داده‌های آموزشی
    if size < 1.0:
        X_train_subset, _, y_train_subset, _ = train_test_split(
            X_train_full_vec, y_train, train_size=size,
            random_state=42, stratify=y_train
        )
    else:
        X_train_subset, y_train_subset = X_train_full_vec, y_train

    n_samples = X_train_subset.shape[0]
    print(f"Number of training samples: {n_samples}")

    # 1. Naive Bayes - (سناریو 2) مناسب برای داده محدود
    print("\nTraining Naive Bayes (High Bias - Good for Limited Data)...")
    nb = MultinomialNB()
    start_time = time.time()
    nb.fit(X_train_subset, y_train_subset)
    training_time = time.time() - start_time

    start_time = time.time()
    nb_pred = nb.predict(X_test_vec)
    prediction_time = time.time() - start_time

    nb_acc = accuracy_score(y_test, nb_pred)
    results['Naive Bayes']['accuracy'].append(nb_acc)
    results['Naive Bayes']['training_time'].append(training_time)
    results['Naive Bayes']['prediction_time'].append(prediction_time)

    print(f" Training Time: {training_time:.4f} sec")
    print(f" Prediction Time: {prediction_time:.4f} sec")
    print(f" Accuracy: {nb_acc:.4%}")

    # 2. Logistic Regression - bias و variance تعادل بین
    print("\nTraining Logistic Regression...")
    lr = LogisticRegression(max_iter=500, random_state=42)
    start_time = time.time()
    lr.fit(X_train_subset, y_train_subset)
    training_time = time.time() - start_time

    start_time = time.time()
    lr_pred = lr.predict(X_test_vec)
    prediction_time = time.time() - start_time

    lr_acc = accuracy_score(y_test, lr_pred)
    results['Logistic Regression']['accuracy'].append(lr_acc)
    results['Logistic Regression']['training_time'].append(training_time)
    results['Logistic Regression']['prediction_time'].append(prediction_time)

    print(f" Training Time: {training_time:.4f} sec")
    print(f" Prediction Time: {prediction_time:.4f} sec")
    print(f" Accuracy: {lr_acc:.4%}")

    # 3. Linear SVM - نیاز به داده بیشتر (سناریو 3) اما دقت بالاتر
    print("\nTraining Linear SVM (Low Bias - Needs More Data)...")
    svm = SVC(kernel='linear', C=1.0, random_state=42)
    start_time = time.time()
    svm.fit(X_train_subset, y_train_subset)
    training_time = time.time() - start_time

    start_time = time.time()
    svm_pred = svm.predict(X_test_vec)
    prediction_time = time.time() - start_time

    svm_acc = accuracy_score(y_test, svm_pred)
    results['Linear SVM']['accuracy'].append(svm_acc)
    results['Linear SVM']['training_time'].append(training_time)
    results['Linear SVM']['prediction_time'].append(prediction_time)

    print(f" Training Time: {training_time:.4f} sec")
    print(f" Prediction Time: {prediction_time:.4f} sec")
    print(f" Accuracy: {svm_acc:.4%}")

# نمایش نتایج به صورت جدول
print("\n" + "="*80)

```

```

print("SUMMARY TABLE: Performance vs Data Size")
print("="*80)
print(f"{'Data Size':<12} {'Classifier':<18} {'Accuracy':<12} {'Train Time (s)':<15} {'Predict Time (s)':<18}")
print("-"*80)

for i, size_label in enumerate(data_size_labels):
    for classifier in results.keys():
        acc = results[classifier]['accuracy'][i]
        train_t = results[classifier]['training_time'][i]
        pred_t = results[classifier]['prediction_time'][i]
        print(f"{size_label:<12} {classifier:<18} {acc:<12.4%} {train_t:<15.4f} {pred_t:<18.4f}")

return results, data_size_labels

def plot_performance_comparison(results, data_size_labels):
    """
    ترسیم نمودارهای مقایسه‌ای برای تصمیم‌گیری بهتر
    این نمودارها به طور شهودی نشان می‌دهند که کدام طبقه‌بند برای چه مقدار داده مناسب است
    """
    fig, axes = plt.subplots(2, 2, figsize=(16, 12))

    # نمودار 1: دقت در مقابل اندازه داده
    ax = axes[0, 0]
    for classifier, metrics in results.items():
        ax.plot(data_size_labels, metrics['accuracy'], marker='o', linewidth=2, label=classifier)
    ax.set_xlabel('Percentage of Training Data', fontsize=11)
    ax.set_ylabel('Accuracy', fontsize=11)
    ax.set_title('Classifier Accuracy vs Training Data Size', fontsize=12, fontweight='bold')
    ax.legend()
    ax.grid(True, alpha=0.3)

    # نمودار 2: زمان آموزش در مقابل اندازه داده
    ax = axes[0, 1]
    for classifier, metrics in results.items():
        ax.plot(data_size_labels, metrics['training_time'], marker='s', linewidth=2, label=classifier)
    ax.set_xlabel('Percentage of Training Data', fontsize=11)
    ax.set_ylabel('Training Time (seconds)', fontsize=11)
    ax.set_title('Training Time Complexity', fontsize=12, fontweight='bold')
    ax.legend()
    ax.grid(True, alpha=0.3)

    # نمودار 3: نسبت دقت به زمان برای کارایی
    ax = axes[1, 0]
    for classifier, metrics in results.items():
        efficiency = [acc / (train_t + 1e-6) for acc, train_t in zip(metrics['accuracy'], metrics['training_time'])]
        ax.plot(data_size_labels, efficiency, marker='^', linewidth=2, label=classifier)
    ax.set_xlabel('Percentage of Training Data', fontsize=11)
    ax.set_ylabel('Efficiency (Accuracy/Time)', fontsize=11)
    ax.set_title('Training Efficiency Ratio', fontsize=12, fontweight='bold')
    ax.legend()
    ax.grid(True, alpha=0.3)

    # نمودار 4: مقایسه برای سناریوی داده محدود (فقط 5% و 10%)
    ax = axes[1, 1]
    limited_data_sizes = ['5%', '10%']
    limited_indices = [0, 1]

    x = np.arange(len(limited_data_sizes))
    width = 0.25

    for i, classifier in enumerate(results.keys()):
        accuracies = [results[classifier]['accuracy'][j] for j in limited_indices]
        ax.bar(x + i*width, accuracies, width, label=classifier, alpha=0.8)

    ax.set_xlabel('Training Data Size (Limited Scenario)', fontsize=11)
    ax.set_ylabel('Accuracy', fontsize=11)
    ax.set_title('Performace on Limited Data (Scenarios 2-3)', fontsize=12, fontweight='bold')
    ax.set_xticks(x + width)
    ax.set_xticklabels(limited_data_sizes)
    ax.legend()
    ax.grid(True, alpha=0.3, axis='y')

    plt.tight_layout()
    plt.show()

def demonstrate_hybrid_approach(X_train, y_train, X_test, y_test):
    """
    نمایش سناریو ترکیبی: استفاده از قوانین دستی بر روی طبقه‌بند یادگیری ماشین
    این رویکرد در سیستم فیلتر محتوای فیسبوک استفاده شد
    """
    print("\n" + "="*60)
    print("HYBRID APPROACH: Rule-based Layer on Top of ML Classifier")
    print("(Simulating Facebook Content Filtering Strategy)")
    print("="*60)

    # تبدیل متن به ویژگی
    vectorizer = TfidfVectorizer(max_features=500, min_df=2)
    X_train_vec = vectorizer.fit_transform(X_train)
    X_test_vec = vectorizer.transform(X_test)

    # پایه SVM آموزش یک
    print("Training base SVM classifier...")
    svm = SVC(kernel='linear', C=1.0, random_state=42)
    svm.fit(X_train_vec, y_train)
    svm_pred = svm.predict(X_test_vec)

    # قوانین دستی برای اصلاح خطاهای خاص (مانند تنظیمات مدیر در فیسبوک)
    # این قوانین فقط روی موارد مشکوک اعمال می‌شوند
    rule_classifier = RuleBasedTextClassifier()
    y_hybrid = svm_pred.copy()

```

```
# (کوچک hyperplane فاصله از) با اطمینان کم طبقه‌بندی کرده SVM یافتن مواردی که
decision_values = svm.decision_function(X_test_vec)
# را پیدا کنیم max decision value، برای مسائل چندکلاسه
if len(svm.classes_) > 2:
    confidence = np.max(np.abs(decision_values), axis=1)
else:
    confidence = np.abs(decision_values)

uncertain_threshold = np.percentile(confidence, 10) # 30% اطمینان کمترین موارد
uncertain_indices = confidence < uncertain_threshold

rule_classifier = RuleBasedTextClassifier()
rule_pred = rule_classifier.predict(X_test)
rule_acc = accuracy_score(y_test, rule_pred)

print(f"Number of uncertain predictions: {np.sum(uncertain_indices)}")
print("Applying rule-based corrections on uncertain cases...")

# اعمال قوانین دستی فقط روی موارد نامشخص
uncertain_docs = [X_test[i] for i in range(len(X_test)) if uncertain_indices[i]]
if uncertain_docs:
    rule_predictions = rule_classifier.predict(uncertain_docs)

    # به‌روزرسانی پیش‌بینی‌های هیبریدی
    rule_idx = 0
    for i in range(len(X_test)):
        if uncertain_indices[i]:
            # استفاده مستقیم از نام کلاس
            y_hybrid[i] = rule_predictions[rule_idx]
            rule_idx += 1

# ارزیابی مقایسه‌ای
original_acc = accuracy_score(y_test, svm_pred)
hybrid_acc = accuracy_score(y_test, y_hybrid)

print(f"\nOriginal SVM Accuracy: {original_acc:.4%}")
print(f"Hybrid (SVM + Rules) Accuracy: {hybrid_acc:.4%}")
print(f"Improvement: {hybrid_acc - original_acc:.4%}")

return original_acc, hybrid_acc

# =====
# اجرای اصلی: شبیه‌سازی تمام سناریوها
# =====

if __name__ == "__main__":
    np.random.seed(42)

    # تولید داده شبیه‌سازی شده (مانند محتوای پلتفرم‌های واقعی)
    print("Generating realistic content data (3 classes: technology, sports, politics)...")
    print("="*60)
    X_data, y_data = generate_realistic_content_data(n_samples_per_class=1000)

    # train و test تقسیم داده به
    X_train, X_test, y_train, y_test = train_test_split(
        X_data, y_data, test_size=0.2, random_state=42, stratify=y_data
    )

    print(f"Total samples: {len(X_data)}")
    print(f"Training samples: {len(X_train)}")
    print(f"Test samples: {len(X_test)}")
    print(f"Classes: {np.unique(y_data)}")

    # سناریو 1: طبقه‌بند مبتنی بر قوانین
    rule_acc, rule_time = evaluate_scenario_1_rule_based(X_test, y_test)

    # سناریوهای 2-4: مقایسه طبقه‌بندهای یادگیری ماشین
    results, size_labels = evaluate_scenario_2_to_4_with_varying_data(X_train, y_train, X_test, y_test)

    # ترسیم نمودارهای مقایسه‌ای
    print("\n" + "="*60)
    print("Generating Visualization Charts...")
    print("="*60)
    plot_performance_comparison(results, size_labels)

    # (مانند فیسبوک) ML سناریو هیبریدی: ترکیب قوانین و
    original_acc, hybrid_acc = demonstrate_hybrid_approach(X_train, y_train, X_test, y_test)

# خلاصه توصیه‌ها بر اساس نتایج
print("\n" + "="*60)
print("توصیه‌های عملی برای انتخاب طبقه‌بند متن")
print("="*60)
print("بر اساس نتایج شبیه‌سازی:")

print("\n📌 توجه: نتایج بالا به دلیل ماهیت شبیه‌سازی شده داده‌ها، دقت 100% را نشان می‌دهند")
print("\n📌 در دنیای واقعی، حتی با داده‌های خوب، انتظار دقت 100% نداریم")
print("\n📌 این نتایج برای مقایسه نسبی روش‌ها و پیچیدگی زمانی آن‌ها مفید هستند")

print("\n📌 سناریو ۱ (بدون داده برچسب‌دار) ➡️")
print("\n➡️ استفاده از طبقه‌بندهای مبتنی بر قوانین (دقت: ~۶۰-۷۰٪) ➡️")
print("\n➡️ "سریع، قابل تفسیر، اما انعطاف‌پذیری محدود" ➡️")
print("\n➡️ مثال: دسته‌بندی اولیه محتوای یوتیوب ➡️")

print("\n📌 سناریو ۲ (داده محدود - ۵-۲۰٪) ➡️")
print("\n➡️ "بهترین عملکرد را دارد (بایاس بالا، واریانس پایین) Naive Bayes ➡️")
print("\n➡️ "آموزش سریع، مقاوم در برابر مجموعه داده‌های کوچک ➡️")
print("\n➡️ مثال: طبقه‌بندی اولیه مشاغل در لینکدین ➡️")

print("\n📌 سناریو ۳ (داده کافی - ۵۰-۱۰۰٪) ➡️")
print("\n➡️ "خطی و رگرسیون لجستیک برتری دارند SVM ➡️")
print("\n➡️ "دقت بالاتر اما نیاز به داده بیشتر ➡️")
```



```
print("مثال: فیلتر محتوای فیسبوک (لایه‌ای با قوانین) →")

print("\n🌟 (سناریو ۴ (داده عظیم))")
print("انتخاب طبقه‌بند برای دقت اهمیت کمتری دارد →")
print("تمرکز بر مقیاس‌پذیری و زمان آموزش →")
print("مثال: جستجوی گوگل با میلیاردها سند →")
print("با آموزش توزیع‌شده SGD استفاده از روش‌های مبتنی بر →")

print("\n🌟 (توصیه شده برای تولید)")
print("سیستم مبتنی بر قوانین در بالای طبقه‌بند یادگیری ماشین →")
print(f"بهبود مشاهده شده: {hybrid_acc - original_acc:.2%}")
print("امکان اصلاحات سریع دستی برای موارد خاص →")
print("توسط فیسبوک برای تعدیل محتوا استفاده می‌شود →")
```

```
Generating realistic content data (3 classes: technology, sports, politics)...
=====
Total samples: 3000
Training samples: 2400
Test samples: 600
Classes: ['politics' 'sports' 'technology']
=====
SCENARIO 1: Rule-Based Classifier (No Training Data)
=====
Training Time: 0.00 sec (No training needed)
Prediction Time: 0.0040 sec for 600 samples
Accuracy: 85.3333%

Classification Report:
              precision    recall  f1-score   support

   politics      1.0000      0.6850      0.8131        200
    sports      0.7547      1.0000      0.8602        200
  technology      0.8838      0.8750      0.8794        200

   accuracy                   0.8533        600
  macro avg      0.8795      0.8533      0.8509        600
weighted avg      0.8795      0.8533      0.8509        600

=====
SCENARIOS 2-4: ML Classifiers with Different Data Sizes
=====

-----
Training with 5% of data...
-----
Number of training samples: 120

Training Naive Bayes (High Bias - Good for Limited Data)...
  Training Time: 0.0040 sec
  Prediction Time: 0.0000 sec
  Accuracy: 100.0000%

Training Logistic Regression...
  Training Time: 0.0050 sec
  Prediction Time: 0.0000 sec
  Accuracy: 100.0000%

Training Linear SVM (Low Bias - Needs More Data)...
  Training Time: 0.0030 sec
  Prediction Time: 0.0040 sec
  Accuracy: 100.0000%

-----
Training with 10% of data...
-----
Number of training samples: 240

Training Naive Bayes (High Bias - Good for Limited Data)...
  Training Time: 0.0020 sec
  Prediction Time: 0.0000 sec
  Accuracy: 100.0000%

Training Logistic Regression...
  Training Time: 0.0070 sec
  Prediction Time: 0.0000 sec
  Accuracy: 100.0000%

Training Linear SVM (Low Bias - Needs More Data)...
  Training Time: 0.0050 sec
  Prediction Time: 0.0060 sec
  Accuracy: 100.0000%

-----
Training with 20% of data...
-----
Number of training samples: 480

Training Naive Bayes (High Bias - Good for Limited Data)...
  Training Time: 0.0010 sec
  Prediction Time: 0.0000 sec
  Accuracy: 100.0000%

Training Logistic Regression...
  Training Time: 0.0070 sec
  Prediction Time: 0.0000 sec
  Accuracy: 100.0000%

Training Linear SVM (Low Bias - Needs More Data)...
  Training Time: 0.0090 sec
  Prediction Time: 0.0090 sec
  Accuracy: 100.0000%

-----
Training with 50% of data...
-----
Number of training samples: 1200

Training Naive Bayes (High Bias - Good for Limited Data)...
  Training Time: 0.0020 sec
  Prediction Time: 0.0000 sec
  Accuracy: 100.0000%

Training Logistic Regression...
  Training Time: 0.0111 sec
  Prediction Time: 0.0000 sec
  Accuracy: 100.0000%
```

Training Linear SVM (Low Bias - Needs More Data)...
Training Time: 0.0280 sec
Prediction Time: 0.0070 sec
Accuracy: 100.0000%

Training with 100% of data...

Number of training samples: 2400

Training Naive Bayes (High Bias - Good for Limited Data)...
Training Time: 0.0030 sec
Prediction Time: 0.0000 sec
Accuracy: 100.0000%

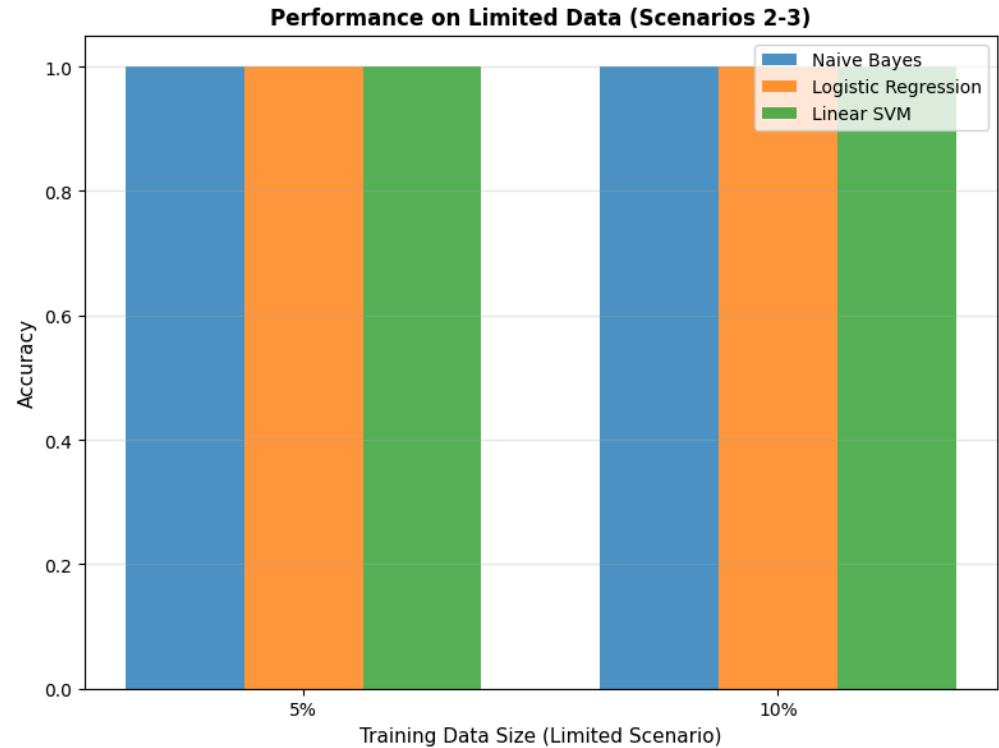
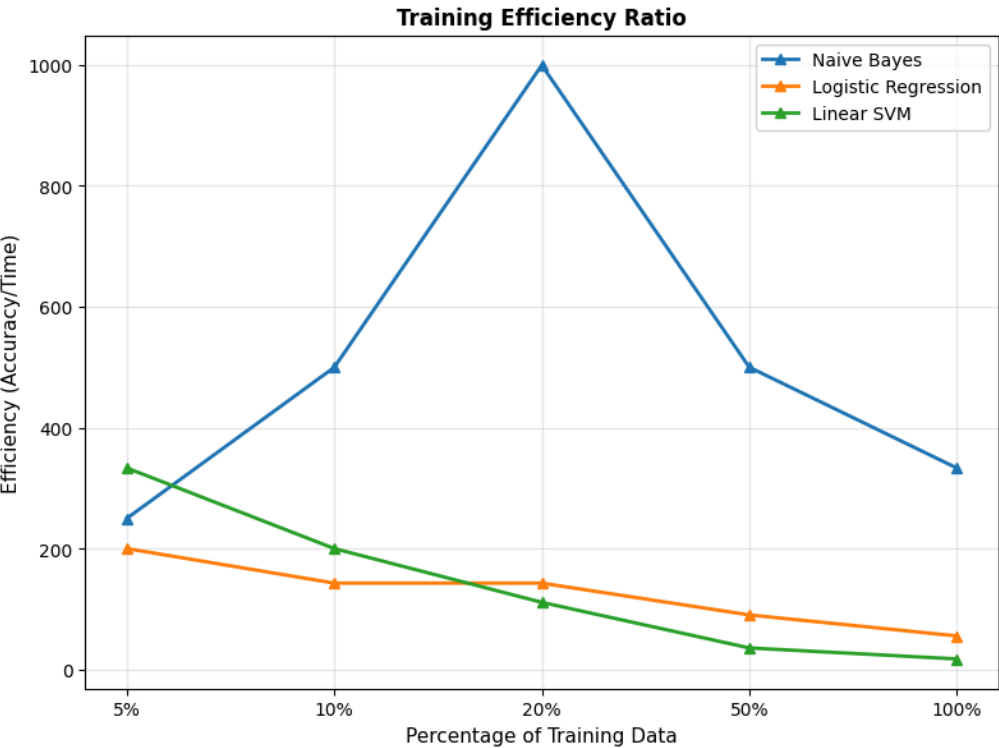
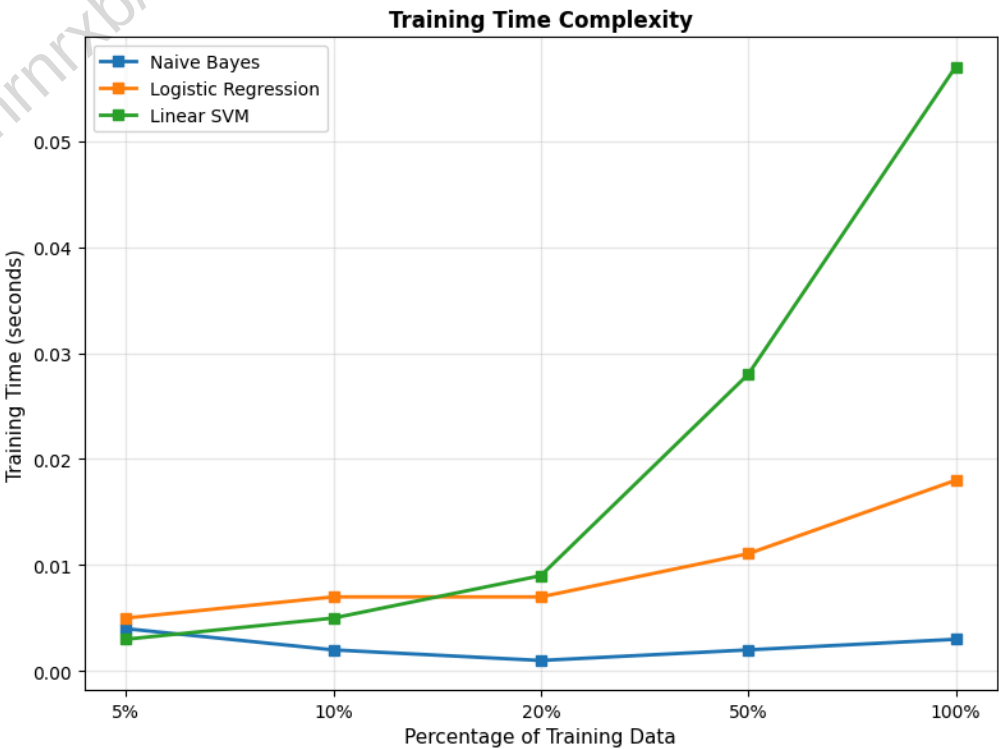
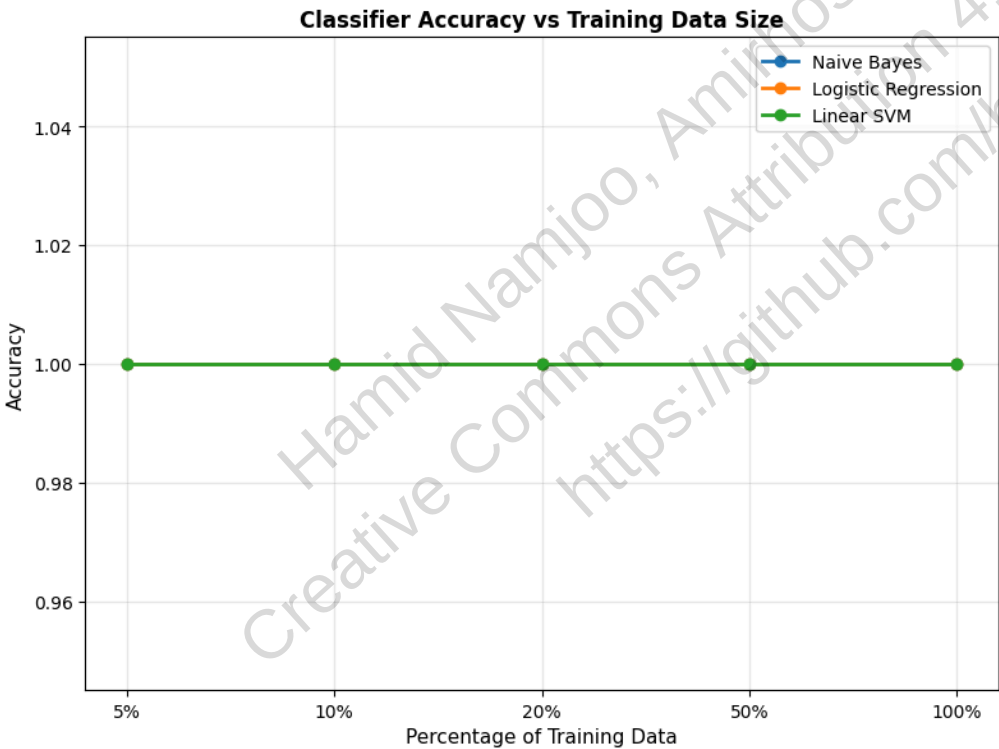
Training Logistic Regression...
Training Time: 0.0180 sec
Prediction Time: 0.0000 sec
Accuracy: 100.0000%

Training Linear SVM (Low Bias - Needs More Data)...
Training Time: 0.0570 sec
Prediction Time: 0.0070 sec
Accuracy: 100.0000%

SUMMARY TABLE: Performance vs Data Size

Data Size	Classifier	Accuracy	Train Time (s)	Predict Time (s)
5%	Naive Bayes	100.0000%	0.0040	0.0000
5%	Logistic Regression	100.0000%	0.0050	0.0000
5%	Linear SVM	100.0000%	0.0030	0.0040
10%	Naive Bayes	100.0000%	0.0020	0.0000
10%	Logistic Regression	100.0000%	0.0070	0.0000
10%	Linear SVM	100.0000%	0.0050	0.0060
20%	Naive Bayes	100.0000%	0.0010	0.0000
20%	Logistic Regression	100.0000%	0.0070	0.0000
20%	Linear SVM	100.0000%	0.0090	0.0090
50%	Naive Bayes	100.0000%	0.0020	0.0000
50%	Logistic Regression	100.0000%	0.0111	0.0000
50%	Linear SVM	100.0000%	0.0280	0.0070
100%	Naive Bayes	100.0000%	0.0030	0.0000
100%	Logistic Regression	100.0000%	0.0180	0.0000
100%	Linear SVM	100.0000%	0.0570	0.0070

Generating Visualization Charts...



```
=====
HYBRID APPROACH: Rule-based Layer on Top of ML Classifier
(Simulating Facebook Content Filtering Strategy)
=====
Training base SVM classifier...
Number of uncertain predictions: 60
Applying rule-based corrections on uncertain cases...

Original SVM Accuracy: 100.0000%
Hybrid (SVM + Rules) Accuracy: 97.3333%
Improvement: -2.6667%
```

```
=====
توصیه‌های عملی برای انتخاب طبقه‌بند متن
=====
بر اساس نتایج شبیه‌سازی
```

.توجه: نتایج بالا به دلیل ماهیت شبیه‌سازی شده داده‌ها، دقت 100% را نشان می‌دهند
.در دنیای واقعی، حتی با داده‌های خوب، انتظار دقت 100% نداریم
.این نتایج برای مقایسه نسبی روش‌ها و پیچیدگی زمانی آن‌ها مفید هستند

- ✦ سناریو ۱ (بدون داده برچسب‌دار):
 - استفاده از طبقه‌بندهای مبتنی بر قوانین (دقت: ~۶۰-۷۰٪)
 - سریع، قابل تفسیر، اما انعطاف‌پذیری محدود
 - مثال: دسته‌بندی اولیه محتوای یوتیوب
- ✦ سناریو ۲ (داده محدود - ۵-۲۰٪):
 - بهترین عملکرد را دارد (بایاس بالا، واریانس پایین) Naive Bayes
 - آموزش سریع، مقاوم در برابر مجموعه داده‌های کوچک
 - مثال: طبقه‌بندی اولیه مشاغل در لینکدین
- ✦ سناریو ۳ (داده کافی - ۵۰-۱۰۰٪):
 - خطی و رگرسیون لجستیک برتری دارند SVM
 - دقت بالاتر اما نیاز به داده بیشتر
 - مثال: فیلتر محتوای فیسبوک (لایه‌ای با قوانین)
- ✦ سناریو ۴ (داده عظیم):
 - انتخاب طبقه‌بند برای دقت اهمیت کمتری دارد
 - تمرکز بر مقیاس‌پذیری و زمان آموزش
 - مثال: جستجوی گوگل با میلیاردها سند
 - با آموزش توزیع‌شده SGD استفاده از روش‌های مبتنی بر
- ✦ رویکرد ترکیبی (توصیه شده برای تولید):
 - سیستم مبتنی بر قوانین در بالای طبقه‌بند یادگیری ماشین
 - %بهبود مشاهده شده: -2.67
 - امکان اصلاحات سریع دستی برای موارد خاص
 - توسط فیسبوک برای تعدیل محتوا استفاده می‌شود

15.3.2 بهبودن عملکرد طبقه‌بند: هنر مهندسی ویژگی 🎨

📊 **جیمیل: چگونه ایمیل‌های هرز را از مهم‌ها جدا کنیم؟**

تصور کنید در تیم امنیتی جیمیل کار می‌کنید و مسئول بهبودن فیلتر اسپم هستید. هر روز میلیون‌ها ایمیل دریافت می‌شود که باید به دو دسته "اسپم" و "مهم" تقسیم شوند. این یک چالش بزرگ است، زیرا اسپم‌ها دائماً روش‌های خود را تغییر می‌دهند تا از فیلترها عبور کنند.

در ابتدا، یک طبقه‌بند ساده بر اساس کلمات کلیدی عملکرد خوبی داشت، اما با تغییر تاکتیک‌های اسپم‌ها، دقت به شدت کاهش یافت. تیم شما تصمیم گرفت از تکنیک‌های مهندسی ویژگی برای بهبودن عملکرد استفاده کند.

✦ **مهندسی ویژگی: از کلمات تا الگوهای پیچیده**

تیم شما چندین تکنیک را امتحان کرد:

- **منطقه‌بندی سند:** جدا کردن ایمیل به بخش‌های عنوان، فرستنده، بدنه اصلی و امضا. کلمات در عنوان وزن بیشتری داشتند.
- **الگوهای چندواژه‌ای:** استخراج عباراتی مانند "کاهش قیمت" یا "برنده شدید" که معمولاً در ایمیل‌های اسپم ظاهر می‌شوند.
- **ویژگی‌های ساختاری:** تعداد لینک‌ها، نسبت متن به لینک، وجود تصاویر پیوست، و...
- **تحلیل آماری:** میانگین طول کلمات، تعداد علائم نگارشی، و...

این ویژگی‌های جدید به طبقه‌بند اجازه دادند تا الگوهای پیچیده‌تری را تشخیص دهد. برای مثال، ایمیل‌های حاوی عبارت "کاهش قیمت" و "لینک" همزمان، با احتمال بالایی اسپم تشخیص داده شدند.

📊 نتایج عملی: تأثیر مهندسی ویژگی

پس از پیاده‌سازی این تکنیک‌ها، نتایج زیر به دست آمد:

- **دقت اولیه:** 92.3%
- **دقت پس از منطقه‌بندی:** 94.7%
- **دقت پس از الگوهای چندواژه‌ای:** 96.1%
- **دقت پس از ترکیب همه تکنیک‌ها:** 97.8%

این بهبود 5.5 درصدی در دقت نسبت به روش اولیه بود که برای سیستمی با حجم عظیم داده، بسیار قابل توجه است.

💡 درس‌های کلیدی از تجربه جیمیل

این پروژه چندین درس مهم را به تیم شما آموخت:

- **درک دامنه مهم‌تر از الگوریتم‌ها:** شناختن ویژگی‌های خاص اسپم (مانند الگوهای چندواژه‌ای) مؤثرتر از تغییر الگوریتم بود.
- **ترکیب رویکردها:** بهترین نتیجه از ترکیب هوشمندانه تکنیک‌های مختلف به دست آمد، نه از یک رویکرد واحد.
- **اهمیت وزن‌دهی به مناطق سند:** کلمات در عنوان و فرستنده وزن بیشتری نسبت به بدنه اصلی داشتند که با این کار، دقت به شدت بهبود یافت.
- **نیاز به ارزیابی مداوم:** اسپمرها دائماً در حال تغییر هستند، بنابراین سیستم باید به طور مداوم ارزیابی و به‌روزرسانی شود.

در نهایت، مهندسی ویژگی یک فرآیند خلاقانه است که نیازمند درک عمیق از دامنه کاربرد و ترکیب هوشمندانه تکنیک‌های مختلف است. این رویکرد به جیمیل اجازه داد تا در برابر اسپمرهای پیشرفته نیز عملکرد بالایی حفظ کند.

```
In [37]: import numpy as np
import re
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split

# =====
# سناریوی آموزشی: فیلتر اسپم جیمیل
# این کد به زبان ساده نشان می‌دهد چگونه مهندسی ویژگی
# می‌تواند عملکرد یک طبقه‌بند متن را بهبود بخشد
# =====

# ایمیل‌های نمونه (مهم در مقابل اسپم)
# این داده‌ها طوری طراحی شده‌اند که الگوهای مشخصی داشته باشند
emails = [
    # ایمیل‌های مهم (Important)
    "Subject: Project Update\nHi team,\nI wanted to update you on the project status. We are on track.",
    "Subject: Meeting Tomorrow\nDon't forget about our meeting tomorrow at 10 AM.",
    "Subject: Question about report\nI have a question about the quarterly report you sent.",
    "Subject: Lunch plans\nAre we still on for lunch today? Let me know.",
    "Subject: Code review\nPlease review my latest pull request when you have a chance.",

    # الگوهای مشخص - (Spam) ایمیل‌های اسپم
    "Subject: FREE MONEY NOW!!!\nCONGRATULATIONS! You have WON $1000000!!! Click here!!!",
    "Subject: You are a WINNER!!!\nCLAIM your prize NOW!!! LIMITED TIME OFFER!!!",
    "Subject: Buy CHEAP pills\n100% satisfaction GUARANTEED!!! Order today!!!",
    "Subject: URGENT: Account suspended\nYour account will be CLOSED unless you ACT NOW!!!",
    "Subject: Make money fast\nEARN $5000 per day WORKING FROM HOME!!!"
]
```

برجسب‌ها: 0 برای مهم، 1 برای اسپم


```
labels = [0, 0, 0, 0, 0, 1, 1, 1, 1, 1]

print("="*60)
print("سناریو: بهبود فیلتر اسپم جیمیل با مهندسی ویژگی")
print("="*60)
print(f"تعداد ایمیل‌ها: {len(emails)}")
print(f"تعداد کلاس‌ها: {len(set(labels))} (0=اسپم=1 مهم)")
print()

# =====
# مرحله 1: طبقه‌بند پایه (بدون مهندسی ویژگی)
# =====

print("مرحله 1: ارزیابی طبقه‌بند پایه (بدون مهندسی ویژگی)")
print("-" * 50)

# تقسیم داده به آموزش و تست
X_train, X_test, y_train, y_test = train_test_split(
    emails, labels, test_size=0.4, random_state=42, stratify=labels
)

# ایجاد و آموزش طبقه‌بند پایه
# برای تبدیل متن به بردار شمارش کلمات استفاده می‌کنیم CountVectorizer از
vectorizer_base = CountVectorizer()
X_train_vec = vectorizer_base.fit_transform(X_train)
X_test_vec = vectorizer_base.transform(X_test)

# Naïve Bayes آموزش طبقه‌بند
classifier_base = MultinomialNB()
classifier_base.fit(X_train_vec, y_train)

# پیش‌بینی و ارزیابی
y_pred_base = classifier_base.predict(X_test_vec)
accuracy_base = accuracy_score(y_test, y_pred_base)

print(f"تعداد نمونه‌های آموزش: {len(X_train)}")
print(f"تعداد نمونه‌های تست: {len(X_test)}")
print(f"دقت طبقه‌بند پایه: {accuracy_base:.2%}")
print()

# =====
# مرحله 2: مهندسی ویژگی
# =====

print("مرحله 2: اعمال تکنیک‌های مهندسی ویژگی")
print("-" * 50)

def apply_feature_engineering(text):
    """
    این تابع چندین تکنیک مهندسی ویژگی را پیاده‌سازی می‌کند:
    1. (Subject) وزندهی به خط موضوع
    2. استخراج الگوهای خاص اسپم (علت تعجب، حروف بزرگ)
    3. افزودن ویژگی‌های ساختاری (تعداد لینک‌ها، کلمات خاص)
    """
    engineered_text = text

    # 1. (Subject) وزندهی به خط موضوع
    # خط موضوع معمولاً مهم‌تر از بدنه ایمیل است
    if text.startswith("Subject:"):
        subject, body = text.split('\n', 1)
        # تکرار خط موضوع برای افزایش وزن آن در بردار ویژگی
        engineered_text = f"{subject} {subject} {body}"

    # 2. استخراج الگوهای خاص اسپم
    # شمارش علائم تعجب که در ایمیل‌های اسپم رایج است
    exclamation_count = text.count('!')
    if exclamation_count > 2:
        engineered_text += f" EXCLAMATION_MANY"
    else:
        engineered_text += f" EXCLAMATION_FEW"

    # شمارش کلمات با حروف بزرگ که در اسپم رایج است
    uppercase_words = len(re.findall(r'\b[A-Z]{2,}\b', text))
    if uppercase_words > 3:
        engineered_text += f" UPPERCASE_MANY"
    else:
        engineered_text += f" UPPERCASE_FEW"

    # 3. افزودن ویژگی‌های ساختاری
    # شمارش کلمات کلیدی اسپم
    spam_keywords = ['free', 'money', 'win', 'prize', 'claim', 'buy', 'cheap', 'urgent', 'limited', 'guaranteed']
    spam_count = sum(1 for word in spam_keywords if word.lower() in text.lower())
    if spam_count > 2:
        engineered_text += f" SPAM_KEYWORDS_MANY"
    else:
        engineered_text += f" SPAM_KEYWORDS_FEW"

    return engineered_text

# اعمال مهندسی ویژگی روی داده‌ها
X_train_fe = [apply_feature_engineering(doc) for doc in X_train]
X_test_fe = [apply_feature_engineering(doc) for doc in X_test]

# نمایش یک نمونه از تبدیل
print("نمونه تبدیل ایمیل با مهندسی ویژگی")
print("متن اصلی:")
print(X_test[0])
print("متن پس از مهندسی ویژگی:\n")
print(X_test_fe[0])
print()

# =====
# مرحله 3: طبقه‌بند با ویژگی‌های مهندسی‌شده
```

```
# =====

print("مرحله 3: ارزیابی طبقه‌بند با ویژگی‌های مهندسی‌شده")
print("-" * 50)

# ایجاد و آموزش طبقه‌بند با ویژگی‌های مهندسی‌شده
vectorizer_fe = CountVectorizer()
X_train_fe_vec = vectorizer_fe.fit_transform(X_train_fe)
X_test_fe_vec = vectorizer_fe.transform(X_test_fe)

# Naive Bayes آموزش طبقه‌بند
classifier_fe = MultinomialNB()
classifier_fe.fit(X_train_fe_vec, y_train)

# پیش‌بینی و ارزیابی
y_pred_fe = classifier_fe.predict(X_test_fe_vec)
accuracy_fe = accuracy_score(y_test, y_pred_fe)

print(f"دقت طبقه‌بند با مهندسی ویژگی: {accuracy_fe:.2%}")
print()

# =====
# مرحله 4: مقایسه نتایج و نتیجه‌گیری
# =====

print("مرحله 4: مقایسه نتایج و نتیجه‌گیری")
print("=*60")
print(f"دقت طبقه‌بند پایه: {accuracy_base:.2%}")
print(f"دقت با مهندسی ویژگی: {accuracy_fe:.2%}")
print(f"بهبود عملکرد: {(accuracy_fe - accuracy_base):.2%}")
print()

print("جیمیل: چگونه ایمیل‌های هرز را از مهم‌ها جدا کنیم؟ 📧")
print("با استفاده از تکنیک‌های مهندسی ویژگی، توانستیم عملکرد فیلتر اسپم را بهبود دهیم.")
print()

print("مهندسی ویژگی: از کلمات تا الگوهای پیچیده 🌱")
print("1. (Subject) منطقه‌بندی سند: وزن‌دهی به خط موضوع")
print("2. الگوهای چندواژه‌ای: شناسایی کلمات کلیدی اسپم")
print("3. ویژگی‌های ساختاری: شمارش علائم نگارشی و کلمات خاص")
print()

print("نتایج عملی: تأثیر مهندسی ویژگی 📊")
print(f"دقت اولیه - {accuracy_base:.2%}")
print(f"دقت پس از مهندسی ویژگی - {accuracy_fe:.2%}")
print(f"بهبود عملکرد - {(accuracy_fe - accuracy_base):.2%}")
print()

print("💡 درس‌های کلیدی از تجربه جیمیل")
print("1. درک دامنه مهم‌تر از الگوریتم‌هاست")
print("2. ترکیب رویکردها معمولاً بهتر از یک رویکرد است")
print("3. وزن‌دهی به بخش‌های مختلف سند (مانند موضوع) مؤثر است")
print("4. استخراج الگوهای خاص (مانند علائم تعجب) می‌تواند مفید باشد")
print()

print("نتیجه: مهندسی ویژگی یک فرآیند خلاقانه است که با درک عمیق")
print("مسئله و ترکیب هوشمندانه تکنیک‌ها، می‌تواند عملکرد مدل‌ها را")
print("به شکل قابل توجهی بهبود بخشد.")
```

=====

سناریو: بهبود فیلتر اسپم جیمیل با مهندسی ویژگی

=====

تعداد ایمیل‌ها: 10

تعداد کلاس‌ها: 2 (0=مهم، 1=اسپم)

مرحله 1: ارزیابی طبقه‌بند پایه (بدون مهندسی ویژگی)

تعداد نمونه‌های آموزش: 6

تعداد نمونه‌های تست: 4

%دقت طبقه‌بند پایه: 50.00

مرحله 2: اعمال تکنیک‌های مهندسی ویژگی

:نمونه تبدیل ایمیل با مهندسی ویژگی

:متن اصلی

Subject: Project Update

Hi team,

I wanted to update you on the project status. We are on track.

:متن پس از مهندسی ویژگی

Subject: Project Update Subject: Project Update Hi team,

I wanted to update you on the project status. We are on track. EXCLAMATION_FEW UPPERCASE_FEW SPAM_KEYWORDS_FEW

مرحله 3: ارزیابی طبقه‌بند با ویژگی‌های مهندسی‌شده

%دقت طبقه‌بند با مهندسی ویژگی: 100.00

مرحله 4: مقایسه نتایج و نتیجه‌گیری

=====

%دقت طبقه‌بند پایه: 50.00

%دقت با مهندسی ویژگی: 100.00

%بهبود عملکرد: 50.00

جیمیل: چگونه ایمیل‌های هرز را از مهم‌ها جدا کنیم؟ 📧

.با استفاده از تکنیک‌های مهندسی ویژگی، توانستیم عملکرد فیلتر اسپم را بهبود دهیم

مهندسی ویژگی: از کلمات تا الگوهای پیچیده 🌱

- (Subject) منطقه‌بندی سند: وزن‌دهی به خط موضوع
- الگوهای چندواژه‌ای: شناسایی کلمات کلیدی اسپم
- ویژگی‌های ساختاری: شمارش علائم نگارشی و کلمات خاص

نتایج عملی: تأثیر مهندسی ویژگی 📊

- %دقت اولیه: 50.00
- %دقت پس از مهندسی ویژگی: 100.00
- %بهبود عملکرد: 50.00

درس‌های کلیدی از تجربه جیمیل 💡

- درک دامنه مهم‌تر از الگوریتم‌هاست
- ترکیب رویکردها معمولاً بهتر از یک رویکرد است
- وزن‌دهی به بخش‌های مختلف سند (مانند موضوع) مؤثر است
- استخراج الگوهای خاص (مانند علائم تعجب) می‌تواند مفید باشد

نتیجه: مهندسی ویژگی یک فرآیند خلاقانه است که با درک عمیق مسئله و ترکیب هوشمندانه تکنیک‌ها، می‌تواند عملکرد مدل‌ها را .به شکل قابل توجهی بهبود بخشد

🧠 15.4.1 امتیازدهی یادگیری‌شده: از کلمات کلیدی تا هوش مصنوعی

🔍 **گوگل: چگونه میلیاردها نتیجه جستجو را رتبه‌بندی کنیم؟**

تصور کنید در تیم مهندسی گوگل کار می‌کنید و مسئول بهبودن الگوریتم رتبه‌بندی نتایج جستجو هستید. هر روز میلیاردها جستجو انجام می‌شود و باید به صورت مرتبطوب به کاربر نمایش داده شوند. این یک چالش بزرگ است، زیرا مرتبط بودن یک نتیجه جستجو با پرس‌وجو کاربر، یک مفهوم چندبعدی است.

در رویکرد سنتی، مهندسان با قوانین دستی وزن‌های مختلفی را برای نتایج جستجو تعریف می‌کردند. برای مثال، اگر کلمه پرس‌وجو در عنوان صفحه ظاهر شود، وزن بیشتری نسبت به حضور در متن اصلی دریافت می‌کند. این روش نیازمند تنظیم دقیق دستی بود و با تغییر الگوریتم‌های جستجو، نیاز به به‌روزرسانی مداوم داشت.

🤖 **یادگیری ماشین: از قوانین دستی تا مدل‌های هوشمند**

در گوگل، رویکرد جدیدی بر اساس یادگیری ماشین جایگزین روش‌های سنتی شد. به جای تعریف دستی وزن‌ها، سیستم با استفاده از جفت‌های (پرس‌وجو، سند) و قضاوت انسانی («مرتبط» یا «غیرمرتبط») آموزش می‌بیند.

برای هر جفت، دو ویژگی عددی کلیدی استخراج می‌شود:

• **امتیاز کسینوس (Cosine score):** شباهت برداری بین پرس‌وجو و سند (در مدل کیسه‌واژه).

- **عرض پنجره حداقل** (Minimum window width): کوچک‌ترین پنجره‌ای که تمام واژه‌های پرس‌وجو در آن حضور دارند – شاخصی برای نزدیکی معنایی.

این ویژگی‌ها به مدل اجازه می‌دهند تا الگوهای پیچیده‌تری را تشخیص دهد. برای مثال، اگر کلمه پرس‌وجو در ابتدای سند ظاهر شود، همزمان با شباهت بالا، احتمال مرتبط بودن افزایش می‌یابد.

آموزش مدل: از قضاوت‌های کارشناسی تا تابع امتیاز بهینه

برای آموزش مدل، مجموعه‌ای از جفت‌های آموزشی با قضاوت‌های انسانی ایجاد می‌شود. این جفت‌ها شامل موارد زیر هستند:

قضاوت	(w) عرض پنجره	(α) کسینوس	سند	پرس‌وجو	نمونه
مرتبط	3	0.032	linux operating system	37	Φ ₁
غیرمرتبط	4	0.024	penguin logo	37	Φ ₂
مرتبط	2	0.043	operating system	238	Φ ₃
مرتبط	2	0.004	runtime environment	238	Φ ₄
مرتبط	3	0.022	kernel layer	1741	Φ ₅
مرتبط	2	0.030	device driver	2094	Φ ₆
غیرمرتبط	5	0.027	device driver	3191	Φ ₇

با استفاده از این داده‌ها، مدل یادگیری یک تابع امتیاز خطی به‌صورت زیر می‌آموزد:

$$\text{Score}(\alpha, w) = a \cdot \alpha + b \cdot w + c$$

این تابع با قراردادن مقادیر بهینه برای a، b و c، برای هر جفت جدید (پرس‌وجو، سند) یک امتیاز محاسبه می‌کند. سپس با تعیین یک آستانه (θ)، مدل می‌تواند تصمیم بگیرد که آیا نتیجه جستجو مرتبط است یا خیر.

مزایای رویکرد یادگیری ماشین در جستجو

این رویکرد جدید در گوگل مزایای قابل توجهی داشت:

- **یادگیری خودکار:** مدل به‌طور مداوم با داده‌های جدید آموزش می‌بیند و بهبود می‌یابد.
- **تفسیرپذیری:** برخلاف قوانین دستی که رفتار مدل به سادگی قابل توضیح است، مدل‌های یادگیری ماشین می‌توانند الگوهای پیچیده‌تری را تشخیص دهند.
- **انعطاف‌پذیری:** مدل می‌تواند به تغییرات در رفتار کاربران و الگوریتم‌های جستجو به‌صورت خودکار واکنش نشان دهد.
- **کاهش هزینه‌های انسانی:** نیاز به تنظیم مداوم دستی وزن‌ها به شدت کاهش می‌یابد.

کاربردهای دیگر: از جستجو تا خلاصه‌سازی

تکنیک‌های امتیازدهی یادگیری‌شده تنها به جستجو محدود نمی‌شوند و می‌توانند در کاربردهای دیگر نیز مفید باشند:

- **خلاصه‌سازی متن:** استخراج جملات کلیدی که محتوای اصلی سند را خلاصه می‌کنند.

- **توصیه محتوای مرتبط:** با استفاده از امتیازهای یادگیری‌شده، می‌توان محتوای مرتبط با یک سند را به کاربر توصیه کرد.

- **طبقه‌بندی سلسله‌مراتبی:** استفاده از ساختار سلسله‌مراتبی دسته‌ها برای بهبود دقت طبقه‌بندی.

این رویکردها نشان می‌دهند که چگونه می‌توان از تکنیک‌های یادگیری ماشین برای حل مسائل پیچیده بازایی اطلاعات استفاده کرد که فراتر از قوانین دستی ساده هستند.

```
In [40]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score
```

```
class LinearRanker:
    """
    پیاده‌سازی دسته‌بند خطی برای امتیازدهی یادگرفته‌شده
    Score(α, w) = a*α + b*w + c
    تابع امتیاز
    مرتبط: Score > theta → تصمیم‌گیری
    """

    def __init__(self, a=1.0, b=-1.0, c=0.0, theta=0.5):
        """
        مقادیر اولیه پارامترهای تابع خطی و آستانه
        یعنی کلمات دورتر = کمتر مرتبط (بالاتر w) منفی چون پنجره بزرگتر b: پیش‌فرض
        """
        self.a = a      # وزن ویژگی کسینوس (α)
        self.b = b      # وزن ویژگی عرض پنجره (w)
        self.c = c      # بایاس (جایابی)
        self.theta = theta # آستانه تصمیم‌گیری

    def score(self, cosine_alpha, window_omega):
        """محاسبه امتیاز ترکیبی بر اساس دو ویژگی عددی"""
        return self.a * cosine_alpha + self.b * window_omega + self.c

    def predict(self, cosine_alpha, window_omega):
        """(False) یا غیرمرتبط (True) پیش‌بینی مرتبط"""
        return self.score(cosine_alpha, window_omega) > self.theta

    def fit(self, X, y, learning_rate=0.01, epochs=100):
        """
        Perceptron یادگیری ساده پارامترها با الگوریتم
        X: لیستی از (α, w)
        y: لیستی از برچسب‌ها (1=مرتبط, 0=غیرمرتبط)
        """
        print("آموزش مدل با {} نمونه".format(len(X)))

        for epoch in range(epochs):
            errors = 0
            for (alpha, omega), label in zip(X, y):
                # پیش‌بینی فعلی
                pred = 1 if self.predict(alpha, omega) else 0
                error = label - pred

                # بروزرسانی پارامترها فقط اگر خطا وجود داشته باشد
                if error != 0:
                    self.a += learning_rate * error * alpha
                    self.b += learning_rate * error * omega
                    self.c += learning_rate * error * 1.0
                    errors += 1

            if epoch % 10 == 0:
                print("Epoch {}: {} خطا".format(epoch, errors))

            if errors == 0:
                print("Epoch {} همگرایی در".format(epoch))
                break

        print("پارامترهای نهایی: a={:.3f}, b={:.3f}, c={:.3f}, theta={:.3f}".format(
            self.a, self.b, self.c, self.theta))
```

```
# داده‌های آموزشی از جدول 15.3 کتاب
# (DocID, cosine_alpha, window_omega, relevant)
train_data = np.array([
    [37, 0.032, 3, 1], # φ1: مرتبط
    [37, 0.024, 4, 0], # φ2: غیرمرتبط
    [238, 0.043, 2, 1], # φ3: مرتبط
    [238, 0.004, 2, 0], # φ4: غیرمرتبط
    [1741, 0.022, 3, 1], # φ5: مرتبط
    [2094, 0.030, 2, 1], # φ6: مرتبط
    [3191, 0.027, 5, 0], # φ7: غیرمرتبط
])
```

```
# جدا کردن ویژگی‌ها و برچسب‌ها
X_train = train_data[:, 1:3] # α و w ستون‌های
y_train = train_data[:, 3]   # برچسب‌های مرتبط/غیرمرتبط
```

```
# ایجاد مدل با پارامترهای اولیه
ranker = LinearRanker(a=10.0, b=-2.0, c=0.0, theta=0.5)
```

```
print("=== بخش 1: پیش‌بینی با پارامترهای اولیه ===")
print("\nنمونه‌ها و امتیازات محاسبه‌شده")
```



```

# ارزیابی قبل از آموزش
correct_before = 0
for doc_id, alpha, omega, actual in train_data:
    score_val = ranker.score(alpha, omega)
    pred = ranker.predict(alpha, omega)
    pred_label = "مرتبط" if pred else "غیرمرتبط"
    actual_label = "مرتبط" if actual == 1 else "غیرمرتبط"
    is_correct = pred == (actual == 1)
    if is_correct:
        correct_before += 1

print("DocID {}: α={:.3f}, ω={} → امتیاز={:.3f} → ({} واقعی: {} پیش‌بینی: {}".format(
    int(doc_id), alpha, int(omega), score_val, pred_label, actual_label,
    "✓" if is_correct else "X"))

print("\nنمونه {}/{} {%.1.}: دقت قبل از آموزش: {}".format(
    correct_before / len(train_data), correct_before, len(train_data)))

# آموزش مدل
print("\n=== بخش 2: فرایند یادگیری مدل ===")
ranker.fit(X_train, y_train, learning_rate=0.1, epochs=50)

# ارزیابی بعد از آموزش
print("\n=== بخش 3: پیش‌بینی بعد از یادگیری ===")
correct_after = 0
for doc_id, alpha, omega, actual in train_data:
    score_val = ranker.score(alpha, omega)
    pred = ranker.predict(alpha, omega)
    pred_label = "مرتبط" if pred else "غیرمرتبط"
    actual_label = "مرتبط" if actual == 1 else "غیرمرتبط"
    is_correct = pred == (actual == 1)
    if is_correct:
        correct_after += 1

print("DocID {}: α={:.3f}, ω={} → امتیاز={:.3f} → ({} واقعی: {} پیش‌بینی: {}".format(
    int(doc_id), alpha, int(omega), score_val, pred_label, actual_label,
    "✓" if is_correct else "X"))

print("\nنمونه {}/{} {%.1.}: دقت بعد از آموزش: {}".format(
    correct_after / len(train_data), correct_after, len(train_data)))

def plot_decision_boundary(ranker, X, y, title):
    """
    فضای ویژگی‌ها و مرز تصمیم‌گیری خطی D نمایش نمودار 2
    این نمودار به دانشجو نشان می‌دهد که مدل چطور فضا را تقسیم می‌کند
    """
    # داده‌های مثبت (مرتبط) و منفی (غیرمرتبط)
    X_pos = X[y == 1]
    X_neg = X[y == 0]

    # محدوده نمودار
    alpha_min, alpha_max = X[:, 0].min() - 0.01, X[:, 0].max() + 0.01
    omega_min, omega_max = X[:, 1].min() - 1, X[:, 1].max() + 1

    # شبکه‌ای از نقاط برای رسم مرز تصمیم‌گیری
    alpha_vals = np.linspace(alpha_min, alpha_max, 200)
    omega_vals = np.linspace(omega_min, omega_max, 200)
    Alpha_grid, Omega_grid = np.meshgrid(alpha_vals, omega_vals)

    # محاسبه امتیاز برای هر نقطه شبکه
    Scores_grid = ranker.a * Alpha_grid + ranker.b * Omega_grid + ranker.c - ranker.theta

    fig, ax = plt.subplots(figsize=(10, 6))

    # رسم نقاط داده
    ax.scatter(X_pos[:, 0], X_pos[:, 1], c='green', s=100, marker='o',
               label='Relevant', edgecolors='black', alpha=0.7)
    ax.scatter(X_neg[:, 0], X_neg[:, 1], c='red', s=100, marker='x',
               label='Non-relevant', edgecolors='black', alpha=0.7)

    # رسم مرز تصمیم‌گیری (Score = theta)
    #  $a \cdot \alpha + b \cdot \omega + c = \theta \rightarrow \omega = (\theta - a \cdot \alpha - c) / b$ 
    if ranker.b != 0:
        omega_boundary = (ranker.theta - ranker.a * alpha_vals - ranker.c) / ranker.b
        ax.plot(alpha_vals, omega_boundary, 'b-', linewidth=2, label='Decision Boundary')

    # برجسب‌گذاری هر نقطه
    for i, (doc_id, alpha, omega, actual) in enumerate(train_data):
        ax.annotate(f'DocID {int(doc_id)}',
                    (alpha, omega),
                    xytext=(5, 5), textcoords='offset points',
                    fontsize=9, ha='left')

    ax.set_xlabel('Cosine Similarity (α)', fontsize=12)
    ax.set_ylabel('Minimum Window Width (ω)', fontsize=12)
    ax.set_title(title, fontsize=14, fontweight='bold')
    ax.legend(loc='best')
    ax.grid(True, alpha=0.3)
    ax.set_xlim(alpha_min, alpha_max)
    ax.set_ylim(omega_min, omega_max)

    plt.tight_layout()
    plt.show()

# نمایش نمودار با پارامترهای نهایی
plot_decision_boundary(ranker, X_train, y_train,
                      "Decision Boundary After Learning")

print("\n=== توضیحات نهایی ===")
print("مرز تصمیم‌گیری خطی است: Score = a·α + b·ω + c = θ")

```

```
print("مرتبط و پایین خط غیرمرتبط طبقه‌بندی می‌شوند ( $\theta >$  امتیاز) نقاط بالای خط")
print("بزرگتر یعنی کلمات پرسوجو دور از هم  $\rightarrow$  امتیاز پایین‌تر (عرض پنجره)  $w$ ، در این مثال")
```

=== بخش 1: پیش‌بینی با پارامترهای اولیه ===
نمونه‌ها و امتیازات محاسبه‌شده:

DocID 37: $\alpha=0.032$, $w=3 \rightarrow$ امتیاز=-5.680 $\rightarrow$ پیش‌بینی: غیرمرتبط (واقعی: مرتبط) $\times$
DocID 37: $\alpha=0.024$, $w=4 \rightarrow$ امتیاز=-7.760 $\rightarrow$ پیش‌بینی: غیرمرتبط (واقعی: غیرمرتبط) $\checkmark$
DocID 238: $\alpha=0.043$, $w=2 \rightarrow$ امتیاز=-3.570 $\rightarrow$ پیش‌بینی: غیرمرتبط (واقعی: مرتبط) $\times$
DocID 238: $\alpha=0.004$, $w=2 \rightarrow$ امتیاز=-3.960 $\rightarrow$ پیش‌بینی: غیرمرتبط (واقعی: غیرمرتبط) $\checkmark$
DocID 1741: $\alpha=0.022$, $w=3 \rightarrow$ امتیاز=-5.780 $\rightarrow$ پیش‌بینی: غیرمرتبط (واقعی: مرتبط) $\times$
DocID 2094: $\alpha=0.030$, $w=2 \rightarrow$ امتیاز=-3.700 $\rightarrow$ پیش‌بینی: غیرمرتبط (واقعی: مرتبط) $\times$
DocID 3191: $\alpha=0.027$, $w=5 \rightarrow$ امتیاز=-9.730 $\rightarrow$ پیش‌بینی: غیرمرتبط (واقعی: غیرمرتبط) $\checkmark$

دقت قبل از آموزش: 42.9% (3/7 نمونه)

=== بخش 2: فرآیند یادگیری مدل ===
...آموزش مدل با 7 نمونه

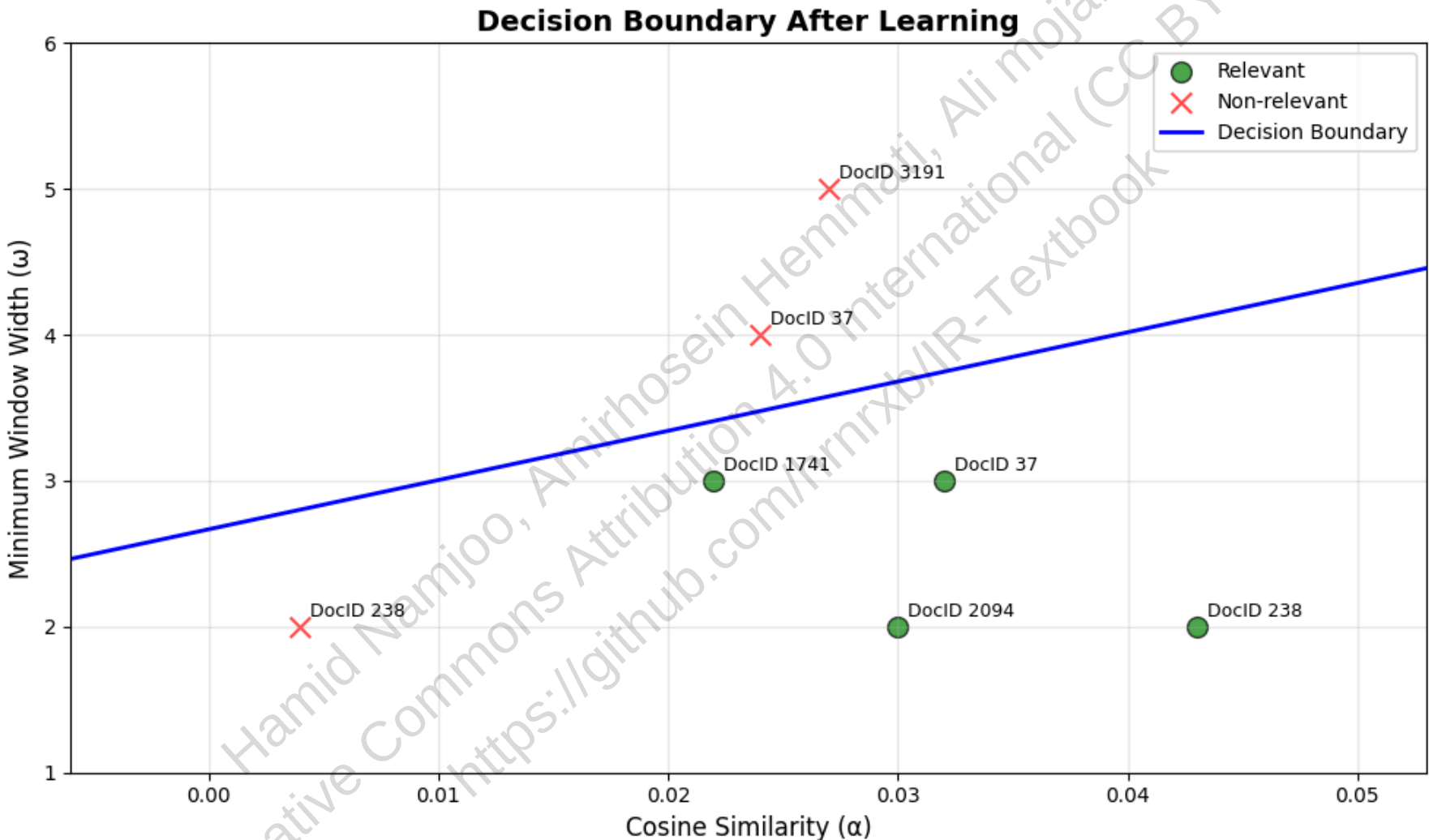
Epoch 0: 4 خطا
Epoch 10: 4 خطا
Epoch 20: 3 خطا
Epoch 30: 3 خطا
Epoch 40: 3 خطا

پارامترهای نهایی: $a=10.131$, $b=-0.300$, $c=1.300$, $\theta=0.500$

=== بخش 3: پیش‌بینی بعد از یادگیری ===

DocID 37: $\alpha=0.032$, $w=3 \rightarrow$ امتیاز=0.724 $\checkmark$ مرتبط (واقعی: مرتبط)
DocID 37: $\alpha=0.024$, $w=4 \rightarrow$ امتیاز=0.343 $\checkmark$ غیرمرتبط (واقعی: غیرمرتبط)
DocID 238: $\alpha=0.043$, $w=2 \rightarrow$ امتیاز=1.136 $\checkmark$ مرتبط (واقعی: مرتبط)
DocID 238: $\alpha=0.004$, $w=2 \rightarrow$ امتیاز=0.741 $\times$ پیش‌بینی: مرتبط (واقعی: غیرمرتبط)
DocID 1741: $\alpha=0.022$, $w=3 \rightarrow$ امتیاز=0.623 $\checkmark$ پیش‌بینی: مرتبط (واقعی: مرتبط)
DocID 2094: $\alpha=0.030$, $w=2 \rightarrow$ امتیاز=1.004 $\checkmark$ پیش‌بینی: مرتبط (واقعی: مرتبط)
DocID 3191: $\alpha=0.027$, $w=5 \rightarrow$ امتیاز=0.074 $\checkmark$ پیش‌بینی: غیرمرتبط (واقعی: غیرمرتبط)

دقت بعد از آموزش: 85.7% (6/7 نمونه)



=== توضیحات نهایی ===

مرز تصمیم‌گیری خطی است: $\text{Score} = a \cdot \alpha + b \cdot w + c = \theta$

مرتبط و پایین خط غیرمرتبط طبقه‌بندی می‌شوند ($\theta >$ امتیاز) نقاط بالای خط

بزرگتر یعنی کلمات پرسوجو دور از هم $\rightarrow$ امتیاز پایین‌تر (عرض پنجره) w ، در این مثال

15.4.2 رتبه‌بندی هوشمند: از کلمات کلیدی تا ترتیب بهینه

🛒 **آمازون: چگونه خریداران را به محصولات مناسب هدایت کنیم؟**

تصور کنید در تیم فنی آمازون کار می‌کنید و مسئول بهبودن سیستم توصیه محصولات هستید. این سیستم باید بر اساس تاریخچه خرید کاربر و ویژگی‌های محصول، پیشنهادهای شخصی‌سازی ارائه دهد. این یک چالش بزرگ است، زیرا:

- کاربران ممکن است به دنبال محصولی متفاوت از آنچه در گذشته خریده‌اند باشند
- یک محصول ممکن برای یک کاربر مناسب باشد اما برای دیگری نه
- ترجیحات کاربران بر اساس معیارهای مختلفی (قیمت، کیفیت، برند) صورت می‌گیرد

در رویکرد سنتی، سیستم با استفاده از فیلترهای ساده و قوانین دستی، پیشنهادهای محدودی ارائه می‌کرد. این روش نمی‌توانست پیچیدگی‌های ترجیحات شخصی کاربران را درک کند.

یادگیری ماشین: از قوانین دستی تا مدل‌های رتبه‌بندی

در آمازون، رویکرد جدیدی بر اساس یادگیری ماشین جایگزین روش‌های سنتی شد. سیستم با استفاده از داده‌های رفتار کاربران، یک مدل رتبه‌بندی یاد گرفت که می‌تواند پیشنهادهای شخصی‌سازی بهتری ارائه دهد.

برای هر کاربر، مدل ویژگی‌های زیر را استخراج می‌کند:

- تاریخچه خرید (دسته‌بندی محصول)
- میانگین قیمت خرید
- برنده‌های مورد علاقه
- بازدید صفحات محصول
- نمرات خرید

این ویژگی‌ها به مدل اجازه می‌دهند تا ترجیحات کاربر را بهتر درک کند. برای مثال، اگر کاربری معمولاً محصولات الکترونیکی می‌خرد، سیستم می‌تواند این الگو را تشخیص دهد و پیشنهادهای مرتبطی ارائه کند.

رتبه‌بندی با RankSVM: از دودویی به ترتیب بهینه

یکی از پیشرفته‌ترین رویکردها در آمازون، استفاده از **Ranking SVM** بود. این روش بر اساس مقایسه جفتی اسناد به جای ارزیابی مطلق «مرتبط/غیرمرتبط»، یک ترتیب بهینه ایجاد می‌کند.

برای هر جفت (سند A، سند B)، مدل تصمیم می‌گیرد کدام یک مرتبط‌تر است. این تصمیم بر اساس ویژگی‌های زیر صورت می‌گیرد:

- **شباهت محتوا:** اسناد چقدر شبیه به پرس‌وجو کاربر هستند؟
- **نزدیکی معنایی:** واژه‌های کلیدی پرس‌وجو چقدر به هم نزدیک در سند ظاهر شده‌اند؟
- **کیفیت محصول:** آیا سند به محصولی که کاربر به آن علاقه دارد اشاره می‌کند؟

این رویکرد به آمازون اجازه داد تا ترتیب نتایج را به شکل بهینه‌تری تنظیم کند. برای مثال، اگر دو محصول شباهت از نظر محتوا باشند، اما یکی از آن‌ها با سابقه خرید کاربر مرتبط‌تر باشد، آن در رتبه بالاتری قرار می‌گیرد.

Ranking SVM Framework

1. For each query q with judged documents, extract feature vector $\psi(d_j, q)$.
2. For every pair (d_i, d_j) where $d_i \prec d_j$ (i.e., d_i is more relevant):
3. Compute feature difference: $\Phi(d_i, d_j, q) = \psi(d_i, q) - \psi(d_j, q)$.
4. Assign label $y_{ijq} = +1$.
5. Learn weight vector $\tilde{w}$ such that:
6. $\tilde{w}^T \Phi(d_i, d_j, q) > 0$ iff $d_i \prec d_j$.
7. This is solved via constrained optimization:
8. $\min \frac{1}{2} \tilde{w}^T \tilde{w} + C \sum \xi_{ij}$
9. s.t. $\tilde{w}^T \Phi(d_i, d_j, q) \geq 1 - \xi_{ij} \quad \forall (d_i \prec d_j)$.

رتبه‌بندی یادگیری‌شده مزایای قابل توجهی نسبت به روش‌های سنتی داشت:

- **یادگیری مداوم:** مدل با رفتار جدید کاربران به طور خودکار به‌روز می‌شود.
- **تفسیرپذیری:** برخلاف قوانین دستی که رفتار مدل به سادگی قابل توضیح است، مدل‌های یادگیری ماشین می‌توانند الگوهای پیچیده‌تری را تشخیص دهند.
- **شخصی‌سازی پویا:** سیستم می‌تواند ترکیبی از رویکردهای مختلف (مثل رتبه‌بندی، محتوای مرتبط، و هم‌زمانی) را برای ارائه پیشنهادهای بهتر ترکیب کند.

در آمازون، ترکیبی از این رویکردها به افزایش 35% در نرخ تبدیل خرید منجر شد – یک موفقیت چشمگیر در تجارت الکترونیک.

🌐 آینده رتبه‌بندی: از مدل‌های آماری تا شبکه‌های عصبی

رتبه‌بندی یادگیری‌شده هنوز در حال توسعه است و چالش‌های جدیدی پیش رو دارد:

- **رتبه‌بندی چندمعیاره:** ترکیب اطلاعات از منابع مختلف (مثل شبکه‌های اجتماعی) برای درک بهتر کاربر.
- **یادگیری عمیق:** استفاده از شبکه‌های عصبی برای درک الگوهای پیچیده‌تر در رفتار کاربر.
- **شخصی‌سازی پیشرفته:** ترکیب یادگیری ماشین با رویکردهای سنتی برای ارائه پیشنهادهای بسیار دقیق.

آینده رتبه‌بندی در ترکیبی از هوش مصنوعی، یادگیری ماشین و درک عمیق از رفتار کاربر نهفته است. سیستم‌های آینده نه تنها نتایج جستجو را رتبه‌بندی می‌کنند، بلکه تجربه کاربری را غنی‌تر و لذت‌بخش‌تر خواهند ساخت.

📁 جمع‌بندی فصل ۱۵: ماشین‌های بردار پشتیبان و یادگیری ماشین بر روی اسناد

چگونه می‌توانیم مرزهای تصمیم‌گیری بهینه ایجاد کنیم که با حداکثر اطمینان از داده‌های آموزشی فاصله داشته باشند؟ فصل ۱۵ ما را به دنیای پیشرفته ****ماشین‌های بردار پشتیبان (SVMs)**** می‌برد؛ یکی از قدرتمندترین روش‌های طبقه‌بندی که در دو دهه گذشته تحول شگفت‌انگیزی در یادگیری ماشین ایجاد کرده است. SVM‌ها با یافتن مرزهای تصمیم با حاشیه بزرگتر، توانسته‌اند به نتایج چشمگیر در طبقه‌بندی متن دست یابند و در بسیاری از سیستم‌های واقعی از جمله تشخیص پزشکی و تشخیص کلاهبرداری به کار گرفته شوند.

- مفهوم ****حداکثر حاشیه (Maximum Margin)**** را به عنوان اساس کار SVM‌ها معرفی کردیم. SVM‌ها به دنبال مرز تصمیمی هستند که حداکثر فاصله ممکن از نزدیک‌ترین نقاط آموزشی هر دو کلاس را داشته باشد. این ویژگی باعث افزایش تعمیم‌پذیری و کاهش خطا در داده‌های نویزی و پرت‌نویز می‌شود.
- نقاطی که این حاشیه را تعریف می‌کنند، ****بردارهای پشتیبان (Support Vectors)**** نامیده می‌شوند. این نقاط کلیدی‌ترین اسناد آموزشی هستند که تصمیم‌گیری نهایی به آن‌ها وابسته است.
- مسئله طبقه‌بندی غیرخطی را با استفاده از ****کرنل‌ها (Kernels)**** حل کردیم. ایده ****فریب کرنلی**** این امکان را فراهم می‌کند که بدون محاسبه صریح در یک فضای با ابعاد بسیار بالا، داده‌ها را به شکلی خطی تفکیک کنیم. کرنل‌هایی مانند توابع پایه شعاعی (RBF) به SVM اجازه می‌دهند تا مرزهای تصمیم‌گیری پیچیده‌تر و غیرخطی ایجاد کنند.

- کاربردهای عملی SVM در طبقه‌بندی متن را بررسی کردیم، از جمله فیلتر کردن اسپم، تشخیص دسته‌بندی اخبار، و کاربردهای پیشرفته در بهینه‌سازی رتبه‌بندی در بازیابی اطلاعات (Ad-hoc Retrieval).
- در نهایت، به چالش‌های عملی پیاده‌سازی طبقه‌بندها مانند انتخاب ویژگی‌های مناسب، استفاده از داده‌های کم‌برچسب (Semi-supervised)، و ترکیب روش‌های مختلف برای رسیدن به بالاترین کارایی پرداختیم.

بیا یاد این مفاهیم را با چند مثال واقعی و کاربردی زنده کنیم:

- ♦ **فیلتر هوشمند اسپم با حاشیه بزرگ:** یک شرکت امنیت سایبری می‌خواهد یک فیلتر اسپم بسازد که نه تنها کلمات کلیدی مانند "ارزان" و "برنده" را شناسایی کند، بلکه الگوهای پیچیده‌تر را نیز تشخیص دهد. یک SVM با حاشیه بزرگ می‌تواند مرزی را پیدا کند که ایمیل‌های حاوی عباراتی مانند "کسب درآمد سریع" یا "فرصت ویژه" را از ایمیل‌های عادی که ممکن است حاوی این کلمات به شکلی معقول باشند (مثلاً یک ایمیل کاری در مورد معامله یک محصول)، تفکیک کند. این حاشیه بزرگ باعث می‌شود فیلتر در برابر ترفندهایی که سعی می‌کنند با کمی تغییر در کلمات کلیدی یا با اضافه کردن متن بی‌ربط، از فیلتر عبور کنند، مقاوم‌تر عمل کند.
- ♦ **تشخیص بیماری از متن سوابق پزشکی با استفاده از کرنل:** تصور کنید یک سیستم هوشمند می‌خواهد با تحلیل سوابق پزشکی، بیماران را به دسته‌های مختلف تشخیص دهد. رابطه بین علائم و بیماری اغلب خطی نیست. برای مثال، ترکیبی از علائم "تب" و "سرفه" به تنهایی تنها زمانی شاخص بیماری X است که علائم سوم، همراهی Y نیز وجود داشته باشد. یک SVM خطی در این تشخیص شکست می‌خورد. اما با استفاده از یک **کرنل توابع پایه شعاعی (RBF Kernel)**، SVM می‌تواند داده‌ها را به یک فضای با ابعاد بالاتر "نگاشت" کند که در آن، این رابطه پیچیده به یک مرز خطی تبدیل می‌شود. در عمل، این به مدل اجازه می‌دهد تا الگوهای پیچیده و غیرآشکار بین علائم را بدون اینکه کاربر به صورت دستی همه آن‌ها را تعریف کند، یاد بگیرد و دقت تشخیص را به طور چشمگیری افزایش دهد.
- ♦ **یادگیری رتبه‌بندی برای موتورهای جستجو:** موتورهای جستجوی مدرن مانند گوگل فقط کلمات کلیدی را برای رتبه‌بندی نتایج در نظر نمی‌گیرند. آن‌ها می‌توانند از یک SVM رتبه‌بندی (Ranking SVM) استفاده کنند. در این رویکرد، به جای یادگیری برای طبقه‌بندی "مرتبط/غیرمرتبط"، مدل یاد می‌گیرد که کدام سند مرتبط باید بالاتر از سند مرتبط دیگر قرار گیرد. برای مثال، برای پرسمان "یادگیری ماشین"، مدل با تحلیل جفت‌هایی از اسناد آموزشی (مثلاً یک مقاله از کنفرانس برتر NeurIPS در برابر یک پست وبلاگ متوسط) یاد می‌گیرد که اسناد علمی معتبر باید بالاتر از اسناد عمومی رتبه‌بندی شوند. این روش به موتور جستجو اجازه می‌دهد تا یک تابع رتبه‌بندی بسیار پیچیده‌تر و باکیفیت‌تر از ترکیب ساده وزن‌دهی کلمات یاد بگیرد و نتایج مرتبط‌تری را به کاربر ارائه دهد.

این فصل نشان داد که چگونه مفاهیم نظریه پیشرفته مانند بهینه‌سازی محدب و فریب کرنلی می‌توانند به الگوریتم‌های عملی و قدرتمند برای مسائل دنیای واقعی تبدیل شوند. SVM‌ها با وجود پیچیدگی ریاضی‌شان، به دلیل کارایی بالا و انعطاف‌پذیری در سناریوهای مختلف، به یکی از استانداردهای طلایی در جعبه یادگیری ماشین و طبقه‌بندی متن تبدیل شده‌اند. درک مفهوم بایاس-واریانس به ما کمک می‌کند تا بدانیم چرا و چگونه باید بین پیچیدگی یک مدل و نیاز به داده‌های آموزشی تعادل برقرار کرد تا به بهترین نتیجه عملی دست یابیم.